

LineScope

A Literate Program for Studying
Resampling and Filtering of Images

`pfiguero@uniandes.edu.co`

September 24, 2026

Contents

1	Introduction	1
1.1	How to read this document	1
1.2	The shape of the program	2
1.3	Building	2
2	Pixels	4
2.1	The representation	4
2.2	Construction and access	5
2.3	Premultiplication	6
2.4	Bridging to Core Graphics	7
3	Resampling	9
3.1	Theory in brief	9
3.2	The methods	10
3.3	Entry points	11
3.3.1	The two passes	12
3.4	Weights	13
3.5	Kernels	15
4	Filters and Operations	17
4.1	The filters	17
4.1.1	Dispatch	18
4.1.2	The Core Image bridge	20
4.1.3	CPU filters	21
4.2	Operations	22
5	Lines, Colors and Spectra	24
5.1	The selection line	24
5.2	Sampling along the line	25
5.3	Color models	27
5.3.1	HSL	28
5.3.2	CMYK	29
5.4	The spectrum	30
6	Test Patterns	34
6.1	The catalog	34
6.2	Specifications	36
6.3	The generator	36
6.3.1	Chirps and gratings	37
6.3.2	Edges and impulses	38
6.3.3	The Siemens star	39
6.3.4	Noise and color	39

6.4	SplitMix64	39
7	Tests	41
7.1	Spectrum	42
7.2	Resampling	42
7.3	Color conversions	44
7.4	Line sampling	44
7.5	Core Graphics and filters	45
7.6	Test patterns	46
8	Documents and the Session Model	47
8.1	The app and its scenes	47
8.2	The image document	48
8.3	Variants, zones and panes	51
8.4	The session	53
8.4.1	State	54
8.4.2	Panes	56
8.4.3	Variants and operations	56
8.4.4	Analysis	57
8.4.5	Export	58
8.4.6	Finding the session from outside the window	59
9	Views	60
9.1	The document window	60
9.2	Zones	63
9.3	Panes and tabs	65
9.4	The image canvas	68
9.5	Charts	72
9.6	The parameter sheet	74
9.7	The inspector	77
9.8	Test-pattern windows	79
10	Menus	84
10.1	Which document does a menu act on?	84
10.2	The File menu	85
10.3	Filters and resampling	85
10.4	The View menu	86
	Bibliography	89
	Index of Chunks	90

Chapter 1

Introduction

LineScope is a small macOS program for *looking* at what resampling and filtering do to an image. You open an image, derive copies from it (blurred, sharpened, shrunk with nearest-neighbor, enlarged with Lanczos, and so on), and draw a line across it. For every copy the program shows three things side by side:

1. the image itself, at its true pixel scale, with the line drawn on it;
2. the *profile*: one color channel sampled along the line;
3. the *spectrum*: the Fourier transform of that profile.

Because the line is shared, moving it in one copy moves it in all of them, and the differences between the methods become visible at once. Aliasing shows as false energy below the Nyquist frequency, over-smoothing as a spectrum that dies too early, and ringing as ripples beside an edge.

1.1 How to read this document

This is a *literate program* in the sense of Donald Knuth [1]: a document written for people, from which the program for the machine is extracted. The code appears in *chunks*. Each chunk has a name and a number, and is written

$$\langle name\ n \rangle \equiv$$

where it is defined. A chunk may use other chunks by name; that is how a long function is explained a piece at a time, in the order that makes the ideas clear rather than the order the compiler wants. When the same name is defined again, the new text is appended to the old, and the header reads $+ \equiv$. Below each chunk, a note tells where it is continued and where it is used.

The chunk index at the end of the book lists every chunk.

Two tools turn this document into something useful:

Tangling

follows the references from each *root* chunk and writes the source file the chunk is named after. The roots of this web are exactly the source files of the project, so tangling rebuilds LineScope from this document.

Weaving

produces the L^AT_EX you are reading, with the cross-references.

The directory LiterateP holds the web (chapters/*.nw), the tool `lit.py` that does both jobs using Norman Ramsey's noweb notation [2], and a Makefile. Running `make check` tangles the web and compares every file, byte for byte, with the real sources in the repository. So this document can't drift away from the program without the check noticing.

1.2 The shape of the program

The code is split in two.

The first part is `LineScopeCore`, a Swift package with no user interface. It holds everything numeric: the pixel buffer (Chapter 2), resampling (Chapter 3), filters (Chapter 4), line sampling, color models and the spectrum (Chapter 5), and generated test patterns (Chapter 6). It is tested on its own (Chapter 7).

The second part is the app, `LineScope`. It holds the document and the per-window session model (Chapter 8), the SwiftUI views (Chapter 9) and the menus (Chapter 10).

This split keeps the mathematics testable without a window, and keeps the views thin: a view asks the session for a profile or a spectrum and draws it.

1.3 Building

The Xcode project isn't kept in the repository. XcodeGen [6] generates it from a short specification. The app target depends on the local package, is a document *viewer* for any image type, and is signed ad hoc so it runs without a developer account.

```
1  <project.yml 1> ≡
  name: LineScope
  options:
    bundleIdPrefix: co.edu.uniandes
    deploymentTarget:
      macOS: "15.0"
  packages:
    LineScopeCore:
      path: Packages/LineScopeCore
  targets:
    LineScope:
      type: application
      platform: macOS
      sources: [LineScope]
      dependencies:
        - package: LineScopeCore
      info:
        path: LineScope/Info.plist
        properties:
          CFBundleDisplayName: LineScope
          LSApplicationCategoryType: public.app-category.graphics-design
          NSHumanReadableCopyright: ""
          CFBundleDocumentTypes:
            - CFBundleTypeName: Image
              CFBundleTypeRole: Viewer
              LSHandlerRank: Alternate
              LSItemContentTypes: [public.image]
      settings:
        base:
          PRODUCT_BUNDLE_IDENTIFIER: co.edu.uniandes.LineScope
          SWIFT_VERSION: "5.0"
          MARKETING_VERSION: "0.1"
          CURRENT_PROJECT_VERSION: "1"
          CODE_SIGN_IDENTITY: "-"
          CODE_SIGN_STYLE: Manual
          ENABLE_HARDENED_RUNTIME: NO
          GENERATE_INFOPLIST_FILE: NO
```

Root chunk, tangled to project.yml.

The rank `Alternate` means `LineScope` offers to open images but never claims to be their default application.

The package manifest declares one library and its tests. One line deserves comment. Swift in a Debug build is compiled without optimization, and the inner loops of resampling and filtering then run tens of times slower, since every array access is bounds-checked and nothing is inlined. The manifest therefore asks for `-O` even in Debug. This uses `unsafeFlags`, which the Swift Package Manager allows only for local packages like this one.

2 $\langle$ Packages/LineScopeCore/Package.swift 2 $\rangle \equiv$

```
// swift-tools-version: 5.10
import PackageDescription

let package = Package(
    name: "LineScopeCore",
    platforms: [.macOS(.v14)],
    products: [
        .library(name: "LineScopeCore", targets: ["LineScopeCore"]),
    ],
    targets: [
        .target(
            name: "LineScopeCore",
            // Pixel loops are unusably slow at -Onone; optimize even in Debug.
            swiftSettings: [.unsafeFlags(["-O"], .when(configuration: .debug))]
        ),
        .testTarget(
            name: "LineScopeCoreTests",
            dependencies: ["LineScopeCore"]
        ),
    ]
)
```

Root chunk, tangled to Packages/LineScopeCore/Package.swift.

With these two files in place, the commands are:

```
xcodegen generate
xcodebuild -project LineScope.xcodeproj -scheme LineScope build
cd Packages/LineScopeCore && swift test
```

Chapter 2

Pixels

Every numeric part of LineScope works on one image representation, the `PixelBuffer`. Choosing it well settles a surprising number of later questions, so this chapter is short on code but long on decisions.

2.1 The representation

A buffer stores its pixels *interleaved*: the four components of a pixel sit next to each other, and pixels follow one another row by row, starting with the *top* row. The component of channel $c \in \{0, 1, 2, 3\}$ (red, green, blue, alpha) at column x and row y of a buffer of width W therefore lives at index

$$4(yW + x) + c.$$

Row 0 at the top matches the way images are stored in files and drawn on the screen, so no flipping is ever needed between the buffer, Core Graphics bitmaps and the views.

Components are 32-bit floats in $[0, 1]$. Floats let a filter produce intermediate values (negative lobes of a Lanczos kernel, overshoot of a sharpening filter) without wrapping around, and they make the arithmetic in resampling exact enough that the tests can compare against closed-form answers.

Two further conventions matter.

Encoded, not linear. The components are the sRGB-*encoded* values, the numbers stored in the file and sent to the display, not the linear-light intensities obtained by undoing the sRGB transfer curve. A physically careful resampler would linearize first; LineScope deliberately does not, because its purpose is to show the profile of the values a user sees and that most software filters operate on. Everything downstream (profiles, spectra, filters) sees the same encoded numbers.

Straight alpha. Colors are stored with *straight* (non-premultiplied) alpha, so a half-transparent red pixel is $(1, 0, 0, \frac{1}{2})$ rather than $(\frac{1}{2}, 0, 0, \frac{1}{2})$. Straight alpha is what a user expects to read in a profile. Filtering, however, must be done on premultiplied values $(\alpha r, \alpha g, \alpha b, \alpha)$; otherwise a fully transparent pixel, whose color is arbitrary, would bleed its color into its neighbors. The buffer therefore offers conversions in both directions (Section 2.3).

The file has two parts: the types themselves, and the bridge to Core Graphics.

3 $\langle \text{Packages/LineScopeCore/Sources/LineScopeCore/PixelBuffer.swift } 3 \rangle \equiv$

```
import CoreGraphics
import Foundation
```

$\langle \text{PixelBuffer: the color sample RGBA } 4 \rangle$

$\langle \text{PixelBuffer: the buffer type } 5 \rangle$

// MARK: - Core Graphics bridging

```
extension PixelBuffer {
     $\langle \text{PixelBuffer: the sRGB color space } 9 \rangle$ 

     $\langle \text{PixelBuffer: decoding a CGImage } 10 \rangle$ 

     $\langle \text{PixelBuffer: encoding a CGImage } 11 \rangle$ 
}
```

Root chunk, tangled to Packages/LineScopeCore/Sources/LineScopeCore/PixelBuffer.swift.

A single color sample is a small value type. It is what a line sampler returns and what the color conversions of Chapter 5 consume. Alpha defaults to opaque, because most samples are.

4 $\langle \text{PixelBuffer: the color sample RGBA } 4 \rangle \equiv$

/// A color sample with straight (non-premultiplied) alpha, sRGB-encoded components in 0...1.

```
public struct RGBA: Sendable, Equatable {
    public var r: Float
    public var g: Float
    public var b: Float
    public var a: Float

    public init(r: Float, g: Float, b: Float, a: Float = 1) {
        self.r = r
        self.g = g
        self.b = b
        self.a = a
    }
}
```

Used in chunk 3.

The buffer is a struct holding its dimensions and one flat array. Swift arrays are copy-on-write, so passing a buffer around (to a background task, into an `ImageVariant`) costs nothing until someone mutates it. The type is `Sendable`, which lets it cross from the main actor to a detached task and back, and `Equatable`, which the tests use to compare whole images.

5 $\langle \text{PixelBuffer: the buffer type } 5 \rangle \equiv$

/// Interleaved RGBA Float32 image. Row 0 is the top row.

/// Components are sRGB-encoded (not linearized) with straight alpha, so values match what is displayed.

```
public struct PixelBuffer: Sendable, Equatable {
     $\langle \text{PixelBuffer: stored properties and initializers } 6 \rangle$ 

     $\langle \text{PixelBuffer: pixel subscript } 7 \rangle$ 

     $\langle \text{PixelBuffer: premultiplication } 8 \rangle$ 
}
```

Used in chunk 3.

2.2 Construction and access

The designated initializer takes the raw array and checks the one invariant everything else relies on: exactly four floats per pixel, and at least one pixel. A buffer with zero width would make the

resampler divide by zero, so it is rejected at the door. The second initializer fills the image with one color; the tests and the test-pattern generator start from it.

6 *⟨PixelBuffer: stored properties and initializers 6⟩ ≡*

```
public let width: Int
public let height: Int
public var data: [Float]

public init(width: Int, height: Int, data: [Float]) {
    precondition(width > 0 && height > 0 && data.count == width * height * 4)
    self.width = width
    self.height = height
    self.data = data
}

public init(width: Int, height: Int, fill: RGBA = RGBA(r: 0, g: 0, b: 0, a: 1)) {
    var data = [Float](repeating: 0, count: width * height * 4)
    for i in stride(from: 0, to: data.count, by: 4) {
        data[i] = fill.r
        data[i + 1] = fill.g
        data[i + 2] = fill.b
        data[i + 3] = fill.a
    }
    self.init(width: width, height: height, data: data)
}
```

Used in chunk 5.

A subscript reads and writes whole pixels using the index formula above. It is convenient rather than fast: the heavy loops in resampling and filtering index `data` directly through unsafe buffer pointers instead.

7 *⟨PixelBuffer: pixel subscript 7⟩ ≡*

```
public subscript(x: Int, y: Int) -> RGBA {
    get {
        let i = (y * width + x) * 4
        return RGBA(r: data[i], g: data[i + 1], b: data[i + 2], a: data[i + 3])
    }
    set {
        let i = (y * width + x) * 4
        data[i] = newValue.r
        data[i + 1] = newValue.g
        data[i + 2] = newValue.b
        data[i + 3] = newValue.a
    }
}
```

Used in chunk 5.

2.3 Premultiplication

To premultiply, scale the three color components of every pixel by its alpha. The result is a bare array, not a `PixelBuffer`, on purpose: premultiplied data is an internal intermediate of the resampler and of the Core Image bridge, and giving it its own type would invite someone to plot it.

Going back divides by alpha. Two safeguards make this robust against what filters produce. Alpha is first clamped to $[0, 1]$, since a Lanczos kernel can overshoot it. Where alpha is (nearly) zero the color is undefined, and dividing would amplify rounding noise into bright garbage; such pixels become transparent black. The color components are clamped too, which is where the overshoot and undershoot of sharpening kernels is finally cut off.

8 *⟨PixelBuffer: premultiplication 8⟩* ≡

// MARK: Premultiplication (used by resampling and Core Image, which expect premultiplied data)

```
func premultiplied() -> [Float] {
    var out = data
    for i in stride(from: 0, to: out.count, by: 4) {
        let a = out[i + 3]
        out[i] *= a
        out[i + 1] *= a
        out[i + 2] *= a
    }
    return out
}

static func fromPremultiplied(width: Int, height: Int, _ p: [Float]) -> PixelBuffer {
    var out = p
    for i in stride(from: 0, to: out.count, by: 4) {
        let a = min(max(out[i + 3], 0), 1)
        out[i + 3] = a
        if a > 1e-6 {
            out[i] = min(max(out[i] / a, 0), 1)
            out[i + 1] = min(max(out[i + 1] / a, 0), 1)
            out[i + 2] = min(max(out[i + 2] / a, 0), 1)
        } else {
            out[i] = 0
            out[i + 1] = 0
            out[i + 2] = 0
        }
    }
    return PixelBuffer(width: width, height: height, data: out)
}
```

Used in chunk 5.

2.4 Bridging to Core Graphics

Images enter the program as `CGImage` values decoded by `ImageIO`, and they leave it as `CGImage` values drawn by `SwiftUI` or written to PNG. Both directions go through the sRGB color space, created once.

9 *⟨PixelBuffer: the sRGB color space 9⟩* ≡

```
static let sRGB = CGColorSpace(name: CGColorSpace.sRGB)!
```

Used in chunk 3.

A decoded image can have any layout: 16 bits per component, CMYK, a grayscale palette, alpha first or last. Rather than handling each case, the decoder lets Core Graphics do the conversion: it creates an 8-bit premultiplied RGBA bitmap in sRGB and draws the image into it. Interpolation is switched off; the image is drawn at its own size, so no resampling should happen at all, and this makes sure of it. The bitmap's memory is borrowed from a Swift array for the duration of the closure, and the bytes are then scaled to $[0, 1]$ and un-premultiplied.

This does lose precision for 16-bit sources. The inspector reports the source's real bit depth, so the user knows when that happened.

10 *⟨PixelBuffer: decoding a CGImage 10⟩* ≡

/// Decodes any CGImage into 8-bit sRGB and converts it to a float buffer.

```
public init?(cgImage: CGImage) {
    let w = cgImage.width
    let h = cgImage.height
    guard w > 0, h > 0 else { return nil }
    var bytes = [UInt8](repeating: 0, count: w * h * 4)
```

```

let drawn = bytes.withUnsafeMutableBytes { raw -> Bool in
    guard let ctx = CGContext(
        data: raw.baseAddress, width: w, height: h, bitsPerComponent: 8, bytesPerRow: w * 4,
        space: PixelBuffer.sRGB, bitmapInfo: CGImageAlphaInfo.premultipliedLast.rawValue
    ) else { return false }
    ctx.interpolationQuality = .none
    ctx.draw(cgImage, in: CGRect(x: 0, y: 0, width: w, height: h))
    return true
}
guard drawn else { return nil }
var p = [Float](repeating: 0, count: w * h * 4)
for i in 0..

```

Used in chunk 3.

The opposite direction clamps, premultiplies and rounds each component to a byte, and wraps the bytes in a data provider. The image is marked `shouldInterpolate: false` so that when SwiftUI draws it, nothing smooths the pixels behind the program's back. Seeing the blocky result of nearest-neighbor upsampling is part of the point.

11 *<PixelBuffer: encoding a CGImage 11> ≡*

```

/// Renders the buffer as an 8-bit premultiplied sRGB CGImage for display or export.
public func makeCGImage() -> CGImage? {
    var bytes = [UInt8](repeating: 0, count: data.count)
    for i in stride(from: 0, to: data.count, by: 4) {
        let a = min(max(data[i + 3], 0), 1)
        bytes[i] = UInt8((min(max(data[i], 0), 1) * a * 255).rounded())
        bytes[i + 1] = UInt8((min(max(data[i + 1], 0), 1) * a * 255).rounded())
        bytes[i + 2] = UInt8((min(max(data[i + 2], 0), 1) * a * 255).rounded())
        bytes[i + 3] = UInt8((a * 255).rounded())
    }
    guard let provider = CGDataProvider(data: Data(bytes) as CFData) else { return nil }
    return CGImage(
        width: width, height: height, bitsPerComponent: 8, bitsPerPixel: 32, bytesPerRow: width * 4,
        space: PixelBuffer.sRGB, bitmapInfo: CGBitmapInfo(rawValue:
            ↪ CGImageAlphaInfo.premultipliedLast.rawValue),
        provider: provider, decode: nil, shouldInterpolate: false, intent: .defaultIntent
    )
}

```

Used in chunk 3.

Chapter 3

Resampling

Resampling changes the size of an image. It is the operation LineScope was written to study, so its implementation puts clarity and exactness first. Each menu entry is *exactly* the textbook algorithm it names. Library routines (vImage, Core Image) choose their own kernels, prefilters and edge rules, and that would blur the comparison the user is trying to make.

3.1 Theory in brief

A digital image is a set of samples of a continuous picture. To resample it to a new grid we (conceptually) reconstruct the continuous picture by convolving the samples with a *kernel* k , then sample the result at the new positions. In one dimension, output pixel i at position u_i in input coordinates is

$$\text{out}_i = \frac{\sum_j k(j + \frac{1}{2} - u_i) \text{in}_j}{\sum_j k(j + \frac{1}{2} - u_i)}.$$

The denominator renormalizes the weights, so a constant image stays constant even where the kernel is cut off at the image border.

Pixel centers. Input pixel j covers the interval $[j, j+1)$ and has its center at $j + \frac{1}{2}$. With scale $s = n_{\text{out}}/n_{\text{in}}$, output pixel i covers $[i, i+1)$ in output coordinates, and its center maps back to

$$u_i = \frac{i + \frac{1}{2}}{s}$$

in input coordinates. This convention keeps the image centered: halving an image of width 4 puts the two output centers at input positions 1 and 3, exactly between the input centers 0.5, 1.5 and 2.5, 3.5. Using $u_i = i/s$ instead would shift the whole image by half a pixel, a classic bug that shows up as asymmetric profiles.

Downscaling needs a prefilter. When $s < 1$ the new grid cannot represent frequencies above its own Nyquist limit, $s/2$ cycles per input pixel. A reconstruction kernel of fixed width lets those frequencies through, and they fold back as aliases. The standard cure is to stretch the kernel by $1/s$, which turns it into a low-pass filter matched to the new grid:

$$k_s(x) = k(sx), \quad \text{with support widened by } 1/s.$$

The normalization above absorbs the factor s that a properly scaled filter would carry. Nearest neighbor is kept *without* this widening, so that the menu offers a method that aliases. Seeing aliasing is instructive.

Separability. All kernels here are applied separably, first along rows, then along columns: $k(x, y) = k(x)k(y)$. That turns an $O(r^2)$ cost per pixel into $O(r)$ and lets both passes share one routine for computing weights.

3.2 The methods

The file holds the list of methods, and the resampler that implements them.

12 *⟨Packages/LineScopeCore/Sources/LineScopeCore/Resampling.swift 12⟩* ≡
`import Foundation`

⟨Resampling: the ResampleMethod enumeration 13⟩

⟨Resampling: the Resampler namespace 14⟩

Root chunk, tangled to Packages/LineScopeCore/Sources/LineScopeCore/Resampling.swift.

The enumeration is what the Resample menu iterates over. Each method has a long name for the menu and a short one for tab titles, where a chain like “Tent $\times 2 \rightarrow$ Mitchell $\times 0.5$ ” must stay readable. CaseIterable lets the menu stay in sync with this list automatically.

13 *⟨Resampling: the ResampleMethod enumeration 13⟩* ≡

```
public enum ResampleMethod: String, CaseIterable, Identifiable, Sendable {
    case nearest
    case tent
    case catmullRom
    case mitchell
    case lanczos3
    case sinc
    case area

    public var id: String { rawValue }

    public var displayName: String {
        switch self {
        case .nearest: "Nearest Neighbor"
        case .tent: "Tent (Bilinear)"
        case .catmullRom: "Catmull-Rom (Bicubic)"
        case .mitchell: "Mitchell-Netravali"
        case .lanczos3: "Lanczos-3"
        case .sinc: "Sinc (truncated, r=\(Int(Resampler.sincRadius)))"
        case .area: "Area Average"
        }
    }

    public var shortName: String {
        switch self {
        case .nearest: "Nearest"
        case .tent: "Tent"
        case .catmullRom: "Catmull-Rom"
        case .mitchell: "Mitchell"
        case .lanczos3: "Lanczos"
        case .sinc: "Sinc"
        case .area: "Area"
        }
    }
}
```

Used in chunk 12.

The resampler is a caseless enum, used only as a namespace. Its documentation comment summarizes the conventions just derived. The body goes from the public entry points down to the kernels.

14 *⟨Resampling: the Resampler namespace 14⟩* ≡

```

/// Separable resampling with explicit, textbook kernels.
///
/// Coordinates follow the pixel-center convention: input pixel `j` covers `[j, j+1)` and its center is
///   ↪ `j + 0.5`.
/// Output pixel `i` maps to input position `(i + 0.5) / scale`.
/// - `nearest` picks one source pixel and does no prefiltering, so it aliases when downscaling.
/// - The kernel methods (tent, Catmull-Rom, Mitchell, Lanczos, sinc) widen their kernel by `1/scale`
///   when downscaling (antialiased).
/// - `area` weights source pixels by how much of the output pixel's footprint they cover.
public enum Resampler {
  ⟨Resampler: sinc radius and output size 15⟩

  ⟨Resampler: resampling by a scale factor 16⟩

  ⟨Resampler: resampling to a given size 17⟩

  // MARK: - Kernels

  ⟨Resampler: contributions of input pixels 20⟩

  ⟨Resampler: trimming and normalizing weights 24⟩

  ⟨Resampler: kernel supports 25⟩

  ⟨Resampler: the sinc function 26⟩

  ⟨Resampler: the BC-spline family 27⟩

  ⟨Resampler: kernel evaluation 28⟩
}

Used in chunk 12.

```

3.3 Entry points

A scale factor becomes an output size by rounding, never below one pixel. The parameter sheet uses the same function to preview the resulting size, so what the user is promised is what they get. The radius of the truncated sinc kernel is a public constant for the same reason: the menu title shows it.

15 *⟨Resampler: sinc radius and output size 15⟩* ≡

```

/// Half-width of the truncated sinc kernel, in source pixels (before widening).
public static let sincRadius = 8.0

public static func outputSize(for size: Int, scale: Double) -> Int {
  max(1, Int((Double(size) * scale).rounded()))
}

Used in chunk 14.

```

16 *⟨Resampler: resampling by a scale factor 16⟩* ≡

```

public static func resample(_ src: PixelBuffer, scale: Double, method: ResampleMethod) -> PixelBuffer {
  resample(
    src,
    width: outputSize(for: src.width, scale: scale),
    height: outputSize(for: src.height, scale: scale),
    method: method
  )
}

```

Used in chunk 14.

Resampling to a given size first premultiplies the source (Section 2.3), so that transparent pixels don't leak their color. It then computes one table of weights per axis, runs the two passes, and converts the result back to straight alpha. Note that the scale in each direction is derived from the rounded sizes, not from the requested factor, so the image maps exactly onto the new grid.

17 *⟨Resampler: resampling to a given size 17⟩* ≡

```
public static func resample(_ src: PixelBuffer, width: Int, height: Int, method: ResampleMethod) ->
    ↳ PixelBuffer {
    let premul = src.premultiplied()
    let xContrib = contributions(inSize: src.width, outSize: width, method: method)
    let yContrib = contributions(inSize: src.height, outSize: height, method: method)

    ⟨Resampler: horizontal pass 18⟩

    ⟨Resampler: vertical pass 19⟩

    return PixelBuffer.fromPremultiplied(width: width, height: height, out)
}
```

Used in chunk 14.

3.3.1 The two passes

The horizontal pass changes only the width. For every row and every output column it takes the weighted sum of a short run of consecutive input pixels, the run described by that column's contribution. The loops use unsafe buffer pointers: this is the one place where bounds checks on every access would noticeably slow the program down, and the contribution tables guarantee the indices are in range.

18 *⟨Resampler: horizontal pass 18⟩* ≡

```
// Horizontal pass: (src.width x src.height) -> (width x src.height)
var tmp = [Float](repeating: 0, count: width * src.height * 4)
premul.withUnsafeBufferPointer { s in
    tmp.withUnsafeMutableBufferPointer { t in
        for y in 0..

```

Used in chunk 17.

The vertical pass works on the intermediate image. Instead of gathering a column pixel by pixel, which would jump through memory, it accumulates whole input rows into each output row: every weight multiplies a contiguous run of $4W$ floats. The inner loop then runs over consecutive memory, which the compiler vectorizes.

```
19  ⟨Resampler: vertical pass 19⟩ ≡
// Vertical pass: (width × src.height) → (width × height)
var out = [Float](repeating: 0, count: width * height * 4)
tmp.withUnsafeBufferPointer { t in
    out.withUnsafeMutableBufferPointer { o in
        let rowStride = width * 4
        for y in 0..

```

Used in chunk 17.

3.4 Weights

A Contribution says which input pixels feed one output pixel: a first index and a list of weights for consecutive pixels. The weights are computed once per output column (or row) and reused for every row (or column), which is what makes separable resampling cheap.

```
20  ⟨Resampler: contributions of input pixels 20⟩ ≡
struct Contribution {
    var start: Int
    var weights: [Float]
}

static func contributions(inSize: Int, outSize: Int, method: ResampleMethod) -> [Contribution] {
    let scale = Double(outSize) / Double(inSize)
    var result: [Contribution] = []
    result.reserveCapacity(outSize)

    for i in 0..

```

Used in chunk 14.

Nearest neighbor takes the single input pixel whose interval contains u_i , that is $j = \lfloor u_i \rfloor$, clamped to the image. It uses no prefilter and no blending.

21 *⟨Resampler: nearest-neighbor contribution 21⟩* $\equiv$

```

case .nearest:
  let j = min(max(Int(center.rounded(.down)), 0), inSize - 1)
  result.append(Contribution(start: j, weights: [1]))

```

Used in chunk 20.

Area averaging is the box filter done exactly. The output pixel's footprint, mapped back to input coordinates, is the interval $[u_i - \frac{1}{2s}, u_i + \frac{1}{2s}]$. Each input pixel is weighted by the length of its overlap with the footprint,

$$w_j = \max(0, \min(hi, j + 1) - \max(lo, j)),$$

so for a factor of $\frac{1}{2}$ the result is the plain mean of pixel pairs, and for non-integer factors the edge pixels count in proportion to how much of them is covered. When upscaling, the footprint is smaller than a pixel and the method degrades gracefully into nearest neighbor with blending at the seams.

22 *⟨Resampler: area-average contribution 22⟩* $\equiv$

```

case .area:
  // Output pixel footprint in input coordinates, exact coverage weights.
  let half = 0.5 / scale
  let lo = center - half
  let hi = center + half
  let first = max(0, Int(lo.rounded(.down)))
  let last = min(inSize - 1, Int(hi.rounded(.up)) - 1)
  var weights: [Float] = []
  if last >= first {
    for j in first...last {
      let overlap = min(hi, Double(j + 1)) - max(lo, Double(j))
      weights.append(Float(max(0, overlap)))
    }
  }
  result.append(normalized(start: first, weights: weights, fallback: center, inSize: inSize))

```

Used in chunk 20.

All the true kernels share one path. The filter scale is $\max(1, 1/s)$: it is 1 when upscaling, where the kernel only interpolates, and $1/s$ when downscaling, where it also prefilters. The support widens by the same factor, and each input pixel within it gets the weight $k((j + \frac{1}{2} - u_i) / \max(1, 1/s))$.

23 *⟨Resampler: kernel contribution 23⟩* $\equiv$

```

case .tent, .catmullRom, .mitchell, .lanczos3, .sinc:
  let filterScale = max(1, 1 / scale)
  let support = kernelSupport(method) * filterScale
  let first = max(0, Int((center - support).rounded(.down)))
  let last = min(inSize - 1, Int((center + support).rounded(.up)))
  var weights: [Float] = []
  if last >= first {
    for j in first...last {
      let x = (Double(j) + 0.5 - center) / filterScale
      weights.append(Float(kernel(method, x)))
    }
  }
  result.append(normalized(start: first, weights: weights, fallback: center, inSize: inSize))

```

Used in chunk 20.

Before a weight list is stored, zero weights at both ends are removed; the support bounds are conservative, and the trimmed list keeps the inner loops short. The weights are then divided by their sum. This is the renormalization in the formula at the start of the chapter: it keeps flat regions flat and handles the image border, where part of the kernel falls outside the image.

If nothing is left, or the sum vanishes (possible in principle for kernels with negative lobes), the function falls back to nearest neighbor rather than dividing by zero.

24 *⟨Resampler: trimming and normalizing weights 24⟩* ≡

```
private static func normalized(start: Int, weights: [Float], fallback center: Double, inSize: Int) ->
  ↪ Contribution {
  // Trim zero weights at both ends to keep inner loops short.
  var first = 0
  var last = weights.count - 1
  while first <= last && weights[first] == 0 { first += 1 }
  while last >= first && weights[last] == 0 { last -= 1 }
  guard first <= last else {
    let j = min(max(Int(center.rounded(.down)), 0), inSize - 1)
    return Contribution(start: j, weights: [1])
  }
  let trimmed = Array(weights[first...last])
  let sum = trimmed.reduce(0, +)
  guard abs(sum) > 1e-12 else {
    let j = min(max(Int(center.rounded(.down)), 0), inSize - 1)
    return Contribution(start: j, weights: [1])
  }
  return Contribution(start: start + first, weights: trimmed.map { $0 / sum })
}
```

Used in chunk 14.

3.5 Kernels

Each kernel is an even function $k(x)$ that is zero outside a radius, its *support*.

25 *⟨Resampler: kernel supports 25⟩* ≡

```
static func kernelSupport(_ method: ResampleMethod) -> Double {
  switch method {
  case .nearest, .area: 0.5
  case .tent: 1
  case .catmullRom, .mitchell: 2
  case .lanczos3: 3
  case .sinc: sincRadius
  }
}
```

Used in chunk 14.

Sinc. The ideal reconstruction filter for band-limited signals is

$$\text{sinc}(x) = \frac{\sin \pi x}{\pi x}, \quad \text{sinc}(0) = 1,$$

whose Fourier transform is a perfect box: it passes every frequency below Nyquist unchanged and removes everything above. Its tails decay only like $1/x$, so it has to be truncated. LineScope truncates it at $|x| < 8$ with a rectangular window, which is the textbook “sinc filter” with its textbook flaw: the sharp cutoff produces visible ringing next to edges (the Gibbs phenomenon). The step-edge test pattern shows this clearly.

26 *⟨Resampler: the sinc function 26⟩* ≡

```
private static func sinc(_ x: Double) -> Double {
  if abs(x) < 1e-9 { return 1 }
  let px = Double.pi * x
  return sin(px) / px
}
```

Used in chunk 14.

The BC-splines. Mitchell and Netravali [3] described a two-parameter family of cubic kernels with support 2 that contains most cubic filters in use:

$$k_{B,C}(x) = \frac{1}{6} \begin{cases} (12 - 9B - 6C)|x|^3 + (-18 + 12B + 6C)|x|^2 + (6 - 2B), & |x| < 1, \\ (-B - 6C)|x|^3 + (6B + 30C)|x|^2 + (-12B - 48C)|x| + (8B + 24C), & 1 \leq |x| < 2, \\ 0, & \text{otherwise.} \end{cases}$$

The choice $B = 0$, $C = \frac{1}{2}$ is the Catmull-Rom spline, which is Keys' cubic convolution kernel with $a = -\frac{1}{2}$ [4]. It interpolates ($k(0) = 1$, $k(\pm 1) = k(\pm 2) = 0$) and reproduces quadratics exactly, but it rings a little. Mitchell and Netravali chose $B = C = \frac{1}{3}$ after a user study, trading a little blur for much less ringing and less anisotropy. That kernel does *not* interpolate: $k(0) = \frac{8}{9}$ and $k(\pm 1) = \frac{1}{18}$, so even resampling at scale 1 smooths the image slightly. The tests check these values.

27 *⟨Resampler: the BC-spline family 27⟩* $\equiv$

/// Mitchell-Netravali / Keys family: (B, C) = (1/3, 1/3) is Mitchell, (0, 1/2) is Catmull-Rom.

```
private static func bcSpline(_ ax: Double, b: Double, c: Double) -> Double {
    if ax < 1 {
        return ((12 - 9 * b - 6 * c) * ax * ax * ax + (-18 + 12 * b + 6 * c) * ax * ax + (6 - 2 * b)) / 6
    }
    if ax < 2 {
        return ((-b - 6 * c) * ax * ax * ax + (6 * b + 30 * c) * ax * ax
            + (-12 * b - 48 * c) * ax + (8 * b + 24 * c)) / 6
    }
    return 0
}
```

Used in chunk 14.

All kernels together. The box, $k(x) = 1$ for $|x| < \frac{1}{2}$, stands in for nearest neighbor and area averaging, although both have their own code paths above and never actually evaluate it. The *tent*, $k(x) = \max(0, 1 - |x|)$, is linear interpolation; applied separably it is bilinear interpolation. *Lanczos-3* tapers the sinc with a wider sinc as the window,

$$L_3(x) = \text{sinc}(x) \text{sinc}(x/3), \quad |x| < 3,$$

which keeps most of the sinc's sharpness and much less of its ringing.

28 *⟨Resampler: kernel evaluation 28⟩* $\equiv$

```
static func kernel(_ method: ResampleMethod, _ x: Double) -> Double {
    let ax = abs(x)
    switch method {
    case .nearest, .area:
        return ax < 0.5 ? 1 : 0
    case .tent:
        return max(0, 1 - ax)
    case .catmullRom:
        return bcSpline(ax, b: 0, c: 0.5)
    case .mitchell:
        return bcSpline(ax, b: 1.0 / 3, c: 1.0 / 3)
    case .lanczos3:
        return ax < 3 ? sinc(ax) * sinc(ax / 3) : 0
    case .sinc:
        // Ideal low-pass, truncated (rectangular window): expect ringing near edges.
        return ax < sincRadius ? sinc(ax) : 0
    }
}
```

Used in chunk 14.

Chapter 4

Filters and Operations

Filters keep the size of an image and change its content. LineScope offers the handful that people most often want to compare: blurs, a median, sharpening, noise reduction, edge detectors, and two per-pixel color maps. Where Core Image already has a good, well-known implementation it is used; where a filter is simple enough to state in a few lines (the 3×3 kernels), it is written out, so the reader can see precisely what was computed.

The second half of the chapter introduces `ImageOperation`, the value that describes *one step* from one image to the next, whether that step is a filter or a resampling.

4.1 The filters

29 `<Packages/LineScopeCore/Sources/LineScopeCore/Filters.swift 29> ≡`

```
import CoreImage
import CoreImage.CIFilterBuiltins
import Foundation
```

`<Filters: the FilterKind enumeration 30>`

`<Filters: number formatting for names 31>`

```
public enum Filters {
    <Filters: dispatching a filter 32>
```

`// MARK: - Core Image`

`<Filters: the Core Image context 35>`

`<Filters: running a Core Image filter on a buffer 36>`

`// MARK: - CPU filters`

`<Filters: per-pixel color maps 37>`

`<Filters: 3x3 convolution 38>`

`<Filters: Sobel gradient magnitude 39>`

```
}
```

Root chunk, tangled to Packages/LineScopeCore/Sources/LineScopeCore/Filters.swift.

A filter is described by an enumeration whose cases carry their parameters. Because the value is `Equatable` and `Sendable`, it can be stored in an image’s history and shipped to a background task. Its `displayName` becomes part of a tab title, so it names the parameters compactly: “Gaussian r=2” tells the user what was applied.

30 *⟨Filters: the FilterKind enumeration 30⟩* ≡

```
public enum FilterKind: Sendable, Equatable {
    case gaussianBlur(radius: Double)
    case boxBlur(radius: Double)
    case median
    case unsharpMask(radius: Double, intensity: Double)
    case noiseReduction(level: Double, sharpness: Double)
    case sobel
    case laplacian
    case emboss
    case grayscale
    case invert

    public var displayName: String {
        switch self {
            case .gaussianBlur(let r): "Gaussian r=\(fmt(r))"
            case .boxBlur(let r): "Box r=\(fmt(r))"
            case .median: "Median 3×3"
            case .unsharpMask(let r, let i): "Sharpen r=\(fmt(r)) i=\(fmt(i))"
            case .noiseReduction(let l, let s): "Denoise \(fmt(l))/\(fmt(s))"
            case .sobel: "Sobel"
            case .laplacian: "Laplacian"
            case .emboss: "Emboss"
            case .grayscale: "Grayscale"
            case .invert: "Invert"
        }
    }
}
```

Used in chunk 29.

Parameters are shown with at most three decimals and no trailing zeros, so 2 reads “2” and 0.125 reads “0.125”. The helper is internal to the package; the resampling and test-pattern names use it too.

31 *⟨Filters: number formatting for names 31⟩* ≡

```
func fmt(_ v: Double) -> String {
    v.formatted(.number.precision(.fractionLength(0...3)))
}
```

Used in chunk 29.

4.1.1 Dispatch

`Filters.apply` maps each case to its implementation. The first group goes to Core Image, each case building one built-in filter with its parameters.

32 *⟨Filters: dispatching a filter 32⟩* ≡

```
public static func apply(_ filter: FilterKind, to src: PixelBuffer) -> PixelBuffer {
    switch filter {
        case <Filters: Core Image cases 33>
        case <Filters: CPU cases 34>
    }
}
```

Used in chunk 29.

The Gaussian blur’s radius is Core Image’s, roughly the standard deviation σ in pixels. The box blur averages over a square of that radius. The median is Core Image’s fixed 3×3 median, good at removing isolated outliers without softening edges. The unsharp mask adds back *intensity* $\times$ (image – blurred image), which boosts high frequencies and is visible in the spectrum as a lift just below Nyquist. Noise reduction is Core Image’s adaptive smoothing, which leaves edges above a sharpness threshold alone.

33 *⟨Filters: Core Image cases 33⟩* ≡

```

case .gaussianBlur(let radius):
  return applyCI(src) { img in
    let f = CIFilter.gaussianBlur()
    f.inputImage = img
    f.radius = Float(radius)
    return f.outputImage
  }
case .boxBlur(let radius):
  return applyCI(src) { img in
    let f = CIFilter.boxBlur()
    f.inputImage = img
    f.radius = Float(radius)
    return f.outputImage
  }
case .median:
  return applyCI(src) { img in
    let f = CIFilter.median()
    f.inputImage = img
    return f.outputImage
  }
case .unsharpMask(let radius, let intensity):
  return applyCI(src) { img in
    let f = CIFilter.unsharpMask()
    f.inputImage = img
    f.radius = Float(radius)
    f.intensity = Float(intensity)
    return f.outputImage
  }
case .noiseReduction(let level, let sharpness):
  return applyCI(src) { img in
    let f = CIFilter.noiseReduction()
    f.inputImage = img
    f.noiseLevel = Float(level)
    f.sharpness = Float(sharpness)
    return f.outputImage
  }

```

Used in chunk 32.

The second group runs on the CPU. The Laplacian

$$\nabla^2 \approx \begin{pmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{pmatrix}$$

responds with both signs around an edge and zero on flat regions, so its absolute value is shown; otherwise half of every edge would be clipped to black. The emboss kernel's weights sum to 1, so flat regions keep their color and edges become a relief lit from the top left. Grayscale uses the Rec. 709 luma weights on the encoded values, $Y = 0.2126R + 0.7152G + 0.0722B$, and inversion replaces every component v with $1 - v$; both leave alpha alone.

34 *⟨Filters: CPU cases 34⟩* ≡

```

case .sobel:
  return sobel(src)
case .laplacian:
  // Absolute response so the edges are visible.
  return convolve3x3(src, kernel: [0, 1, 0, 1, -4, 1, 0, 1, 0], absolute: true)
case .emboss:
  return convolve3x3(src, kernel: [-2, -1, 0, -1, 1, 1, 0, 1, 2], absolute: false)
case .grayscale:
  return mapRGB(src) { c in
    let y = 0.2126 * c.r + 0.7152 * c.g + 0.0722 * c.b

```

```

        return RGBA(r: y, g: y, b: y, a: c.a)
    }
    case .invert:
        return mapRGB(src) { c in RGBA(r: 1 - c.r, g: 1 - c.g, b: 1 - c.b, a: c.a) }
}
Used in chunk 32.

```

4.1.2 The Core Image bridge

By default Core Image color-manages everything: it converts inputs to a linear working space, filters there, and converts back. That is usually right, but here it would make the Core Image filters disagree with the CPU filters and with the resampler, which all work on encoded values (Chapter 2). The context is therefore created with color management switched off for both the working and the output space. Intermediate caching is off too, since every image is filtered once and then thrown away. A `CIColorContext` is expensive to create and safe to share between threads, so there is one for the whole program.

35 *⟨Filters: the Core Image context 35⟩* ≡

```

/// Works directly on encoded values (no color management) so results are consistent with the CPU
    ↪ filters.
private static let context = CIColorContext(options: [
    .workingColorSpace: NSNull(),
    .outputColorSpace: NSNull(),
    .cacheIntermediates: false,
])
Used in chunk 29.

```

Running a filter means wrapping the premultiplied floats as a `CIColorImage` in the `RGBAf` format (four 32-bit floats per pixel, hence 16 bytes per pixel in a row) with no color space, which again means “don’t convert”. Two details keep the image the same size and free of dark borders. First, the input is *clamped to its extent*, which extends the border pixels to infinity, so a blur near the edge averages real edge colors instead of transparent black. Second, the output is cropped back to the original rectangle, since a blur’s output would otherwise grow by its radius. The rendered floats are then converted back to straight alpha.

If a filter can’t be built, the source is returned unchanged rather than failing the operation.

36 *⟨Filters: running a Core Image filter on a buffer 36⟩* ≡

```

private static func applyCI(_ src: PixelBuffer, _ build: (CIColorImage) -> CIColorImage?) -> PixelBuffer {
    let w = src.width
    let h = src.height
    let premul = src.premultiplied()
    let data = premul.withUnsafeBytes { Data($0) }
    let input = CIColorImage(
        bitmapData: data, bytesPerRow: w * 16, size: CGSize(width: w, height: h),
        format: .RGBAf, colorSpace: nil
    )
    let extent = input.extent
    guard let output = build(input.clampedToExtent())?.cropped(to: extent) else { return src }
    var result = [Float](repeating: 0, count: w * h * 4)
    result.withUnsafeMutableBytes { raw in
        context.render(
            output, toBitmap: raw.baseAddress!, rowBytes: w * 16, bounds: extent,
            format: .RGBAf, colorSpace: nil
        )
    }
    return PixelBuffer.fromPremultiplied(width: w, height: h, result)
}
Used in chunk 29.

```

4.1.3 CPU filters

A color map applies a function to every pixel independently.

37 *⟨Filters: per-pixel color maps 37⟩* $\equiv$

```
private static func mapRGB(_ src: PixelBuffer, _ f: (RGBA) -> RGBA) -> PixelBuffer {
    var out = src
    for y in 0..

```

Used in chunk 29.

The 3×3 convolution computes, for each color channel,

$$\text{out}(x, y) = \sum_{k_y=-1}^1 \sum_{k_x=-1}^1 K(k_x, k_y) \text{in}(\text{clamp}(x + k_x), \text{clamp}(y + k_y)),$$

where clamping the coordinates repeats the border pixels, the same edge rule as the Core Image path. The kernel is given row by row. Results are clamped to $[0, 1]$, optionally after taking the absolute value, and alpha is copied through unchanged. Since these kernels are applied to straight colors, not premultiplied ones, they are meant for opaque images, which is what they are used on.

38 *⟨Filters: 3x3 convolution 38⟩* $\equiv$

```
/// 3x3 convolution on RGB with clamped edges; alpha is preserved.
static func convolve3x3(_ src: PixelBuffer, kernel k: [Float], absolute: Bool) -> PixelBuffer {
    var out = src
    let w = src.width
    let h = src.height
    src.data.withUnsafeBufferPointer { s in
        out.data.withUnsafeMutableBufferPointer { o in
            for y in 0..

```

Used in chunk 29.

The Sobel operator estimates the two components of the gradient with the kernels

$$G_x = \begin{pmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{pmatrix}, \quad G_y = G_x^T,$$

each a derivative in one direction combined with a small smoothing in the other. The output is the gradient magnitude $\sqrt{s_x^2 + s_y^2}$, per channel and clamped to 1. Both sums are accumulated in the same pass over the neighborhood, so this can't be written as two calls to the general convolution without reading every pixel twice.

39 *⟨Filters: Sobel gradient magnitude 39⟩* ≡

```
/// Per-channel Sobel gradient magnitude.
static func sobel(_ src: PixelBuffer) -> PixelBuffer {
    let gx: [Float] = [-1, 0, 1, -2, 0, 2, -1, 0, 1]
    let gy: [Float] = [-1, -2, -1, 0, 0, 0, 1, 2, 1]
    var out = src
    let w = src.width
    let h = src.height
    src.data.withUnsafeBufferPointer { s in
        out.data.withUnsafeMutableBufferPointer { o in
            for y in 0..

```

Used in chunk 29.

4.2 Operations

The app never calls the filters or the resampler directly. It works with `ImageOperation` values, which unify the two kinds of step behind one name and one `apply` function. That gives the session model a single thing to run in the background, and a single thing to record. A derived image stores the list of operations that produced it from the original, its *lineage*, and the inspector prints that list.

40 $\langle \text{Packages/LineScopeCore/Sources/LineScopeCore/ImageOperation.swift } 40 \rangle \equiv$
`import Foundation`

/// One step that derives a new image from an existing one.

```
public enum ImageOperation: Sendable, Equatable {
    case filter(FilterKind)
    case resample(ResampleMethod, scale: Double)
```

$\langle \text{ImageOperation: display name } 41 \rangle$

$\langle \text{ImageOperation: applying the step } 42 \rangle$

```
}
```

Root chunk, tangled to Packages/LineScopeCore/Sources/LineScopeCore/ImageOperation.swift.

41 $\langle \text{ImageOperation: display name } 41 \rangle \equiv$

```
public var displayName: String {
    switch self {
    case .filter(let f): f.displayName
    case .resample(let m, let s): "\(m.shortName) x\(fmt(s))"
    }
}
```

Used in chunk 40.

Applying an operation is a pure function from buffer to buffer. It touches no shared state, which is what makes it safe to run on a detached task.

42 $\langle \text{ImageOperation: applying the step } 42 \rangle \equiv$

```
public func apply(to buffer: PixelBuffer) -> PixelBuffer {
    switch self {
    case .filter(let f): Filters.apply(f, to: buffer)
    case .resample(let m, let s): Resampler.resample(buffer, scale: s, method: m)
    }
}
```

Used in chunk 40.

Chapter 5

Lines, Colors and Spectra

The analysis part of LineScope is a pipeline of three steps. The selection line is sampled in the pixel grid of one image; each sample is converted to the chosen color model and reduced to one channel; the resulting profile goes through a real FFT. This chapter follows that order: first the line and its sampler, then the color models, and last the spectrum.

5.1 The selection line

The line is shared by all the copies of an image, and the copies differ in size. So the line can't be stored in pixels. It is stored in *normalized* coordinates instead, where (0,0) is the top-left corner of the image and (1,1) the bottom-right corner, whatever the size. A resampled copy only scales the image, so the same normalized line falls on the same content in every copy. Converting to the pixels of an image of size $W \times H$ is one multiplication per axis: $x = uW$ and $y = vH$.

43 `<Packages/LineScopeCore/Sources/LineScopeCore/LineSampler.swift 43> ≡`

`import Foundation`

`<LineSampler: normalized point 44>`

`<LineSampler: line segment 45>`

`<LineSampler: sample record 46>`

`<LineSampler: sampler 47>`

Root chunk, tangled to Packages/LineScopeCore/Sources/LineScopeCore/LineSampler.swift.

A normalized point is a plain pair of doubles. The views need to keep the point inside the image while it is dragged, so it can clamp itself to the unit square.

44 `<LineSampler: normalized point 44> ≡`

/// A point relative to the image extent: (0,0) is the top-left corner, (1,1) the bottom-right corner.

```
public struct NormalizedPoint: Sendable, Hashable {
    public var x: Double
    public var y: Double

    public init(x: Double, y: Double) {
        self.x = x
        self.y = y
    }

    public func clamped() -> NormalizedPoint {
        NormalizedPoint(x: min(max(x, 0), 1), y: min(max(y, 0), 1))
    }
}
```

Used in chunk 43.

The segment has a direction: position 0 of the profile is at `start` and position 1 at `end`. The initial line crosses the full width at mid-height. Its length depends on the image it is measured in. With $\Delta u = u_1 - u_0$ and $\Delta v = v_1 - v_0$,

$$L = \sqrt{(\Delta u W)^2 + (\Delta v H)^2}.$$

A copy at half size therefore has a line half as long, in its own pixels.

45 *⟨LineSampler: line segment 45⟩* ≡

```
/// The selection line, in normalized coordinates so it lands on the same content in every resampled
    ↪ copy.
public struct LineSegment: Sendable, Hashable {
    public var start: NormalizedPoint
    public var end: NormalizedPoint

    public init(start: NormalizedPoint, end: NormalizedPoint) {
        self.start = start
        self.end = end
    }

    /// Full-width horizontal line at mid-height.
    public static let initial = LineSegment(start: .init(x: 0, y: 0.5), end: .init(x: 1, y: 0.5))

    public func pixelLength(width: Int, height: Int) -> Double {
        let dx = (end.x - start.x) * Double(width)
        let dy = (end.y - start.y) * Double(height)
        return (dx * dx + dy * dy).squareRoot()
    }
}
```

Used in chunk 43.

5.2 Sampling along the line

The result of sampling keeps, for every sample, its position $t \in [0, 1]$ along the line and its color. It also records the line length and the spacing between samples, both in pixels. The spectrum needs the spacing to express frequencies in cycles per pixel.

46 *⟨LineSampler: sample record 46⟩* ≡

```
public struct LineSamples: Sendable {
    /// Position along the line, 0 at `start` and 1 at `end`.
    public var positions: [Double]
    public var colors: [RGBA]
    /// Line length in the sampled image's pixels.
    public var pixelLength: Double
    /// Distance between consecutive samples, in pixels ≈( 1).
    public var spacing: Double
}
```

Used in chunk 43.

The sampler has two parts: the walk along the segment, and the interpolation at one point.

47 *⟨LineSampler: sampler 47⟩* ≡

```
public enum LineSampler {
    ⟨LineSampler: sample along the line 48⟩

    ⟨LineSampler: bilinear interpolation 49⟩
}
```

Used in chunk 43.

How many samples should a line of length L get? The choice matters more than it seems. If every copy got the same number of samples, a half-size copy would be sampled twice per pixel. Its profile would look as smooth as the original's, and the loss of detail from shrinking would be hidden. So LineScope samples each copy in its *own* grid, about once per pixel:

$$n = \max(2, \lceil L \rceil + 1), \quad t_i = \frac{i}{n-1}, \quad i = 0, \dots, n-1,$$

so the spacing $L/(n-1)$ is at most one pixel, and very close to one for long lines. For the initial line of a W -pixel-wide image, $n = W + 1$ and the spacing is exactly one pixel. At least two samples keep a degenerate line from dividing by zero.

48 $\langle$ LineSampler: sample along the line 48 $\rangle \equiv$

```
/// Samples the line in the buffer's own pixel grid: `ceil(length) + 1` bilinear samples,
/// so a half-size copy yields about half as many samples.
public static func sample(_ buffer: PixelBuffer, along line: LineSegment) -> LineSamples {
    let w = Double(buffer.width)
    let h = Double(buffer.height)
    let length = line.pixelLength(width: buffer.width, height: buffer.height)
    let count = max(2, Int(length.rounded(.up)) + 1)
    let x0 = line.start.x * w, y0 = line.start.y * h
    let x1 = line.end.x * w, y1 = line.end.y * h

    var positions = [Double](repeating: 0, count: count)
    var colors = [RGBA](repeating: RGBA(r: 0, g: 0, b: 0, a: 0), count: count)
    for i in 0..

```

Used in chunk 47.

The sample points fall between pixel centers, so the color there must be interpolated. The convention used throughout LineScope is that pixel j covers $[j, j+1)$ and has its center at $j + \frac{1}{2}$. So a continuous position x lies between the centers of pixels $i = \lfloor x - \frac{1}{2} \rfloor$ and $i+1$, at fraction $t_x = (x - \frac{1}{2}) - i$, and the same holds for y . Bilinear interpolation mixes the four neighbors a, b, e, f (top-left, top-right, bottom-left, bottom-right) in two steps:

$$top = a + (b - a)t_x, \quad bottom = e + (f - e)t_x, \quad c = top + (bottom - top)t_y.$$

Indices outside the image are clamped to the border. Near an edge this repeats the border pixel, which is also what the resampler does. Bilinear interpolation is deliberately simple: it is a measuring instrument, and it must not add ringing of its own to the profiles it measures.

49 $\langle$ LineSampler: bilinear interpolation 49 $\rangle \equiv$

```
/// Bilinear interpolation at a continuous position (pixel centers at +0.5), edges clamped.
public static func bilinear(_ buffer: PixelBuffer, x: Double, y: Double) -> RGBA {
    let fx = x - 0.5
    let fy = y - 0.5
    let ix = Int(fx.rounded(.down))
    let iy = Int(fy.rounded(.down))
    let tx = Float(fx - Double(ix))
    let ty = Float(fy - Double(iy))
    let x0 = min(max(ix, 0), buffer.width - 1)
```

```

let x1 = min(max(ix + 1, 0), buffer.width - 1)
let y0 = min(max(iy, 0), buffer.height - 1)
let y1 = min(max(iy + 1, 0), buffer.height - 1)

let d = buffer.data
func mix(_ c: Int) -> Float {
    let a = d[(y0 * buffer.width + x0) * 4 + c]
    let b = d[(y0 * buffer.width + x1) * 4 + c]
    let e = d[(y1 * buffer.width + x0) * 4 + c]
    let f = d[(y1 * buffer.width + x1) * 4 + c]
    let top = a + (b - a) * tx
    let bottom = e + (f - e) * tx
    return top + (bottom - top) * ty
}
return RGBA(r: mix(0), g: mix(1), b: mix(2), a: mix(3))
}

```

Used in chunk 47.

5.3 Color models

A profile is one number per sample. The user picks a color model and one of its channels, and every sample is converted to it. The conversions work on the sRGB-encoded values as stored, the same values the filters see (Chapter 2).

50 `<Packages/LineScopeCore/Sources/LineScopeCore/ColorSpaces.swift 50> ≡`

```
import Foundation
```

```
<ColorSpaces: color model 51>
```

```
<ColorSpaces: conversions 52>
```

Root chunk, tangled to Packages/LineScopeCore/Sources/LineScopeCore/ColorSpaces.swift.

A color model knows the names of its channels and the range in which to plot each of them. Hue is an angle in degrees; every other channel lies in $[0, 1]$. Fixing the plot range, instead of fitting it to the data, is what makes two profiles comparable at a glance.

51 `<ColorSpaces: color model 51> ≡`

```

public enum ColorModel: String, CaseIterable, Identifiable, Sendable {
    case rgba
    case hsl
    case cmyk

    public var id: String { rawValue }

    public var displayName: String {
        switch self {
        case .rgba: "RGBA"
        case .hsl: "HSL"
        case .cmyk: "CMYK"
        }
    }

    public var channels: [String] {
        switch self {
        case .rgba: ["Red", "Green", "Blue", "Alpha"]
        case .hsl: ["Hue", "Saturation", "Lightness"]
        case .cmyk: ["Cyan", "Magenta", "Yellow", "Black"]
        }
    }
}

```

/// Plot range for a channel; hue is in degrees, everything else in 0...1.

```

public func range(ofChannel channel: Int) -> ClosedRange<Double> {
    self == .hsl && channel == 0 ? 0...360 : 0...1
}
}

```

Used in chunk 50.

The conversions come in pairs, color model to RGB and back. Only the forward direction is used by the app; the inverses exist so that the tests can check round trips (Chapter 7).

52 $\langle \text{ColorSpaces: conversions 52} \rangle \equiv$

```

public enum ColorConversion {
    // MARK: HSL (hue in degrees)

     $\langle \text{ColorSpaces: RGB to HSL 53} \rangle$ 

     $\langle \text{ColorSpaces: HSL to RGB 54} \rangle$ 

    // MARK: CMYK (naive device-independent conversion,  $K = 1 - \max(R, G, B)$ )

     $\langle \text{ColorSpaces: RGB to CMYK and back 55} \rangle$ 

    // MARK: Channel extraction

     $\langle \text{ColorSpaces: channel extraction 56} \rangle$ 
}

```

Used in chunk 50.

5.3.1 HSL

With $M = \max(R, G, B)$, $m = \min(R, G, B)$ and chroma $d = M - m$, the standard definitions are

$$L = \frac{M + m}{2}, \quad S = \frac{d}{1 - |2L - 1|}, \quad H = 60^\circ \times \begin{cases} \frac{G - B}{d} \bmod 6 & \text{if } M = R, \\ \frac{B - R}{d} + 2 & \text{if } M = G, \\ \frac{R - G}{d} + 4 & \text{if } M = B. \end{cases}$$

A gray has $d = 0$. Its hue is undefined, and it is reported as 0 with zero saturation. The remainder operation in Swift keeps the sign of its dividend, so a negative hue is brought back into $[0^\circ, 360^\circ)$ at the end. The saturation is clamped because rounding can push it a hair outside $[0, 1]$ when L is close to 0 or 1.

Hue is circular, so a profile across a red area can jump between 0° and 360° . The spectrum then sees a sharp step that isn't really in the image. That's a property of hue itself, and it's worth remembering when reading hue spectra.

53 $\langle \text{ColorSpaces: RGB to HSL 53} \rangle \equiv$

```

public static func hsl(from c: RGBA) -> (h: Float, s: Float, l: Float) {
    let maxV = max(c.r, c.g, c.b)
    let minV = min(c.r, c.g, c.b)
    let l = (maxV + minV) / 2
    let d = maxV - minV
    guard d > 1e-7 else { return (0, 0, l) }
    let s = d / (1 - abs(2 * l - 1))
    var h: Float
    if maxV == c.r {
        h = ((c.g - c.b) / d).truncatingRemainder(dividingBy: 6)
    } else if maxV == c.g {
        h = (c.b - c.r) / d + 2
    } else {

```

```

        h = (c.r - c.g) / d + 4
    }
    h *= 60
    if h < 0 { h += 360 }
    return (h, min(max(s, 0), 1), 1)
}

```

Used in chunk 52.

The inverse builds the color from its chroma $C = (1 - |2L - 1|)S$. With $H' = H/60^\circ$, the second-largest component is $X = C(1 - |H' \bmod 2 - 1|)$. The sextant $\lfloor H' \rfloor$ decides which of R, G, B receive C, X and 0. Finally the lightness offset $m = L - C/2$ is added to all three.

54 $\langle \text{ColorSpaces: HSL to RGB 54} \rangle \equiv$

```

public static func rgb(h: Float, s: Float, l: Float, alpha: Float = 1) -> RGBA {
    let c = (1 - abs(2 * l - 1)) * s
    let hp = h / 60
    let x = c * (1 - abs(hp.truncatingRemainder(dividingBy: 2) - 1))
    let (r, g, b): (Float, Float, Float)
    switch hp {
    case ..<1: (r, g, b) = (c, x, 0)
    case ..<2: (r, g, b) = (x, c, 0)
    case ..<3: (r, g, b) = (0, c, x)
    case ..<4: (r, g, b) = (0, x, c)
    case ..<5: (r, g, b) = (x, 0, c)
    default: (r, g, b) = (c, 0, x)
    }
    let m = l - c / 2
    return RGBA(r: r + m, g: g + m, b: b + m, a: alpha)
}

```

Used in chunk 52.

5.3.2 CMYK

Real CMYK depends on inks and paper, and needs an ICC profile. LineScope uses the naive conversion instead, which is enough to see how a filter treats the “amount of black” or of one ink:

$$K = 1 - \max(R, G, B), \quad C = \frac{1 - R - K}{1 - K}, \quad M = \frac{1 - G - K}{1 - K}, \quad Y = \frac{1 - B - K}{1 - K}.$$

Pure black ($K = 1$) would divide by zero, so it is returned as $(0, 0, 0, 1)$. The inverse is $R = (1 - C)(1 - K)$, and likewise for G and B .

55 $\langle \text{ColorSpaces: RGB to CMYK and back 55} \rangle \equiv$

```

public static func cmyk(from c: RGBA) -> (c: Float, m: Float, y: Float, k: Float) {
    let k = 1 - max(c.r, c.g, c.b)
    guard k < 1 - 1e-7 else { return (0, 0, 0, 1) }
    let d = 1 - k
    return ((1 - c.r - k) / d, (1 - c.g - k) / d, (1 - c.b - k) / d, k)
}

public static func rgb(c: Float, m: Float, y: Float, k: Float, alpha: Float = 1) -> RGBA {
    RGBA(r: (1 - c) * (1 - k), g: (1 - m) * (1 - k), b: (1 - y) * (1 - k), a: alpha)
}

```

Used in chunk 52.

The session calls this once per sample: it converts the color and picks the channel by its index, in the order given by `ColorModel.channels`.

56 *⟨ColorSpaces: channel extraction 56⟩* ≡

```
public static func value(of color: RGBA, model: ColorModel, channel: Int) -> Float {
    switch model {
    case .rgba:
        return [color.r, color.g, color.b, color.a][channel]
    case .hsl:
        let v = hsl(from: color)
        return [v.h, v.s, v.l][channel]
    case .cmyk:
        let v = cmyk(from: color)
        return [v.c, v.m, v.y, v.k][channel]
    }
}
```

Used in chunk 52.

5.4 The spectrum

The last step turns a profile of n samples into an amplitude spectrum. It uses the FFT of Apple’s Accelerate framework (vDSP).

57 *⟨Packages/LineScopeCore/Sources/LineScopeCore/Spectrum.swift 57⟩* ≡

```
import Accelerate
import Foundation
```

⟨Spectrum: window functions 58⟩

⟨Spectrum: result 59⟩

⟨Spectrum: analyzer 60⟩

Root chunk, tangled to Packages/LineScopeCore/Sources/LineScopeCore/Spectrum.swift.

The DFT treats its input as one period of a periodic signal. The two ends of a profile rarely match, so the implied periodic signal has a jump, and that jump leaks energy into every frequency. A window tapers the ends to zero and trades a little frequency resolution for much less leakage. LineScope offers no window (“rectangular”) and the Hann window [5],

$$w_i = \frac{1}{2} - \frac{1}{2} \cos \frac{2\pi i}{n-1}, \quad i = 0, \dots, n-1.$$

58 *⟨Spectrum: window functions 58⟩* ≡

```
public enum WindowFunction: String, CaseIterable, Identifiable, Sendable {
    case none
    case hann

    public var id: String { rawValue }
    public var displayName: String { self == .none ? "Rectangular" : "Hann" }
}
```

Used in chunk 57.

The result gives each bin’s amplitude and two frequency axes. *Cycles per pixel* are measured in the grid of the sampled image, where the Nyquist limit is $\frac{1}{2}$. *Cycles per line* count whole periods over the length of the line. The same image content has the same cycles-per-line frequency in every copy, so that axis is the one for comparing an original with its resampled copies. On that axis, each copy’s Nyquist limit sits at half its own sample count.

The peak is searched among the non-DC bins. The mean usually dominates bin 0 and is rarely what one wants to find.

59 *⟨Spectrum: result 59⟩* ≡

```
public struct SpectrumResult: Sendable {
    /// Bin frequencies in cycles per pixel of the sampled image (Nyquist ≈ 0.5).
    public var cyclesPerPixel: [Double]
    /// Bin frequencies in cycles over the whole line; comparable across resampled copies.
    public var cyclesPerLine: [Double]
    /// Single-sided amplitude: a sine of amplitude A peaks at ≈ A.
    public var amplitudes: [Double]
    public var fftSize: Int
    public var sampleCount: Int

    public static let empty = SpectrumResult(cyclesPerPixel: [], cyclesPerLine: [], amplitudes: [],
        ↪ fftSize: 0, sampleCount: 0)

    /// Index of the largest non-DC bin.
    public var peakIndex: Int? {
        guard amplitudes.count > 1 else { return nil }
        return amplitudes.indices.dropFirst().max { amplitudes[$0] < amplitudes[$1] }
    }
}
```

Used in chunk 57.

The analyzer is one function with five steps.

60 *⟨Spectrum: analyzer 60⟩* ≡

```
public enum SpectrumAnalyzer {
    /// Real FFT of a profile, zero-padded to the next power of two.
    /// - Parameter spacing: distance between samples, in pixels.
    public static func compute(
        _ values: [Float], spacing: Double, window: WindowFunction, removeMean: Bool
    ) -> SpectrumResult {
        ⟨Spectrum: choose the FFT size 61⟩

        ⟨Spectrum: condition the signal 62⟩
        ⟨Spectrum: zero-pad 63⟩

        ⟨Spectrum: packed real FFT 64⟩

        ⟨Spectrum: single-sided amplitudes 65⟩

        ⟨Spectrum: frequency axes 66⟩
    }
}
```

Used in chunk 57.

The radix-2 FFT of vDSP needs a power-of-two length $N = 2^{\lceil \log_2 n \rceil}$. A profile with fewer than two samples has no spectrum.

61 *⟨Spectrum: choose the FFT size 61⟩* ≡

```
let n = values.count
guard n >= 2 else { return .empty }
let log2n = vDSP_Length(Int(ceil(log2(Double(n))))))
let fftSize = 1 << Int(log2n)
let half = fftSize / 2
```

Used in chunk 60.

Removing the mean is optional. Without it, bin 0 holds the average brightness, which in dB scale flattens everything else. The window is applied after the mean is removed, so it tapers the fluctuations and not the offset. Its sum $\sum w_i$ is kept for normalization: with no window it is simply n .

62 *⟨Spectrum: condition the signal 62⟩* ≡

```

var signal = values
if removeMean {
    let mean = vDSP.mean(signal)
    signal = vDSP.add(-mean, signal)
}
var windowSum = Float(n)
if window == .hann {
    // Symmetric Hann over the n real samples.
    var w = [Float](repeating: 0, count: n)
    for i in 0..

```

Used in chunk 60.

The n conditioned samples are followed by $N - n$ zeros. Zero padding adds no information. It interpolates the spectrum more finely, so that peaks are drawn at about the right place.

63 *⟨Spectrum: zero-pad 63⟩* ≡

```

var padded = [Float](repeating: 0, count: fftSize)
padded.replaceSubrange(0..

```

Used in chunk 60.

vDSP computes a real FFT of length N as a complex FFT of length $N/2$. First vDSP_ctoz reads the real array as $N/2$ complex numbers: even samples become real parts and odd samples imaginary parts, in split form. Then vDSP_fft_zrip transforms it in place.

The output is *packed*. For $1 \leq k < N/2$, `real[k]` and `imag[k]` hold $2 \operatorname{Re} X_k$ and $2 \operatorname{Im} X_k$. The two purely real bins that have no partner share the first slot: `real[0]` holds $2X_0$ (the DC term) and `imag[0]` holds $2X_{N/2}$ (the Nyquist term). The factor 2 is vDSP's scaling convention for the forward real transform.

64 *⟨Spectrum: packed real FFT 64⟩* ≡

```

var real = [Float](repeating: 0, count: half)
var imag = [Float](repeating: 0, count: half)
guard let setup = vDSP_create_fftsetup(log2n, FFTRadix(kFFTRadix2)) else { return .empty }
defer { vDSP_destroy_fftsetup(setup) }

real.withUnsafeMutableBufferPointer { rp in
    imag.withUnsafeMutableBufferPointer { ip in
        var split = DSPSplitComplex(realp: rp.baseAddress!, imagp: ip.baseAddress!)
        padded.withUnsafeBytes { raw in
            vDSP_ctoz(raw.bindMemory(to: DSPComplex.self).baseAddress!, 2, &split, 1, vDSP_Length(half))
        }
        vDSP_fft_zrip(setup, &split, 1, log2n, FFTDirection(FFT_FORWARD))
    }
}

```

Used in chunk 60.

The amplitudes are made *single-sided* and normalized so they can be read as the amplitude of a sinusoid. A real signal $A \cos(2\pi fi + \varphi)$ puts $|X_k| = \frac{A}{2} \sum w_i$ into bin k and the same into its mirror bin $N - k$, which isn't stored. Therefore

$$A_k = \frac{2|X_k|}{\sum_i w_i} \quad (0 < k < N/2), \quad A_0 = \frac{|X_0|}{\sum_i w_i}, \quad A_{N/2} = \frac{|X_{N/2}|}{\sum_i w_i},$$

since DC and Nyquist have no mirror. Dividing by the window sum rather than by n corrects for the energy the window removed. With it, a sine of amplitude 1 reads about 1 with either window. The halving in the code undoes vDSP's factor 2.

65 *⟨Spectrum: single-sided amplitudes 65⟩* ≡

```
// vDSP's packed output is scaled by 2; DC is in real[0] and Nyquist in imag[0].
var amplitudes = [Double](repeating: 0, count: half + 1)
let norm = Double(max(windowSum, 1e-12))
amplitudes[0] = abs(Double(real[0]) / 2) / norm
amplitudes[half] = abs(Double(imag[0]) / 2) / norm
for k in 1..

```

Used in chunk 60.

Bin k of an N -point transform of samples spaced δ pixels apart has frequency $k/(N\delta)$ cycles per pixel, which reaches $1/(2\delta) \approx \frac{1}{2}$ at $k = N/2$. The line spans $(n - 1)\delta$ pixels, so the same bin has $k(n - 1)/N$ cycles per line.

66 *⟨Spectrum: frequency axes 66⟩* ≡

```
let lineLength = spacing * Double(n - 1)
var cpp = [Double](repeating: 0, count: half + 1)
var cpl = [Double](repeating: 0, count: half + 1)
for k in 0...half {
    cpp[k] = Double(k) / (Double(fftSize) * spacing)
    cpl[k] = cpp[k] * lineLength
}
return SpectrumResult(
    cyclesPerPixel: cpp, cyclesPerLine: cpl, amplitudes: amplitudes,
    fftSize: fftSize, sampleCount: n
)
```

Used in chunk 60.

Chapter 6

Test Patterns

Natural photographs make poor test inputs for a resampler. Their spectrum is unknown, so it's hard to tell whether a ripple in a profile comes from the image or from the filter. Synthetic patterns have known spectra, and each one stresses one property: a chirp sweeps all frequencies, an edge shows ringing, an impulse shows the point-spread function. LineScope can generate eleven such patterns. The generator lives in the core package, so the tests can use it too.

6.1 The catalog

67 *<Packages/LineScopeCore/Sources/LineScopeCore/TestPattern.swift 67> ≡*

```
import Foundation
```

```
<TestPattern: pattern kinds 68>
```

```
<TestPattern: specification 73>
```

```
<TestPattern: generator 74>
```

```
<TestPattern: SplitMix64 81>
```

Root chunk, tangled to Packages/LineScopeCore/Sources/LineScopeCore/TestPattern.swift.

A kind is an enumeration case. It is Codable because a pattern window is restored after a relaunch from its specification alone (Chapter 9).

68 *<TestPattern: pattern kinds 68> ≡*

```
/// Synthetic images commonly used to evaluate resampling and filtering.
```

```
public enum TestPatternKind: String, CaseIterable, Identifiable, Codable, Sendable {
```

```
<TestPattern: the cases 69>
```

```
    public var id: String { rawValue }
```

```
<TestPattern: display names 70>
```

```
<TestPattern: what each pattern is for 71>
```

```
<TestPattern: the tunable parameter 72>
```

```
}
```

Used in chunk 67.

69 $\langle \text{TestPattern: the cases 69} \rangle \equiv$

```
case zonePlate
case linearChirp
case sineGrating
case squareGrating
case checkerboard
case stepEdge
case slantedEdge
case impulses
case siemensStar
case whiteNoise
case hueRamp
```

Used in chunk 68.

70 $\langle \text{TestPattern: display names 70} \rangle \equiv$

```
public var displayName: String {
    switch self {
        case .zonePlate: "Zone Plate"
        case .linearChirp: "Linear Chirp"
        case .sineGrating: "Sine Grating"
        case .squareGrating: "Square-Wave Bars"
        case .checkerboard: "Checkerboard"
        case .stepEdge: "Step Edge"
        case .slantedEdge: "Slanted Edge"
        case .impulses: "Impulse Grid"
        case .siemensStar: "Siemens Star"
        case .whiteNoise: "White Noise"
        case .hueRamp: "Hue / Lightness Ramp"
    }
}
```

Used in chunk 68.

The generator window shows one sentence about each pattern. These sentences are the short version of this chapter, and they say which filter behavior each pattern reveals.

71 $\langle \text{TestPattern: what each pattern is for 71} \rangle \equiv$

/// What the pattern is good for, shown in the generator window.

```
public var purpose: String {
    switch self {
        case .zonePlate: "Circular chirp: frequency grows with radius up to the chosen maximum. Aliasing
        ↪ appears as false ring centers."
        case .linearChirp: "Horizontal sweep from 0 to the chosen frequency. Put the line across it to read
        ↪ a filter's frequency response."
        case .sineGrating: "Single pure frequency along x. The spectrum should show one peak."
        case .squareGrating: "Bars with odd harmonics. Shows how filters treat harmonics near Nyquist."
        case .checkerboard: "Two-dimensional square wave. Small cells alias badly under nearest-neighbor."
        case .stepEdge: "One hard vertical edge. Reveals ringing (sinc, Lanczos) versus blur (tent,
        ↪ Mitchell)."
        case .slantedEdge: "Edge tilted a few degrees, the standard target for measuring sharpness (MTF)."
        case .impulses: "Isolated single pixels. Each dot shows the filter's point-spread function."
        case .siemensStar: "Radial spokes: spatial frequency rises toward the center, in every orientation."
        case .whiteNoise: "Uniform random values with a flat spectrum. The profile's spectrum shows the
        ↪ filter's transfer curve."
        case .hueRamp: "Hue across x, lightness down y. Use it with the HSL and CMYK channels."
    }
}
```

Used in chunk 68.

Every pattern has at most one parameter. Keeping it to one number keeps the user interface simple, and a specification stays a small value that is easy to compare and store. The parameter's name, default and range are given here, next to the patterns, so the generator window needs no knowledge of what the numbers mean.

72 *⟨TestPattern: the tunable parameter 72⟩* ≡

```

/// The single tunable parameter, or nil if the pattern has none.
public var parameter: (name: String, defaultValue: Double, range: ClosedRange<Double>)? {
    switch self {
        case .zonePlate, .linearChirp: ("Max frequency (cycles/px)", 0.5, 0.01...1)
        case .sineGrating: ("Frequency (cycles/px)", 0.1, 0.001...1)
        case .squareGrating: ("Period (px)", 8, 2...512)
        case .checkerboard: ("Cell size (px)", 8, 1...512)
        case .slantedEdge: ("Angle (degrees)", 5, -45...45)
        case .impulses: ("Spacing (px)", 32, 2...1024)
        case .siemensStar: ("Spokes", 36, 4...360)
        case .whiteNoise: ("Seed", 1, 0...1_000_000)
        case .stepEdge, .hueRamp: nil
    }
}

```

Used in chunk 68.

6.2 Specifications

A specification is everything needed to generate a pattern: kind, size and parameter. Because it is Hashable and Codable, it can identify a window directly (it is its own id), and SwiftUI can store it to restore that window. A missing parameter takes the kind's default.

The title names the window. It shows the parameter only when it tells something: the seed of white noise is interesting only if it isn't the default.

73 *⟨TestPattern: specification 73⟩* ≡

```

public struct TestPatternSpec: Codable, Hashable, Identifiable, Sendable {
    public var kind: TestPatternKind
    public var width: Int
    public var height: Int
    public var parameter: Double

    public var id: Self { self }

    public init(kind: TestPatternKind, width: Int = 512, height: Int = 512, parameter: Double? = nil) {
        self.kind = kind
        self.width = width
        self.height = height
        self.parameter = parameter ?? kind.parameter?.defaultValue ?? 0
    }

    public var title: String {
        var t = "\(kind.displayName) \((width)×\((height))"
        if let p = kind.parameter, kind != .whiteNoise || parameter != p.defaultValue {
            t += " (\(fmt(parameter)))"
        }
        return t
    }
}

```

Used in chunk 67.

6.3 The generator

Generation is one function: set up a canvas, then fill it according to the kind.

74 $\langle \text{TestPattern: generator 74} \rangle \equiv$

```
public enum TestPatternGenerator {
    public static func make(_ spec: TestPatternSpec) -> PixelBuffer {
         $\langle \text{TestPattern: set up the canvas 75} \rangle$ 

         $\langle \text{TestPattern: gray-level helper 76} \rangle$ 

        switch spec.kind {
             $\langle \text{TestPattern: chirps and gratings 77} \rangle$ 
             $\langle \text{TestPattern: edges and impulses 78} \rangle$ 
             $\langle \text{TestPattern: Siemens star 79} \rangle$ 
             $\langle \text{TestPattern: noise and color ramp 80} \rangle$ 
        }
        return buf
    }
}
```

Used in chunk 67.

The center (c_x, c_y) is the geometric center of the image, which for an even size lies on a pixel corner.

75 $\langle \text{TestPattern: set up the canvas 75} \rangle \equiv$

```
let w = max(1, spec.width)
let h = max(1, spec.height)
let p = spec.parameter
var buf = PixelBuffer(width: w, height: h)
let cx = Double(w) / 2
let cy = Double(h) / 2
```

Used in chunk 74.

Most patterns are gray levels given by a function $f(x, y)$. The helper evaluates it at pixel centers $(x + \frac{1}{2}, y + \frac{1}{2})$, the convention of the whole program. Evaluating at corners would shift every pattern by half a pixel against the line sampler. Point sampling like this is deliberate: no pattern is antialiased, so patterns with frequencies near or above the Nyquist limit already alias in the original. Those are exactly the cases that test a filter.

76 $\langle \text{TestPattern: gray-level helper 76} \rangle \equiv$

```
func gray(_ f: (Double, Double) -> Double) {
    for y in 0..

```

Used in chunk 74.

6.3.1 Chirps and gratings

A *zone plate* is a circular chirp,

$$f(r) = \frac{1}{2} + \frac{1}{2} \cos(\pi k r^2),$$

where r is the distance from the center. Its phase is $\phi(r) = \pi k r^2$, so its local frequency is $\frac{1}{2\pi} \frac{d\phi}{dr} = k r$ cycles per pixel, growing linearly with the radius. The constant k is chosen so that the frequency reaches the parameter p at $r = \max(w, h)/2$, the ends of the default horizontal line. Along that line a zone plate thus sweeps every frequency from 0 to p and back. When a copy can't represent the high end, the excess folds back and shows as false ring centers: the classic picture of aliasing.

The *linear chirp* is the one-dimensional version, a sweep across x only. Its phase $\pi p x^2 / W$ has instantaneous frequency $p x / W$, from 0 at the left edge to p at the right. Read along a horizontal line, its profile is the frequency response of the filter, with frequency on the horizontal axis.

The *sine grating* has a single frequency p , so its spectrum should show a single peak. The *square-wave bars* contain the odd harmonics 1, 3, 5, ... of 1/period, with amplitudes falling as $1/n$. The *checkerboard* is their two-dimensional product.

77 $\langle \text{TestPattern: chirps and gratings 77} \rangle \equiv$

```
case .zonePlate:
    // Local frequency k·r reaches p at r = max(w,h)/2 (the left/right edges of the midline).
    let k = p / (Double(max(w, h)) / 2)
    gray { x, y in
        let r2 = (x - cx) * (x - cx) + (y - cy) * (y - cy)
        return 0.5 + 0.5 * cos(Double.pi * k * r2)
    }
case .linearChirp:
    // Phase  $\pi 2 \cdot p \cdot x^2 / (2W)$ : instantaneous frequency  $p \cdot x / W$ .
    gray { x, _ in 0.5 + 0.5 * cos(Double.pi * p * x * x / Double(w)) }
case .sineGrating:
    gray { x, _ in 0.5 + 0.5 * sin(2 * Double.pi * p * x) }
case .squareGrating:
    let period = max(p, 1)
    gray { x, _ in (x - 0.5).truncatingRemainder(dividingBy: period) < period / 2 ? 1 : 0 }
case .checkerboard:
    let cell = max(Int(p.rounded()), 1)
    gray { x, y in ((Int(x) / cell) + (Int(y) / cell)) % 2 == 0 ? 1 : 0 }

Used in chunk 74.
```

6.3.2 Edges and impulses

A *step edge* is the input that separates the kernels most clearly. An interpolating kernel with negative lobes (Catmull-Rom, Lanczos, sinc) overshoots on both sides of the step and rings, and a truncated sinc rings the longest. A non-negative kernel (tent, area) only blurs, and Mitchell-Netravali sits in between [3]. The levels 0.1 and 0.9 leave room for overshoot to show instead of being clipped at 0 and 1.

A *slanted edge* is the same edge tilted by α degrees. Because the edge crosses the pixel grid at a slight angle, successive rows sample it at different sub-pixel phases, which is why it is the standard target for measuring sharpness. A pixel is on the bright side when $(x - c_x) \cos \alpha + (y - c_y) \sin \alpha \geq 0$.

The *impulse grid* puts single bright pixels on a black ground, far enough apart that their responses don't overlap. After a filter, each dot is the kernel's two-dimensional point-spread function, sampled.

78 $\langle \text{TestPattern: edges and impulses 78} \rangle \equiv$

```
case .stepEdge:
    gray { x, _ in x < cx ? 0.1 : 0.9 }
case .slantedEdge:
    // Dark/light halves split by a line through the center tilted from vertical.
    let a = p * Double.pi / 180
    gray { x, y in ((x - cx) * cos(a) + (y - cy) * sin(a)) < 0 ? 0.1 : 0.9 }
case .impulses:
    let s = max(Int(p.rounded()), 1)
    let off = s / 2
    gray { x, y in (Int(x) % s == off && Int(y) % s == off) ? 1 : 0 }

Used in chunk 74.
```

6.3.3 The Siemens star

A *Siemens star* has s bright and s dark spokes, given by the sign of $\sin(s\theta)$. At radius r the circumference $2\pi r$ holds s periods, so the tangential frequency is $s/(2\pi r)$ cycles per pixel. It rises toward the center without limit, and it does so in every direction, which the one-dimensional patterns can't show. Outside the inscribed circle the image is mid-gray.

79 $\langle \text{TestPattern: Siemens star 79} \rangle \equiv$

```
case .siemensStar:
  let spokes = max(p.rounded(), 2)
  let radius = Double(min(w, h)) / 2
  gray { x, y in
    let dx = x - cx, dy = y - cy
    guard dx * dx + dy * dy <= radius * radius else { return 0.5 }
    let theta = atan2(dy, dx)
    return sin(spokes * theta) >= 0 ? 1 : 0
  }
```

Used in chunk 74.

6.3.4 Noise and color

White noise has, on average, a flat spectrum. After filtering, the spectrum of a profile is therefore the filter's transfer function times a constant, up to random fluctuations. The values must be reproducible, so that a restored window shows the same noise. So they come from a seeded generator rather than the system's random source. The top 53 bits of each 64-bit output divided by 2^{53} give a uniform double in $[0, 1)$, with every representable value equally likely.

The *hue/lightness ramp* is the only colored pattern. Hue runs across x at full saturation, and lightness falls from 1 at the top to 0 at the bottom. It is meant for the HSL and CMYK channels of the profile. The conversion can land a rounding error outside $[0, 1]$, so every component is clamped.

80 $\langle \text{TestPattern: noise and color ramp 80} \rangle \equiv$

```
case .whiteNoise:
  var rng = SplitMix64(seed: UInt64(max(p, 0)))
  for y in 0..

```

Used in chunk 74.

6.4 SplitMix64

SplitMix64 is a tiny, fast generator with good statistical quality for a problem like this. Its state advances by a fixed odd constant (the “golden gamma” $\lfloor 2^{64}/\varphi \rfloor$), and each state is scrambled by two xor-shift-multiply rounds. The wrapping operators $\&+=$ and $\&*$ give the arithmetic modulo 2^{64} that the algorithm needs; the ordinary operators would trap on overflow.

81 $\langle \text{TestPattern: SplitMix64 } 81 \rangle \equiv$

/// Small deterministic generator so noise patterns are reproducible from their seed.

```

struct SplitMix64 {
    private var state: UInt64

    init(seed: UInt64) { state = seed }

    mutating func next() -> UInt64 {
        state &+= 0x9E37_79B9_7F4A_7C15
        var z = state
        z = (z ^ (z >> 30)) &* 0xBF58_476D_1CE4_E5B9
        z = (z ^ (z >> 27)) &* 0x94D0_49BB_1331_11EB
        return z ^ (z >> 31)
    }
}

```

Used in chunk 67.

Chapter 7

Tests

The core package has one test file. It doesn't try to cover every line. Each test pins down a *property* that the earlier chapters claim, one the rest of the program depends on and that a careless change could break without anyone seeing it. If the FFT normalization were off by a factor of two, for instance, every spectrum would still look plausible. Only a test that knows the right answer catches it. Run the tests with `swift test` in the package directory, or one of them with `swift test --filter SpectrumTests/testSinePeak`.

82 `<Packages/LineScopeCore/Tests/LineScopeCoreTests/LineScopeCoreTests.swift 82> ≡`

```
import XCTest
@testable import LineScopeCore
```

`<Tests: gradient fixture 83>`

`<Tests: spectrum 84>`

`<Tests: resampling 85>`

`<Tests: color conversions 90>`

`<Tests: line sampler 91>`

`<Tests: Core Graphics bridge and filters 92>`

`<Tests: test patterns 93>`

Root chunk, tangled to Packages/LineScopeCore/Tests/LineScopeCoreTests/LineScopeCoreTests.swift.

`@testable` lets the tests reach internal members, such as the kernel function `Resampler.kernel` and `PixelBuffer`'s premultiplication helpers.

Several tests need an image whose content is known at every pixel. A gradient does the job: red grows with x , green with y , and blue is constant. So the pixel at (x, y) is $(x/W, y/H, \frac{1}{4})$, and a test can predict what any resampling or sampling of it should give.

83 `<Tests: gradient fixture 83> ≡`

```
private func gradient(width: Int, height: Int) -> PixelBuffer {
    var buf = PixelBuffer(width: width, height: height)
    for y in 0..
```

Used in chunk 82.

7.1 Spectrum

The first test feeds the analyzer a sine that completes exactly $k = 16$ periods in $n = 256$ samples. Since n is already a power of two, there is no padding and no leakage. All the energy must land in bin 16, with amplitude 1 under the single-sided normalization of Chapter 5, at frequency $16/256$ cycles per pixel. The last bin must sit at the Nyquist frequency $\frac{1}{2}$. The test proves the packing, the factor 2 and the frequency axis all at once.

The second test checks the other special bin. A constant 0.3 with the mean kept must show amplitude 0.3 in bin 0, which has no mirror image and so no factor 2. Its 100 samples also check that the FFT size is rounded up to 128.

84 $\langle \text{Tests: spectrum 84} \rangle \equiv$

```
final class SpectrumTests: XCTestCase {
    func testSinePeak() {
        let n = 256
        let k = 16
        let values = (0..

```

Used in chunk 82.

7.2 Resampling

The resampling tests check four properties.

85 $\langle \text{Tests: resampling 85} \rangle \equiv$

```
final class ResamplingTests: XCTestCase {
     $\langle \text{Tests: kernel shapes 86} \rangle$ 

     $\langle \text{Tests: identity at scale one 87} \rangle$ 

     $\langle \text{Tests: area average halves 88} \rangle$ 

     $\langle \text{Tests: constants stay constant 89} \rangle$ 
}
```

Used in chunk 82.

First, the kernels themselves. An *interpolating* kernel k has $k(0) = 1$ and $k(j) = 0$ at every nonzero integer j , so that resampling at the original positions returns the original samples. The tent, Catmull-Rom, Lanczos and sinc kernels all have this property. Mitchell-Netravali doesn't. It is *approximating*: with $B = C = \frac{1}{3}$, the formula of Chapter 3 gives

$$k(0) = \frac{6 - 2B}{6} = \frac{8}{9}, \quad k(1) = \frac{-B - 6C + 6B + 30C - 12B - 48C + 8B + 24C}{6} = \frac{B}{6} = \frac{1}{18},$$

and $k(2) = 0$ where the support ends. The value $k(\frac{1}{2}) = \frac{9}{16}$ of Catmull-Rom checks the cubic piece away from the integers. The last assertion checks that the sinc really is truncated at its radius.

86 *<Tests: kernel shapes 86>* $\equiv$

```
func testKernelShapes() {
    // Interpolating kernels are 1 at 0 and 0 at nonzero integers.
    for method in [ResampleMethod.tent, .catmullRom, .lanczos3, .sinc] {
        XCTAssertEqual(Resampler.kernel(method, 0), 1, accuracy: 1e-12, "\(method)")
        for k in 1...3 {
            XCTAssertEqual(Resampler.kernel(method, Double(k)), 0, accuracy: 1e-12, "\(method) at \(k)")
        }
    }
    // Mitchell-Netravali (B = C = 1/3) is approximating: k(0) = 8/9, k(1) = 1/18.
    XCTAssertEqual(Resampler.kernel(.mitchell, 0), 8.0 / 9, accuracy: 1e-12)
    XCTAssertEqual(Resampler.kernel(.mitchell, 1), 1.0 / 18, accuracy: 1e-12)
    XCTAssertEqual(Resampler.kernel(.mitchell, 2), 0, accuracy: 1e-12)
    XCTAssertEqual(Resampler.kernel(.catmullRom, 0.5), 0.5625, accuracy: 1e-12)
    // Sinc is truncated at its radius.
    XCTAssertEqual(Resampler.kernel(.sinc, Resampler.sincRadius + 0.5), 0)
}
```

Used in chunk 85.

Second, the consequence in whole images: at scale 1 every interpolating method must return the input unchanged. This test also exercises the pixel-center convention. With an off-by-half error in the mapping $(i + \frac{1}{2})/s$, the output would come out shifted and the test would fail. Mitchell is excluded, since by the previous test it slightly smooths even at scale 1.

87 *<Tests: identity at scale one 87>* $\equiv$

```
func testIdentityAtScaleOne() {
    let src = gradient(width: 17, height: 9)
    // Mitchell is approximating (slightly smoothing), so it is not an identity at x1.
    for method in ResampleMethod.allCases where method != .mitchell {
        let out = Resampler.resample(src, scale: 1, method: method)
        XCTAssertEqual(out.width, 17)
        XCTAssertEqual(out.height, 9)
        for (a, b) in zip(out.data, src.data) {
            XCTAssertEqual(a, b, accuracy: 1e-5, "\(method)")
        }
    }
}
```

Used in chunk 85.

Third, the area average at scale $\frac{1}{2}$ must average exact pairs of pixels: $\frac{0+1}{2} = 0.5$ and $\frac{0.2+0.6}{2} = 0.4$. This checks the footprint-coverage weights.

88 *<Tests: area average halves 88>* $\equiv$

```
func testAreaHalvesAverages() {
    var src = PixelBuffer(width: 4, height: 1)
    let values: [Float] = [0, 1, 0.2, 0.6]
    for (x, v) in values.enumerated() { src[x, 0] = RGBA(r: v, g: v, b: v) }
    let out = Resampler.resample(src, scale: 0.5, method: .area)
    XCTAssertEqual(out.width, 2)
    XCTAssertEqual(out[0, 0].r, 0.5, accuracy: 1e-5)
    XCTAssertEqual(out[1, 0].r, 0.4, accuracy: 1e-5)
}
```

Used in chunk 85.

Fourth, weights must sum to one for every output pixel, at every scale and with every method, including at the borders where the kernel is truncated and renormalized. The simplest observable consequence is that a constant image stays constant. The scales mix downscaling, upscaling, and a non-integer ratio.

89 *⟨Tests: constants stay constant 89⟩* ≡

```
func testConstantImageStaysConstant() {
    let src = PixelBuffer(width: 13, height: 7, fill: RGBA(r: 0.4, g: 0.5, b: 0.6))
    for method in ResampleMethod.allCases {
        for scale in [0.3, 0.5, 2.0, 3.7] {
            let out = Resampler.resample(src, scale: scale, method: method)
            for i in stride(from: 0, to: out.data.count, by: 4) {
                XCTAssertEqual(out.data[i], 0.4, accuracy: 1e-4, "\\(method) \\(scale)")
            }
        }
    }
}
```

Used in chunk 85.

7.3 Color conversions

Each conversion is checked by a round trip, color model and back, over a saturated color, the primaries, a nearly pure blue and a gray. The gray is the degenerate case where hue is undefined. One absolute value pins the hue convention: pure blue is at 240°. The CMYK round trip includes black and white, the two ends where K is 1 and 0.

90 *⟨Tests: color conversions 90⟩* ≡

```
final class ColorTests: XCTestCase {
    func testHSLRoundTrip() {
        for c in [RGBA(r: 0.2, g: 0.7, b: 0.4), RGBA(r: 1, g: 0, b: 0), RGBA(r: 0.1, g: 0.1, b: 0.9),
        ↪ RGBA(r: 0.5, g: 0.5, b: 0.5)] {
            let hsl = ColorConversion.hsl(from: c)
            let back = ColorConversion.rgb(h: hsl.h, s: hsl.s, l: hsl.l)
            XCTAssertEqual(back.r, c.r, accuracy: 1e-5)
            XCTAssertEqual(back.g, c.g, accuracy: 1e-5)
            XCTAssertEqual(back.b, c.b, accuracy: 1e-5)
        }
        XCTAssertEqual(ColorConversion.hsl(from: RGBA(r: 0, g: 0, b: 1)).h, 240, accuracy: 1e-4)
    }

    func testCMYKRoundTrip() {
        for c in [RGBA(r: 0.2, g: 0.7, b: 0.4), RGBA(r: 0, g: 0, b: 0), RGBA(r: 1, g: 1, b: 1)] {
            let v = ColorConversion.cmyk(from: c)
            let back = ColorConversion.rgb(c: v.c, m: v.m, y: v.y, k: v.k)
            XCTAssertEqual(back.r, c.r, accuracy: 1e-5)
            XCTAssertEqual(back.g, c.g, accuracy: 1e-5)
            XCTAssertEqual(back.b, c.b, accuracy: 1e-5)
        }
    }
}
```

Used in chunk 82.

7.4 Line sampling

The line sampler has two promises. First, the number of samples follows the image's own grid: $[L] + 1$ gives 201 samples on a 200-pixel-wide image and 101 on its half-size copy, with a spacing of exactly one pixel. This is what makes a shrunk copy show less detail in its profile. Second, the samples come from the right place. A vertical line at $u = 0.505$ passes through the center of pixel column 50 of a 100-pixel image, so red is exactly 50/100 along the whole line while green increases.

91 *⟨Tests: line sampler 91⟩* ≡

```
final class LineSamplerTests: XCTestCase {
    func testSampleCountScalesWithResample() {
        let src = gradient(width: 200, height: 100)
        let half = Resampler.resample(src, scale: 0.5, method: .tent)
        let a = LineSampler.sample(src, along: .initial)
        let b = LineSampler.sample(half, along: .initial)
        XCTAssertEqual(a.colors.count, 201)
        XCTAssertEqual(b.colors.count, 101)
        XCTAssertEqual(a.spacing, 1, accuracy: 1e-9)
    }

    func testSamplesFollowContent() {
        let src = gradient(width: 100, height: 100)
        let line = LineSegment(start: .init(x: 0.505, y: 0.1), end: .init(x: 0.505, y: 0.9))
        let s = LineSampler.sample(src, along: line)
        // Vertical line through pixel column 50: red is constant, green increases.
        XCTAssertEqual(s.colors.first!.r, 0.5, accuracy: 1e-4)
        XCTAssertEqual(s.colors.last!.r, 0.5, accuracy: 1e-4)
        XCTAssertEqual(s.colors.first!.g, s.colors.last!.g)
    }
}
```

Used in chunk 82.

7.5 Core Graphics and filters

The conversion to an 8-bit `CGImage` and back may lose at most one quantization step, $\frac{1}{255}$, and must keep the orientation and the size. A flipped row order would change every value of the gradient, far more than the tolerance.

The filters are checked less strictly. They come from different sources (Core Image and the CPU kernels), and the property that the rest of the program depends on is that they *keep the image size*: the normalized line relies on it. One qualitative check makes sure the Core Image path doesn't scramble the image: a blurred horizontal gradient still increases from left to right.

92 *⟨Tests: Core Graphics bridge and filters 92⟩* ≡

```
final class CGBridgeTests: XCTestCase {
    func testCGImageRoundTrip() throws {
        let src = gradient(width: 32, height: 16)
        let cg = try XCTUnwrap(src.makeCGImage())
        let back = try XCTUnwrap(PixelBuffer(cgImage: cg))
        XCTAssertEqual(back.width, 32)
        XCTAssertEqual(back.height, 16)
        for (a, b) in zip(back.data, src.data) {
            XCTAssertEqual(a, b, accuracy: 1.0 / 255 + 1e-4)
        }
    }

    func testFiltersKeepSize() {
        let src = gradient(width: 24, height: 12)
        let filters: [FilterKind] = [.gaussianBlur(radius: 2), .boxBlur(radius: 2), .median,
        ↪ .unsharpMask(radius: 2, intensity: 0.5),
        ↪ .noiseReduction(level: 0.02, sharpness: 0.4), .sobel, .laplacian,
        ↪ .emboss, .grayscale, .invert]
        for f in filters {
            let out = Filters.apply(f, to: src)
            XCTAssertEqual(out.width, 24, f.displayName)
            XCTAssertEqual(out.height, 12, f.displayName)
        }
        let blurred = Filters.apply(.gaussianBlur(radius: 1), to: src)
        // Horizontal gradient stays roughly monotone and in range after blur.
    }
}
```

```

        XCTAssertLessThan(blurred[2, 6].r, blurred[20, 6].r)
    }
}

```

Used in chunk 82.

7.6 Test patterns

Every pattern must honor its requested size and stay in $[0, 1]$. The range check caught a real bug: the hue ramp produced tiny negative components from rounding in the HSL conversion, which is why that pattern clamps (Chapter 6).

The sine grating test is an end-to-end check of the whole analysis pipeline: generate a grating of 0.125 cycles per pixel, sample it along the default line, and take the Hann-windowed spectrum. The peak must be within one bin of 0.125. Here generator, sampler and analyzer must all agree about pixel centers and frequency units.

The last test checks two exact facts: the checkerboard cells alternate at the right place, and the same seed gives the same noise, which window restoration relies on.

93 $\langle$ Tests: test patterns 93 $\rangle \equiv$

```

final class TestPatternTests: XCTestCase {
    func testAllPatternsGenerateRequestedSize() {
        for kind in TestPatternKind.allCases {
            let buf = TestPatternGenerator.make(TestPatternSpec(kind: kind, width: 64, height: 48))
            XCTAssertEqual(buf.width, 64, kind.displayName)
            XCTAssertEqual(buf.height, 48, kind.displayName)
            XCTAssertTrue(buf.data.allSatisfy { $0 >= 0 && $0 <= 1 }, kind.displayName)
        }
    }

    func testSineGratingPeaksAtItsFrequency() {
        let spec = TestPatternSpec(kind: .sineGrating, width: 256, height: 8, parameter: 0.125)
        let samples = LineSampler.sample(TestPatternGenerator.make(spec), along: .initial)
        let values = samples.colors.map(\.r)
        let s = SpectrumAnalyzer.compute(values, spacing: samples.spacing, window: .hann, removeMean:
↳ true)
        let peak = try! XCTUnwrap(s.peakIndex)
        XCTAssertEqual(s.cyclesPerPixel[peak], 0.125, accuracy: 1.0 / Double(s.fftSize) + 1e-9)
    }

    func testCheckerboardAndNoiseDeterminism() {
        let board = TestPatternGenerator.make(TestPatternSpec(kind: .checkerboard, width: 8, height: 8,
↳ parameter: 2))
        XCTAssertEqual(board[0, 0].r, 1)
        XCTAssertEqual(board[2, 0].r, 0)
        XCTAssertEqual(board[2, 2].r, 1)
        let a = TestPatternGenerator.make(TestPatternSpec(kind: .whiteNoise, width: 16, height: 16,
↳ parameter: 7))
        let b = TestPatternGenerator.make(TestPatternSpec(kind: .whiteNoise, width: 16, height: 16,
↳ parameter: 7))
        XCTAssertEqual(a, b)
    }
}

```

Used in chunk 82.

Chapter 8

Documents and the Session Model

With the numeric core in place, we turn to the app. This chapter describes the data the user interface is built on: the scenes that make up the app, the document that brings an image in, the small value types that describe images, zones and panes, and the `Session`, the one object per window that holds everything the views show.

8.1 The app and its scenes

A SwiftUI app is a list of *scenes*, each a kind of window. `LineScope` has four.

Image documents

One window per opened image file, supplied by `DocumentGroup` in its *viewing* form: the app opens documents but never creates or saves them, so there is no New or Save command to disable.

Test patterns

One window per generated pattern (Chapter 6). A `WindowGroup` keyed by a `TestPatternSpec` value opens a new window for each spec; because the spec is `Codable`, macOS can restore these windows after a relaunch.

Pattern generator

A single window for choosing a pattern and its parameters.

Inspector

A single window describing the image in focus, placed at the top right.

The menus of Chapter 10 are attached to the document scene, which puts them in the app's menu bar.

94 `<LineScope/LineScopeApp.swift 94> ≡`

```
import LineScopeCore
import SwiftUI

@main
struct LineScopeApp: App {
    var body: some Scene {
        <LineScopeApp: document windows 95>

        <LineScopeApp: test-pattern windows 96>

        <LineScopeApp: pattern generator window 97>

        <LineScopeApp: inspector window 98>
    }
}
```

Root chunk, tangled to LineScope/LineScopeApp.swift.

A document window gets the document and a name to show and export under. The file name falls back to “Untitled” for documents without a URL.

```
95  <LineScopeApp: document windows 95> ≡
    DocumentGroup(viewing: ImageDocument.self) { file in
        MainView(document: file.document, fileName: file.fileURL?.lastPathComponent ?? "Untitled")
    }
    .defaultSize(width: 1100, height: 900)
    .commands { LineScopeCommands() }
```

Used in chunk 94.

The window group passes a binding to an optional spec, which is nil only in the moment before SwiftUI has decoded a restored window’s value; nothing is shown then.

```
96  <LineScopeApp: test-pattern windows 96> ≡
    WindowGroup("Test Pattern", for: TestPatternSpec.self) { $spec in
        if let spec {
            PatternWindow(spec: spec)
        }
    }
    .defaultSize(width: 1100, height: 900)
```

Used in chunk 94.

```
97  <LineScopeApp: pattern generator window 97> ≡
    Window("New Test Pattern", id: TestPatternGeneratorView.windowID) {
        TestPatternGeneratorView()
    }
    .windowResizability(.contentSize)
```

Used in chunk 94.

```
98  <LineScopeApp: inspector window 98> ≡
    Window("Inspector", id: InspectorView.windowID) {
        InspectorView()
    }
    .defaultSize(width: 340, height: 760)
    .defaultPosition(.topTrailing)
```

Used in chunk 94.

8.2 The image document

A document is decoded once, when it is opened, into three things: the float `PixelBuffer` all analysis works on, a `CGImage` made from that buffer for display, and a record of facts about the file. Displaying the buffer’s own rendering, not the decoded file image, guarantees that what the user sees is exactly the data being analyzed.

```
99  <LineScope/Document/ImageDocument.swift 99> ≡
    import ImageIO
    import LineScopeCore
    import SwiftUI
    import UniformTypeIdentifiers
```

<ImageDocument: source information 100>

<ImageDocument: the document type 101>

Root chunk, tangled to LineScope/Document/ImageDocument.swift.

The facts are gathered before the conversion to 8-bit sRGB can hide them (Chapter 2): the original bit depth, color space and alpha. The inspector shows them, so a user who opened a 16-bit or CMYK file knows it was converted.

100 *<ImageDocument: source information 100> ≡*

/// Facts about the file as decoded, before conversion to the working float buffer.

```
struct SourceInfo {
    var fileType: String
    var pixelWidth: Int
    var pixelHeight: Int
    var bitsPerComponent: Int
    var bitsPerPixel: Int
    var colorSpaceName: String
    var hasAlpha: Bool
    var dpi: Double?
}
```

Used in chunk 99.

The document accepts any image type ImageIO can read. It is a value type holding immutable data, which is all FileDocument needs for viewing.

101 *<ImageDocument: the document type 101> ≡*

*/// Read-only image document. Pixels are decoded once into a `PixelBuffer`; EXIF orientation is ignored
/// on purpose so analysis runs on the stored pixel grid.*

```
struct ImageDocument: FileDocument {
    static var readableContentTypes: [UTType] { [.image] }

    let buffer: PixelBuffer
    let displayImage: CGImage
    let info: SourceInfo
}
```

<ImageDocument: reading a file 102>

<ImageDocument: a generated test pattern 105>

<ImageDocument: refusing to write 106>

```
}
```

Used in chunk 99.

Reading takes the first image of the file, converts it and renders it back. Any failure along the chain is reported as a corrupt file, which makes the system show its standard error alert.

One decision is visible only by its absence: the EXIF orientation tag is *not* applied. A photo taken in portrait may therefore appear rotated. Applying the tag would mean rotating or mirroring the pixel grid, and LineScope's purpose is to analyze the grid exactly as stored.

102 *<ImageDocument: reading a file 102> ≡*

```
init(configuration: ReadConfiguration) throws {
    <ImageDocument: decoding the first image 103>

    <ImageDocument: recording facts about the source 104>
}
```

Used in chunk 101.

103 *<ImageDocument: decoding the first image 103> ≡*

```
guard
  let data = configuration.file.regularFileContents,
  let source = CGImageSourceCreateWithData(data as CFData, nil),
  let cg = CGImageSourceCreateImageAtIndex(source, 0, nil),
  let buffer = PixelBuffer(cgImage: cg),
  let display = buffer.makeCGImage()
else { throw CocoaError(.fileReadCorruptFile) }
```

Used in chunk 102.

The file type comes from ImageIO’s type identifier, turned into a readable description such as “PNG image”. The color space name loses its `kCGColorSpace` prefix. An image has alpha unless its alpha info says the fourth channel is absent or to be skipped.

104 *<ImageDocument: recording facts about the source 104> ≡*

```
let props = CGImageSourceCopyPropertiesAtIndex(source, 0, nil) as? [CFString: Any]
let type = (CGImageSourceGetType(source) as String?).flatMap { UTType($0)?.localizedDescription }
let alpha = cg.alphaInfo
self.buffer = buffer
self.displayImage = display
self.info = SourceInfo(
  fileType: type ?? "Image",
  pixelWidth: cg.width,
  pixelHeight: cg.height,
  bitsPerComponent: cg.bitsPerComponent,
  bitsPerPixel: cg.bitsPerPixel,
  colorSpaceName: (cg.colorSpace?.name as String?).replacingOccurrences(of: "kCGColorSpace", with:
    ↪ "") ?? "Unknown",
  hasAlpha: !(alpha == .none || alpha == .noneSkipFirst || alpha == .noneSkipLast),
  dpi: props?[kCGImagePropertyDPIWidth] as? Double
)
```

Used in chunk 102.

A generated pattern becomes a document too, so that it can be shown in exactly the same window as a file. Its facts describe the float buffer itself: 32 bits per component, 128 per pixel, and no alpha.

105 *<ImageDocument: a generated test pattern 105> ≡*

```
/// A generated test pattern, shown in the same analysis window as a file.
init?(pattern spec: TestPatternSpec) {
  let buffer = TestPatternGenerator.make(spec)
  guard let display = buffer.makeCGImage() else { return nil }
  self.buffer = buffer
  self.displayImage = display
  self.info = SourceInfo(
    fileType: "Generated test pattern",
    pixelWidth: buffer.width,
    pixelHeight: buffer.height,
    bitsPerComponent: 32,
    bitsPerPixel: 128,
    colorSpaceName: "sRGB (float)",
    hasAlpha: false,
    dpi: nil
  )
}
```

Used in chunk 101.

`FileDocument` requires a way to write, even for a viewer. It is never called, because the viewing document group offers no Save command; derived images leave the program through Export instead (Section 8.4.5).

106 *<ImageDocument: refusing to write 106> ≡*

```
func fileWrapper(configuration: WriteConfiguration) throws -> FileWrapper {
    throw CocoaError(.featureUnsupported)
}
```

Used in chunk 101.

8.3 Variants, zones and panes

The next file holds the small types the session is made of.

107 *<LineScope/Model/ImageVariant.swift 107> ≡*

```
import CoreGraphics
import Foundation
import LineScopeCore
```

<ImageVariant: the variant type 108>
<ImageVariant: zones 109>
<ImageVariant: panes 110>
<ImageVariant: the frequency axis 111>
<ImageVariant: operation requests 112>
Root chunk, tangled to LineScope/Model/ImageVariant.swift.

A *variant* is the original image or one copy derived from it. Besides its pixels and their rendering, it remembers its parent and the operation that produced it, and the whole *lineage*: the operations from the original down to this copy. Storing the lineage, rather than walking parent links, keeps a variant's history intact when an intermediate tab is closed. A variant's name, shown on its tab, is the lineage joined with arrows.

Variants are immutable. An operation never changes a variant; it creates a new one.

108 *<ImageVariant: the variant type 108> ≡*

```
/// The original image or one derived copy. Each copy records the chain of operations back to the
    ↪ original.
struct ImageVariant: Identifiable {
    let id = UUID()
    let buffer: PixelBuffer
    let cgImage: CGImage
    let parentID: UUID?
    let operation: ImageOperation?
    /// Operations applied to the original, in order. Empty for the original.
    let lineage: [ImageOperation]

    var isOriginal: Bool { operation == nil }

    var name: String {
        lineage.isEmpty ? "Original" : lineage.map(\.displayName).joined(separator: " - ")
    }
}
```

Used in chunk 107.

The window is divided into three horizontal *zones*. The raw values index the session's pane arrays.

```

109 <ImageVariant: zones 109> ≡
    /// Which kind of content a zone shows.
    enum Zone: Int, CaseIterable, Identifiable {
        case images, profile, spectrum

        var id: Int { rawValue }

        var title: String {
            switch self {
            case .images: "Images"
            case .profile: "Line Profile"
            case .spectrum: "Spectrum"
            }
        }
    }
}

```

Used in chunk 107.

Each zone can be split into several side-by-side *panes*, each showing one variant. A pane has its own identity, separate from the variant it shows, so SwiftUI keeps a pane’s view state (such as its scroll position) when the pane switches to another variant.

```

110 <ImageVariant: panes 110> ≡
    /// One side-by-side pane inside a zone; it shows one variant at a time.
    struct PaneState: Identifiable, Equatable {
        let id = UUID()
        var variantID: UUID
    }
}

```

Used in chunk 107.

The spectrum can be plotted against two frequency scales, discussed with the analysis in Chapter 5: cycles per pixel of the variant itself, or cycles along the whole line, which is the same for every copy.

```

111 <ImageVariant: the frequency axis 111> ≡
    enum FrequencyAxis: String, CaseIterable, Identifiable {
        case cyclesPerPixel
        case cyclesPerLine

        var id: String { rawValue }
        var displayName: String { self == .cyclesPerPixel ? "cycles/pixel" : "cycles/line" }
    }
}

```

Used in chunk 107.

Some menu commands need parameters before they can run: a blur radius, a scale factor. Such a command doesn’t run the operation; it stores an `OperationRequest` in the session, and the window presents a sheet for it. The request is `Identifiable` because SwiftUI’s `sheet(item:)` requires it.

```

112 <ImageVariant: operation requests 112> ≡
    /// Operations that need parameters before running; shown as a sheet.
    enum OperationRequest: Identifiable {
        case gaussianBlur, boxBlur, unsharpMask, noiseReduction
        case resample(ResampleMethod)

        var id: String {
            switch self {
            case .gaussianBlur: "gaussian"
            case .boxBlur: "box"
            case .unsharpMask: "unsharp"
            case .noiseReduction: "noise"
            case .resample(let m): "resample-\(m.rawValue)"
            }
        }
    }
}

```

```

    }

    var title: String {
        switch self {
            case .gaussianBlur: "Gaussian Blur"
            case .boxBlur: "Box Blur"
            case .unsharpMask: "Sharpen (Unsharp Mask)"
            case .noiseReduction: "Noise Reduction"
            case .resample(let m): "Resample: \(m.displayName)"
        }
    }
}

```

Used in chunk 107.

8.4 The session

Every window has one `Session`. It owns the variants, the shared line, the analysis settings, the pane layout and the focus, and it performs operations. The views hold no state of their own beyond transient gestures; they read the session and write to it.

The class is marked `@Observable`. SwiftUI records which properties each view's body reads, and re-renders exactly those views when one of the properties changes. This is what keeps the line in sync everywhere: dragging it in one pane writes `line`, and every pane in all three zones read `line` while drawing, so all of them update in the same frame, with no notification code. The class is also `@MainActor`: all its state is touched only on the main thread, and background work has to hop back to it explicitly.

113 *<LineScope/Model/Session.swift 113> ≡*

```

import AppKit
import ImageIO
import LineScopeCore
import SwiftUI
import UniformTypeIdentifiers

```

<Session: profile and spectrum point types 114>

<Session: the session class 115>

<Session: the focused-value entry 129>

<Session: the active session 130>

Root chunk, tangled to LineScope/Model/Session.swift.

The views plot two kinds of series. A profile keeps the raw channel values (for the spectrum and the inspector's statistics), the samples they came from, and a list of points for the chart. The points have integer identifiers because Swift Charts identifies marks, and the sample index is a natural, stable key.

114 *<Session: profile and spectrum point types 114> ≡*

```

struct Profile {
    struct Point: Identifiable {
        let id: Int
        let x: Double
        let y: Double
    }

    var points: [Point]
    var values: [Float]
    var samples: LineSamples
}

```

```

struct SpectrumPoint: Identifiable {
  let id: Int
  let frequency: Double
  let magnitude: Double
}

```

Used in chunk 113.

115 $\langle$ Session: the session class 115 $\rangle \equiv$

/// All state for one document window: the variants, the shared line, and view settings.

```

@MainActor @Observable
final class Session {
   $\langle$ Session: stored state 116 $\rangle$ 

   $\langle$ Session: initialization 120 $\rangle$ 

   $\langle$ Session: looking up variants 121 $\rangle$ 

  // MARK: Panes

   $\langle$ Session: selecting and splitting panes 122 $\rangle$ 

  // MARK: Variants

   $\langle$ Session: closing a variant 123 $\rangle$ 

   $\langle$ Session: performing an operation in the background 124 $\rangle$ 

   $\langle$ Session: showing a new variant 125 $\rangle$ 

  // MARK: Analysis

   $\langle$ Session: computing a profile 126 $\rangle$ 

   $\langle$ Session: computing a spectrum 127 $\rangle$ 

  // MARK: Export

   $\langle$ Session: exporting the focused variant 128 $\rangle$ 
}

```

Used in chunk 113.

8.4.1 State

116 $\langle$ Session: stored state 116 $\rangle \equiv$

$\langle$ Session: identity, variants and the line 117 $\rangle$

$\langle$ Session: analysis settings 118 $\rangle$

$\langle$ Session: panes, focus and pending work 119 $\rangle$

Used in chunk 115.

The variants can only be changed by the session's own methods, which maintain the invariant that index 0 is the original and is never removed.

The line is stored in *normalized* coordinates, $(u, v) \in [0, 1]^2$ relative to the image extent. A filtered copy has the same size as its source and a resampled copy is only scaled, so the same normalized line lands on the same content in every copy, however large it is. This is how a single

line can be “in the same position of all images, after considering filtering and resampling”: each pane converts it to its own pixel grid only when it draws or samples.

117 *⟨Session: identity, variants and the line 117⟩* ≡

```
let fileName: String
let sourceInfo: SourceInfo

private(set) var variants: [ImageVariant]
var line: LineSegment = .initial

Used in chunk 116.
```

The analysis settings are shared by all panes, deliberately: comparing the red channel of one copy with the hue of another, or a Hann-windowed spectrum with a rectangular one, would be misleading. The one subtle setting is the channel index. The color models have different numbers of channels (RGBA and CMYK have four, HSL three), so switching the model resets an index that would fall out of range.

118 *⟨Session: analysis settings 118⟩* ≡

```
var colorModel: ColorModel = .rgba {
  didSet { if channelIndex >= colorModel.channels.count { channelIndex = 0 } }
}
var channelIndex = 0
var window: WindowFunction = .hann
var removeMean = true
var logMagnitude = true
var frequencyAxis: FrequencyAxis = .cyclesPerPixel
var useDevicePixels = false

Used in chunk 116.
```

The *focused variant* is the one menu operations apply to and the inspector describes: the last one the user clicked, in any zone. Applying operations to it, rather than always to the original, is what lets operations chain: blur a copy, then downsample the blurred copy.

pendingRequest drives the parameter sheet, runningOperations the progress indicator, and errorMessage an alert.

119 *⟨Session: panes, focus and pending work 119⟩* ≡

```
/// Panes per zone, indexed by `Zone.rawValue`.
var panes: [[PaneState]]
/// The variant operations apply to and the inspector describes.
var focusedVariantID: UUID
var pendingRequest: OperationRequest?
private(set) var runningOperations = 0
var errorMessage: String?

Used in chunk 116.
```

A new session has just the original, focused, shown in one pane of each zone.

120 *⟨Session: initialization 120⟩* ≡

```
init(document: ImageDocument, fileName: String) {
  let original = ImageVariant(
    buffer: document.buffer, cgImage: document.displayImage,
    parentID: nil, operation: nil, lineage: []
  )
  self.fileName = fileName
  self.sourceInfo = document.info
  self.variants = [original]
  self.focusedVariantID = original.id
  self.panes = Zone.allCases.map { _ in [PaneState(variantID: original.id)] }
}

Used in chunk 115.
```

Looking a variant up by identifier is a linear search. A session holds a handful of variants, and an array keeps the tab order for free. If the focused variant has disappeared, the original stands in for it, so `focusedVariant` never fails.

121 *⟨Session: looking up variants 121⟩* ≡

```
var original: ImageVariant { variants[0] }

var focusedVariant: ImageVariant {
    variant(focusedVariantID) ?? original
}

func variant(_ id: UUID) -> ImageVariant? {
    variants.first { $0.id == id }
}

var channelName: String { colorModel.channels[channelIndex] }
Used in chunk 115.
```

8.4.2 Panes

Selecting a tab in a pane both shows that variant there and focuses it. Splitting a zone appends a pane showing the first variant not yet visible in that zone, which is usually what the user wants to compare with; if every variant is already shown, the new pane shows the focused one. Unsplitting removes the last pane, never the only one.

122 *⟨Session: selecting and splitting panes 122⟩* ≡

```
func select(_ variantID: UUID, zone: Zone, pane: Int) {
    panes[zone.rawValue][pane].variantID = variantID
    focusedVariantID = variantID
}

func split(_ zone: Zone) {
    let shown = Set(panes[zone.rawValue].map(\.variantID))
    let next = variants.first { !shown.contains($0.id) } ?? focusedVariant
    panes[zone.rawValue].append(PaneState(variantID: next.id))
}

func unsplit(_ zone: Zone) {
    guard panes[zone.rawValue].count > 1 else { return }
    panes[zone.rawValue].removeLast()
}
Used in chunk 115.
```

8.4.3 Variants and operations

Closing a variant removes its tab. The original can't be closed. Panes that showed the closed variant fall back to the original, and so does the focus. Variants derived from the closed one keep working, since each carries its own lineage.

123 *⟨Session: closing a variant 123⟩* ≡

```
func closeVariant(_ id: UUID) {
    guard let index = variants.firstIndex(where: { $0.id == id }), index > 0 else { return }
    variants.remove(at: index)
    for z in panes.indices {
        for p in panes[z].indices where panes[z][p].variantID == id {
            panes[z][p].variantID = original.id
        }
    }
    if focusedVariantID == id { focusedVariantID = original.id }
}
```

Used in chunk 115.

Performing an operation must not freeze the window: resampling a large photograph with a wide kernel takes a noticeable fraction of a second. The session captures the source variant and its buffer on the main actor, then starts a task. The task itself inherits the main actor, but the heavy call runs in a *detached* task at user-initiated priority, which runs on a background thread. This is safe because `apply` is a pure function of a `Sendable` buffer (Section 4.2). When the result comes back, the rest of the task runs on the main actor again: it renders the image, builds the variant with its extended lineage, appends it and shows it.

The source is captured *before* the task starts, so the operation applies to the variant that was focused when the user chose the command, even if the focus moves while it is running.

124 *⟨Session: performing an operation in the background 124⟩* ≡

/// Applies an operation to the focused variant in the background and adds the result as a new tab.

```
func perform(_ operation: ImageOperation) {
    let source = focusedVariant
    let input = source.buffer
    runningOperations += 1
    Task {
        let output = await Task.detached(priority: .userInitiated) {
            operation.apply(to: input)
        }.value
        runningOperations -= 1
        guard let cg = output.makeCGImage() else {
            errorMessage = "Could not render the result of \(operation.displayName)."
            return
        }
        let variant = ImageVariant(
            buffer: output, cgImage: cg, parentID: source.id,
            operation: operation, lineage: source.lineage + [operation]
        )
        variants.append(variant)
        show(variant.id)
    }
}
```

Used in chunk 115.

Where should a new variant appear? When a zone is split, the user is usually comparing: the left pane holds the baseline and the right pane the candidate. So the new variant goes into the *last* pane of each zone, which is also the only pane when the zone is not split. It becomes the focus, so the next operation chains from it.

125 *⟨Session: showing a new variant 125⟩* ≡

/// Shows a new variant in every zone: in the last pane when split (so the left pane keeps the comparison baseline), otherwise in the only pane.

```
private func show(_ id: UUID) {
    for z in panes.indices {
        panes[z][panes[z].count - 1].variantID = id
    }
    focusedVariantID = id
}
```

Used in chunk 115.

8.4.4 Analysis

A profile samples the line in the variant's own pixel grid (Chapter 5) and extracts the chosen channel of the chosen color model from every sample. The chart's *x* coordinate is the normalized position along the line, not the sample index. A half-size copy has half as many samples, and with positions in `[0, 1]` its profile still lines up with the original's.

These functions are called from view bodies, so they run again on every change of the line or the settings. That is cheap: a few thousand bilinear samples and one FFT.

126 *⟨Session: computing a profile 126⟩* ≡

```
func profile(for variant: ImageVariant) -> Profile {
    let samples = LineSampler.sample(variant.buffer, along: line)
    let values = samples.colors.map {
        ColorConversion.value(of: $0, model: colorModel, channel: channelIndex)
    }
    let points = values.indices.map { Profile.Point(id: $0, x: samples.positions[$0], y:
        ↪ Double(values[$0])) }
    return Profile(points: points, values: values, samples: samples)
}
```

Used in chunk 115.

The spectrum of a profile is computed with the current window and mean setting. The sample spacing is guarded away from zero, which it would be for a line of zero length. For plotting, each bin gets its frequency on the chosen axis, and its amplitude, optionally in decibels,

$$A_{\text{dB}} = \max(20 \log_{10} A, -100).$$

The floor keeps empty bins, whose logarithm would be $-\infty$, on the chart.

127 *⟨Session: computing a spectrum 127⟩* ≡

```
func spectrum(for profile: Profile) -> SpectrumResult {
    SpectrumAnalyzer.compute(
        profile.values, spacing: max(profile.samples.spacing, 1e-9),
        window: window, removeMean: removeMean
    )
}

func spectrumPoints(_ s: SpectrumResult) -> [SpectrumPoint] {
    let freqs = frequencyAxis == .cyclesPerPixel ? s.cyclesPerPixel : s.cyclesPerLine
    return s.amplitudes.indices.map { i in
        let a = s.amplitudes[i]
        return SpectrumPoint(
            id: i, frequency: freqs[i],
            magnitude: logMagnitude ? max(20 * log10(max(a, 1e-12)), Self.dBFloor) : a
        )
    }
}
```

```
static let dBFloor = -100.0
```

Used in chunk 115.

8.4.5 Export

Exporting writes the focused variant as a PNG. The proposed name combines the file's base name with the variant's lineage, with the arrows and slashes that can't appear in a file name replaced. The image written is the variant's display image, which is exactly what is on screen: 8-bit, sRGB.

128 *⟨Session: exporting the focused variant 128⟩* ≡

```
func exportFocused() {
    let variant = focusedVariant
    let panel = NSSavePanel()
    panel.allowedContentTypes = [.png]
    let base = (fileName as NSString).deletingPathExtension
    let suffix = variant.isOriginal ? "original" : variant.name
        .replacingOccurrences(of: " → ", with: "_")
        .replacingOccurrences(of: "/", with: "-")
    panel.nameFieldStringValue = "\(base)-\(suffix).png"
```

```

guard panel.runModal() == .OK, let url = panel.url else { return }
guard
    let dest = CGImageDestinationCreateWithURL(url as CFURL, UTType.png.identifier as CFString, 1,
    ↪ nil)
else {
    errorMessage = "Could not create \(url.lastPathComponent)."
    return
}
CGImageDestinationAddImage(dest, variant.cgImage, nil)
if !CGImageDestinationFinalize(dest) {
    errorMessage = "Could not write \(url.lastPathComponent)."
}
}

```

Used in chunk 115.

8.4.6 Finding the session from outside the window

Menus must act on the session of the window the user is working in. SwiftUI's *focused values* provide exactly this: a window publishes its session, and the menu commands read the value of the key window. The entry is declared with the `@Entry` macro.

129 *<Session: the focused-value entry 129> ≡*

```

extension FocusedValues {
    @Entry var session: Session?
}

```

Used in chunk 113.

Focused values fail in one case, and it matters here: the inspector. While the user interacts with the inspector window, *it* is the key window, and it has no session, so a focused value would be `nil` just when the inspector needs it. The app therefore also keeps track of the session of the most recently key *document* window, in a shared observable object. Document windows update it as they become key (Chapter 9), the inspector reads it, and the menus use it as a fallback.

130 *<Session: the active session 130> ≡*

```

/// The session of the most recently key document window, for the inspector window (which is itself
/// key while being used, so focused values would not reach it).
@MainActor @Observable
final class ActiveSession {
    static let shared = ActiveSession()
    var session: Session?
}

```

Used in chunk 113.

Chapter 9

Views

The views are deliberately thin. Every number they show comes from the `Session` of Chapter 8: a view asks it for the variants, the shared line, a profile or a spectrum, and draws the answer. What the views do own is *interaction*: which pane shows which variant, how a drag on an image turns into a new line, and how the controls in each zone header write back into the session. Because the session is an observable object, writing the line from one pane is enough to redraw every image, profile and spectrum in the window.

The chapter follows the window from the outside in: the window and its three zones, the panes and their tabs, the image canvas where the line is drawn, the two charts, and then the auxiliary windows (the parameter sheet, the inspector and the test-pattern windows).

9.1 The document window

A document window is three zones stacked vertically: images, line profile, spectrum. The window owns one `Session`, created from the document when the window opens.

131 *<LineScope/Views/MainView.swift 131> ≡*

```
import LineScopeCore
import SwiftUI

struct MainView: View {
    <MainView: the session is created once per window 132>

    var body: some View {
        <MainView: three zones stacked vertically 133>
        <MainView: publishing the session to menus and inspector 134>
        <MainView: sheets, alerts and the progress indicator 135>
    }
}
```

<MainView.swift: the window observer 136>

Root chunk, tangled to LineScope/Views/MainView.swift.

SwiftUI may call a view's initializer many times, but a `@State` property is initialized only the first time; later values passed to `State(initialValue:)` are ignored. That is exactly the lifetime we want: one session per window, surviving every redraw.

132 *<MainView: the session is created once per window 132> ≡*

```
@State private var session: Session

init(document: ImageDocument, fileName: String) {
    _session = State(initialValue: Session(document: document, fileName: fileName))
}
```

Used in chunk 131.

The zones sit in a `VSplitView`, so the user can drag the dividers between them. The ideal heights give the images most of the room at first; the minimum sizes keep every zone's header and at least a sliver of its content visible.

133 *⟨MainView: three zones stacked vertically 133⟩* ≡

```
VSplitView {
    ZoneView(zone: .images, session: session)
        .frame(minHeight: 180, idealHeight: 420)
    ZoneView(zone: .profile, session: session)
        .frame(minHeight: 140, idealHeight: 220)
    ZoneView(zone: .spectrum, session: session)
        .frame(minHeight: 140, idealHeight: 220)
}
.frame(minWidth: 720, minHeight: 560)
```

Used in chunk 131.

Two other parts of the app need to find this window's session. The menus (Chapter 10) read it as a *focused scene value*, which SwiftUI provides for the key window. The inspector cannot do that. It is a window of its own, and while the user works in it *it* is the key window, so the focused value would vanish just when it is needed. Instead, the document window reports itself to the shared `ActiveSession` whenever it becomes key, and withdraws when it closes (but only if it is still the active one, since another window may have become key in between).

134 *⟨MainView: publishing the session to menus and inspector 134⟩* ≡

```
.focusedSceneValue(\.session, session)
.background(WindowKeyObserver(
    onBecomeKey: { ActiveSession.shared.session = session },
    onClose: {
        if ActiveSession.shared.session === session { ActiveSession.shared.session = nil }
    }
))
```

Used in chunk 131.

The session also drives three transient pieces of UI. Setting its `pendingRequest` presents the parameter sheet (Section 9.6). Setting its `errorMessage` shows an alert, and dismissing the alert clears the message again through the hand-made binding. While background operations run, a small spinner appears in the toolbar.

135 *⟨MainView: sheets, alerts and the progress indicator 135⟩* ≡

```
.sheet(item: $session.pendingRequest) { request in
    ParameterSheet(request: request, session: session)
}
.alert(
    "Operation failed",
    isPresented: Binding(get: { session.errorMessage != nil }, set: { if !$0 { session.errorMessage =
        ↪ nil } }),
    presenting: session.errorMessage
) { _ in
    Button("OK") {}
} message: { Text($0) }
.toolbar {
    ToolbarItem(placement: .status) {
        if session.runningOperations > 0 {
            ProgressView().controlSize(.small)
        }
    }
}
```

Used in chunk 131.

SwiftUI has no notification for “this window became key”. So we place an invisible AppKit view in the background of the window’s content. An `NSView` knows its `NSWindow`, and through it can observe the window’s notifications. The representable just forwards the two callbacks.

136 *⟨MainView.swift: the window observer 136⟩* ≡

```
/// Reports when the hosting window becomes key or closes.
struct WindowKeyObserver: NSViewRepresentable {
    var onBecomeKey: () -> Void
    var onClose: () -> Void

    func makeNSView(context: Context) -> ObserverView { ObserverView() }

    func updateNSView(_ view: ObserverView, context: Context) {
        view.onBecomeKey = onBecomeKey
        view.onClose = onClose
    }

    ⟨WindowKeyObserver: the observing NSView 137⟩
}
```

Used in chunk 131.

The view learns its window in `viewDidMoveToWindow`, which also runs if the view ever moves to another window, so old observers are removed first. Notifications are delivered on the main queue. The closures are nonisolated as far as the compiler knows, so `MainActor.assumeIsolated` states what is true at run time. A window that is *already* key when the view arrives won’t send `didBecomeKeyNotification` again, so that case is reported directly, one run-loop turn later, after SwiftUI has finished installing the view.

137 *⟨WindowKeyObserver: the observing NSView 137⟩* ≡

```
final class ObserverView: NSView {
    var onBecomeKey: () -> Void = {}
    var onClose: () -> Void = {}
    private var tokens: [NSObjectProtocol] = []

    override func viewDidMoveToWindow() {
        super.viewDidMoveToWindow()
        tokens.forEach(NotificationCenter.default.removeObserver)
        tokens = []
        guard let window else { return }
        let center = NotificationCenter.default
        tokens.append(center.addObserver(forName: NSWindow.didBecomeKeyNotification, object: window,
        ↪ queue: .main) { [weak self] _ in
            MainActor.assumeIsolated { self?.onBecomeKey() }
        })
        tokens.append(center.addObserver(forName: NSWindow.willCloseNotification, object: window, queue:
        ↪ .main) { [weak self] _ in
            MainActor.assumeIsolated { self?.onClose() }
        })
        if window.isKeyWindow {
            DispatchQueue.main.async { [weak self] in self?.onBecomeKey() }
        }
    }

    deinit {
        tokens.forEach(NotificationCenter.default.removeObserver)
    }
}
```

Used in chunk 136.

9.2 Zones

A zone is a header bar followed by one or more panes side by side. The panes of a zone are listed in `session.panes`, indexed by the zone’s raw value, so splitting a zone is only a matter of appending to that list. `ZoneView` is `@Bindable` because its header controls write straight into the session.

138 *<LineScope/Views/ZoneView.swift 138> ≡*

```
import LineScopeCore
import SwiftUI

/// One horizontal zone: a header with its controls, and one or more side-by-side panes.
struct ZoneView: View {
    let zone: Zone
    @Bindable var session: Session

    <ZoneView: header above side-by-side panes 139>

    <ZoneView: the header row 140>

    <ZoneView: controls specific to each zone 141>

    <ZoneView: split and unsplit buttons 145>

    <ZoneView: describing the line 146>
}

Root chunk, tangled to LineScope/Views/ZoneView.swift.
```

The panes are laid out in an `HSplitView`, so their widths can be adjusted by hand. Each pane gets its index, which is how it finds its own entry in the session’s pane list. The `ForEach` is keyed by the pane’s identity rather than its index, so removing the last pane doesn’t rebuild the others.

139 *<ZoneView: header above side-by-side panes 139> ≡*

```
var body: some View {
    VStack(spacing: 0) {
        header
            .padding(.horizontal, 10)
            .padding(.vertical, 5)
            .background(.bar)
        Divider()
        HSplitView {
            ForEach(Array(session.panes[zone.rawValue].enumerated()), id: \.element.id) { index, _ in
                PaneView(zone: zone, paneIndex: index, session: session)
                    .frame(minWidth: 200)
            }
        }
    }
}

Used in chunk 138.
```

Every header has the same shape: the zone’s title on the left, then its own controls, then the split buttons on the right.

140 *<ZoneView: the header row 140> ≡*

```
private var header: some View {
    HStack(spacing: 12) {
        Text(zone.title).font(.headline)
        Spacer(minLength: 8)
        controls
        Divider().frame(height: 16)
        splitButtons
    }
    .controlSize(.small)
```

```
}
```

Used in chunk 138.

The controls differ by zone.

141 *<ZoneView: controls specific to each zone 141> ≡*

```
@ViewBuilder
private var controls: some View {
    switch zone {
    case .images:
        <ZoneView: image-zone controls 142>
    case .profile:
        <ZoneView: profile-zone controls 143>
    case .spectrum:
        <ZoneView: spectrum-zone controls 144>
    }
}
```

Used in chunk 138.

The image zone shows where the line is and offers to put it back in its initial place: the full width of the image, at mid-height.

142 *<ZoneView: image-zone controls 142> ≡*

```
Text(lineDescription)
    .font(.caption.monospacedDigit())
    .foregroundColor(.secondary)
Button("Reset Line") { session.line = .initial }
```

Used in chunk 141.

The profile zone chooses the color model and the channel. They are settings of the *session*, not of a pane, so every profile and every spectrum in the window shows the same channel. Comparisons between variants stay meaningful. The list of channels depends on the model (RGBA has four, HSL three), so the second picker is rebuilt from `colorModel.channels`.

143 *<ZoneView: profile-zone controls 143> ≡*

```
Picker("Color space", selection: $session.colorModel) {
    ForEach(ColorModel.allCases) { Text($0.displayName).tag($0) }
}
.pickerStyle(.segmented)
.fixedSize()
Picker("Channel", selection: $session.channelIndex) {
    ForEach(Array(session.colorModel.channels.enumerated()), id: \.offset) { i, name in
        Text(name).tag(i)
    }
}
.pickerStyle(.segmented)
.fixedSize()
```

Used in chunk 141.

The spectrum zone controls how the profile is transformed. There is the window function (a Hann window [5] reduces the leakage caused by the finite line), the unit of the frequency axis, whether to subtract the mean first (otherwise the DC term dwarfs everything else), and whether amplitudes are shown in decibels.

```

144 <ZoneView: spectrum-zone controls 144> ≡
  Picker("Window", selection: $session.window) {
    ForEach(WindowFunction.allCases) { Text($0.displayName).tag($0) }
  }
  .fixedSize()
  Picker("Axis", selection: $session.frequencyAxis) {
    ForEach(FrequencyAxis.allCases) { Text($0.displayName).tag($0) }
  }
  .fixedSize()
  Toggle("Remove mean", isOn: $session.removeMean)
  Toggle("dB", isOn: $session.logMagnitude)

```

Used in chunk 141.

Splitting adds a pane and unsplitting removes the last one; a zone always keeps at least one. The same actions are in the View menu.

```

145 <ZoneView: split and unsplit buttons 145> ≡
  private var splitButtons: some View {
    HStack(spacing: 4) {
      Button {
        session.split(zone)
      } label: {
        Image(systemName: "rectangle.split.2x1")
      }
      .help("Split: add a pane to \"(zone.title)\"")
      Button {
        session.unsplit(zone)
      } label: {
        Image(systemName: "rectangle")
      }
      .help("Unsplit: remove the last pane")
      .disabled(session.panes[zone.rawValue].count < 2)
    }
  }
  .labelsHidden()
}

```

Used in chunk 138.

The line is stored in normalized coordinates, with (0,0) at the top-left corner and (1,1) at the bottom-right corner of every image. So the same numbers describe it in all variants, whatever their size. Three decimals are finer than a pixel for any image up to a thousand pixels wide.

```

146 <ZoneView: describing the line 146> ≡
  private var lineDescription: String {
    let l = session.line
    func p(_ v: Double) -> String { v.formatted(.number.precision(.fractionLength(3))) }
    return "(\(p(l.start.x)), \(p(l.start.y))) -> (\(p(l.end.x)), \(p(l.end.y)))"
  }
}

```

Used in chunk 138.

9.3 Panes and tabs

Each pane shows one variant, chosen with a strip of tabs that lists all of them. The tab strips of all panes list the same variants in the same order: the order in which they were made. What a pane shows depends on its zone: the image, its profile, or its spectrum.

147 *<LineScope/Views/PaneView.swift 147>* ≡

```
import SwiftUI

/// A tab strip listing every variant, plus the zone's content for the selected one.
struct PaneView: View {
    let zone: Zone
    let paneIndex: Int
    let session: Session

    <PaneView: which variant this pane shows 148>

    <PaneView: tab strip above the zone's content 149>
}

<PaneView.swift: the tab strip 150>

Root chunk, tangled to LineScope/Views/PaneView.swift.
```

A pane's selection is looked up rather than stored in the view, so it survives redraws and can be changed by the session. When a variant is closed, the session moves every pane that showed it back to the original. The fallbacks cover the brief moments during an update when a pane index or a variant no longer exists: the original is always there.

148 *<PaneView: which variant this pane shows 148>* ≡

```
private var selectedID: UUID {
    let panes = session.panes[zone.rawValue]
    return paneIndex < panes.count ? panes[paneIndex].variantID : session.original.id
}

Used in chunk 147.
```

Selecting a tab also makes that variant the *focused* one, the target of the next menu operation.

149 *<PaneView: tab strip above the zone's content 149>* ≡

```
var body: some View {
    let variant = session.variant(selectedID) ?? session.original
    VStack(spacing: 0) {
        TabStrip(session: session, selectedID: variant.id) { id in
            session.select(id, zone: zone, pane: paneIndex)
        }
        Divider()
        Group {
            switch zone {
            case .images: ImageCanvas(variant: variant, session: session)
            case .profile: ProfileChart(variant: variant, session: session)
            case .spectrum: SpectrumChart(variant: variant, session: session)
            }
        }
        .frame(maxWidth: .infinity, maxHeight: .infinity)
    }
}

Used in chunk 147.
```

The tab strip scrolls horizontally when there are more variants than fit. Each tab is a plain button whose label is drawn by hand. Its context menu offers the per-variant commands.

150 *<PaneView.swift: the tab strip 150>* ≡

```
private struct TabStrip: View {
    let session: Session
    let selectedID: UUID
    let onSelect: (UUID) -> Void

    var body: some View {
        ScrollView(.horizontal, showsIndicators: false) {
```

```

HStack(spacing: 2) {
  ForEach(session.variants) { v in
    let selected = v.id == selectedID
    Button {
      onSelect(v.id)
    } label: {
      <TabStrip: a tab's label 151>
    }
    .buttonStyle(.plain)
    .help(v.name)
    .contextMenu {
      <TabStrip: a tab's context menu 152>
    }
  }
}
.padding(.horizontal, 6)
.padding(.vertical, 3)
}
}
}

```

Used in chunk 147.

A tab shows the variant's name (its chain of operations) and its pixel size, which is the quickest way to tell a resampled copy from a filtered one. Two kinds of state are marked: a tinted background for the variant this pane shows, and a small dot for the window's focused variant. They differ as soon as a zone is split.

151 <TabStrip: a tab's label 151> ≡

```

HStack(spacing: 4) {
  if v.id == session.focusedVariantID {
    Circle().fill(Color.accentColor).frame(width: 5, height: 5)
  }
  Text(v.name).lineLimit(1)
  Text("\${v.buffer.width}x${v.buffer.height}")
    .foregroundStyle(.secondary)
}
.font(.caption)
.padding(.horizontal, 8)
.padding(.vertical, 3)
.background(
  RoundedRectangle(cornerRadius: 5)
    .fill(selected ? Color.accentColor.opacity(0.22) : Color.clear)
)
.contentShape(Rectangle())

```

Used in chunk 150.

Export works on the focused variant, so the tab is selected first. The original can't be closed: every other variant descends from it.

152 *<TabStrip: a tab’s context menu 152> ≡*

```
Button("Export ...Image") {
    onSelect(v.id)
    session.exportFocused()
}
Button("Close Tab") { session.closeVariant(v.id) }
    .disabled(v.isOriginal)
```

Used in chunk 150.

9.4 The image canvas

The image zone must show each variant *at its true scale*: one pixel of the image is one point on the screen. A copy shrunk to half size then really looks half as big, and its blockiness or blur is seen as it is, not hidden by the display’s own interpolation. On a Retina display a point is two device pixels, so by default each image pixel is a 2×2 block. The View menu’s “Actual Device Pixels” option maps one image pixel to one device pixel instead.

On top of the image lies the selection line, which the user can drag.

153 *<LineScope/Views/ImageCanvas.swift 153> ≡*

```
import AppKit
import LineScopeCore
import SwiftUI

/// Shows a variant at its true scale (1 image pixel = 1 point, or 1 device pixel) with scrolling,
/// and lets the user drag the shared selection line.
struct ImageCanvas: View {
    let variant: ImageVariant
    @Bindable var session: Session
    @Environment(\.displayScale) private var displayScale
    @State private var drag: DragState?

    <ImageCanvas: the state of a drag 154>

    <ImageCanvas: drawing the image at true scale 155>

    <ImageCanvas: the line gesture 156>

    <ImageCanvas: hit testing the line 157>

    <ImageCanvas: updating the line during a drag 158>

    <ImageCanvas: Shift snapping 160>

    <ImageCanvas: coordinate conversions 161>
}

<ImageCanvas.swift: the line overlay 162>
```

Root chunk, tangled to LineScope/Views/ImageCanvas.swift.

A drag does one of four things, depending on where it starts. It moves the start point, moves the end point, moves the whole segment, or draws a new segment from scratch. The line as it was when the drag began is kept, because every update is computed from that initial line and the gesture’s total movement, never from the previous update. Small errors therefore cannot pile up over a long drag.

154 *⟨ImageCanvas: the state of a drag 154⟩* ≡

```
private struct DragState {
    enum Mode { case start, end, whole, draw }
    var mode: Mode
    var initial: LineSegment
}
```

Used in chunk 153.

`Image(decorative:scale:)` takes the scale of the bitmap. A scale of 1 draws a pixel per point, and the display's scale factor draws a pixel per device pixel. In both cases `size` is the image's extent in points, and interpolation is turned off so enlarged pixels stay square.

The image and its overlay are given exactly that size, and the gesture is attached to that fixed-size frame, *inside* the scroll view. As a result the gesture's coordinates are coordinates in the image. They don't depend on scrolling or on the empty margin around a small image, and dividing by `size` gives normalized coordinates directly. The outer frame, as large as the pane, only centers a small image in it. A tap focuses the variant without touching the line.

155 *⟨ImageCanvas: drawing the image at true scale 155⟩* ≡

```
var body: some View {
    let scale = session.useDevicePixels ? displayScale : 1
    let size = CGSize(width: CGFloat(variant.buffer.width) / scale, height:
        ↪ CGFloat(variant.buffer.height) / scale)
    GeometryReader { geo in
        ScrollView([.horizontal, .vertical]) {
            ZStack {
                Image(decorative: variant.cgImage, scale: scale)
                    .interpolation(.none)
                LineOverlay(line: session.line)
                    .allowsHitTesting(false)
            }
            .frame(width: size.width, height: size.height)
            .contentShape(Rectangle())
            .gesture(lineGesture(size: size))
            .simultaneousGesture(TapGesture().onEnded { session.focusedVariantID = variant.id })
            .frame(minWidth: geo.size.width, minHeight: geo.size.height)
        }
    }
    .background(Color(nsColor: .underPageBackgroundColor))
}
```

Used in chunk 153.

The drag decides its mode on its first change and remembers it until it ends. A minimum distance of two points keeps a plain click from being taken as a zero-length new line. Writing `session.line` is all it takes to move the line in every pane of every zone.

156 *⟨ImageCanvas: the line gesture 156⟩* ≡

```
private func lineGesture(size: CGSize) -> some Gesture {
    DragGesture(minimumDistance: 2)
        .onChanged { g in
            if drag == nil {
                drag = DragState(mode: hitTest(g.startLocation, size: size), initial: session.line)
                session.focusedVariantID = variant.id
            }
            guard let drag else { return }
            session.line = updatedLine(drag, gesture: g, size: size)
        }
        .onEnded { _ in drag = nil }
}
```

Used in chunk 153.

Hit testing works in points, so the targets are the same size whatever the image's scale. The endpoints come first and are a little more generous (9 points) than the segment (6 points), so an

endpoint can always be grabbed even where it touches the segment. A drag that starts anywhere else draws a new line.

157 *⟨ImageCanvas: hit testing the line 157⟩* ≡

```
private func hitTest(_ p: CGPoint, size: CGSize) -> DragState.Mode {
    let a = point(session.line.start, size)
    let b = point(session.line.end, size)
    if distance(p, a) <= 9 { return .start }
    if distance(p, b) <= 9 { return .end }
    if distanceToSegment(p, a, b) <= 6 { return .whole }
    return .draw
}
```

Used in chunk 153.

Moving an endpoint or drawing a new line places a point where the pointer is. That point has been clamped to the image, so a line can reach the image's edge but never pass it.

158 *⟨ImageCanvas: updating the line during a drag 158⟩* ≡

```
private func updatedLine(_ drag: DragState, gesture g: DragGesture.Value, size: CGSize) -> LineSegment {
    let shift = NSEvent.modifierFlags.contains(.shift)
    let current = normalized(g.location, size)
    var line = drag.initial
    switch drag.mode {
    case .start:
        line.start = snapped(current, anchor: line.end, size: size, enabled: shift)
    case .end:
        line.end = snapped(current, anchor: line.start, size: size, enabled: shift)
    case .draw:
        line.start = normalized(g.startLocation, size)
        line.end = snapped(current, anchor: line.start, size: size, enabled: shift)
    case .whole:
        ⟨ImageCanvas: moving the whole segment without leaving the image 159⟩
    }
    return line
}
```

Used in chunk 153.

Moving the whole segment is different. Clamping each endpoint separately would squash the segment against the edge and change its length and angle. Instead the *offset* is clamped, so that both endpoints stay inside. If the initial endpoints have x coordinates x_s and x_e , the horizontal offset Δx must satisfy

$$-\min(x_s, x_e) \leq \Delta x \leq 1 - \max(x_s, x_e),$$

and the same holds vertically. Within that range the segment keeps its shape exactly; at the limit it slides along the edge.

159 *⟨ImageCanvas: moving the whole segment without leaving the image 159⟩* ≡

```
// Move both endpoints, limiting the offset so neither leaves the image.
var dx = g.translation.width / size.width
var dy = g.translation.height / size.height
let i = drag.initial
dx = min(max(dx, -min(i.start.x, i.end.x)), 1 - max(i.start.x, i.end.x))
dy = min(max(dy, -min(i.start.y, i.end.y)), 1 - max(i.start.y, i.end.y))
line.start = NormalizedPoint(x: i.start.x + dx, y: i.start.y + dy)
line.end = NormalizedPoint(x: i.end.x + dx, y: i.end.y + dy)
```

Used in chunk 158.

Holding Shift makes the segment exactly horizontal or vertical, which is what a profile along a row or a column needs. Which of the two is decided by the larger displacement *in pixels*, not in normalized units. In a wide image a normalized step of 0.1 across is much longer than 0.1 down, and comparing normalized values would snap the wrong way.

160 *<ImageCanvas: Shift snapping 160> ≡*

```

/// With Shift held, constrains the segment to horizontal or vertical (in pixel space).
private func snapped(_ p: NormalizedPoint, anchor: NormalizedPoint, size: CGSize, enabled: Bool) ->
    NormalizedPoint {
    guard enabled else { return p }
    let dx = abs(p.x - anchor.x) * size.width
    let dy = abs(p.y - anchor.y) * size.height
    return dx >= dy ? NormalizedPoint(x: p.x, y: anchor.y) : NormalizedPoint(x: anchor.x, y: p.y)
}

```

Used in chunk 153.

The remaining helpers convert between points and normalized coordinates, and measure distances. The distance to a segment projects the point onto the segment's line, with the parameter t clamped to $[0, 1]$ so the nearest point stays between the endpoints. A degenerate segment is treated as a point.

161 *<ImageCanvas: coordinate conversions 161> ≡*

```

private func normalized(_ p: CGPoint, _ size: CGSize) -> NormalizedPoint {
    NormalizedPoint(x: p.x / size.width, y: p.y / size.height).clamped()
}

private func point(_ n: NormalizedPoint, _ size: CGSize) -> CGPoint {
    CGPoint(x: n.x * size.width, y: n.y * size.height)
}

private func distance(_ a: CGPoint, _ b: CGPoint) -> CGFloat {
    hypot(a.x - b.x, a.y - b.y)
}

private func distanceToSegment(_ p: CGPoint, _ a: CGPoint, _ b: CGPoint) -> CGFloat {
    let ab = CGPoint(x: b.x - a.x, y: b.y - a.y)
    let len2 = ab.x * ab.x + ab.y * ab.y
    guard len2 > 0 else { return distance(p, a) }
    let t = min(max(((p.x - a.x) * ab.x + (p.y - a.y) * ab.y) / len2, 0), 1)
    return distance(p, CGPoint(x: a.x + ab.x * t, y: a.y + ab.y * t))
}

```

Used in chunk 153.

The overlay is a Canvas the size of the image, so normalized coordinates become points by a single multiplication. The line has to be visible on any image, light or dark. So it is drawn twice: a wide dark stroke with a thin yellow stroke on top. The two handles differ on purpose. The filled one is the start, position 0 on the profile's horizontal axis, and the hollow one is the end. Without that, a profile read from right to left would be easy to misread.

162 *<ImageCanvas.swift: the line overlay 162> ≡*

```

/// Draws the selection line: a filled handle at the start (position 0) and a hollow one at the end.
private struct LineOverlay: View {
    let line: LineSegment

    var body: some View {
        Canvas { ctx, size in
            let a = CGPoint(x: line.start.x * size.width, y: line.start.y * size.height)
            let b = CGPoint(x: line.end.x * size.width, y: line.end.y * size.height)
            var path = Path()
            path.move(to: a)
            path.addLine(to: b)
            ctx.stroke(path, with: .color(.black.opacity(0.7)), lineWidth: 3)
            ctx.stroke(path, with: .color(.yellow), lineWidth: 1.25)

            let r: CGFloat = 5
            let startDot = Path(ellipseIn: CGRect(x: a.x - r, y: a.y - r, width: 2 * r, height: 2 * r))
            ctx.fill(startDot, with: .color(.yellow))
        }
    }
}

```

```

        ctx.stroke(startDot, with: .color(.black.opacity(0.7)), lineWidth: 1)
        let endDot = Path(ellipseIn: CGRect(x: b.x - r, y: b.y - r, width: 2 * r, height: 2 * r))
        ctx.fill(endDot, with: .color(.black.opacity(0.35)))
        ctx.stroke(endDot, with: .color(.yellow), lineWidth: 1.5)
    }
}
}

```

Used in chunk 153.

9.5 Charts

Both charts use Swift Charts' vectorized `LinePlot`, which draws thousands of points quickly enough to follow a drag. Each chart computes its data in `body`, from the session. Because the session is observable, any change to the line, the channel or the spectrum settings redraws the chart.

163 *<LineScope/Views/Charts.swift 163> ≡*

```

import Charts
import LineScopeCore
import SwiftUI

```

<Charts.swift: the profile chart 164>

<Charts.swift: the spectrum chart 166>

Root chunk, tangled to LineScope/Views/Charts.swift.

The profile's horizontal axis is the position along the line, from 0 at the start handle to 1 at the end. The number of samples depends on the variant: an image at half size gives half as many samples along the same line. So the axis label reports the count. Using normalized position lets profiles of differently sized copies line up side by side.

The vertical axis is *fixed* to the channel's natural range: `[0, 1]` for most channels, and `[0, 360]` degrees for hue. If each chart chose its own range, a faint ripple and a full-contrast square wave would look alike. With a shared, fixed domain, the charts of different variants can be compared by eye.

164 *<Charts.swift: the profile chart 164> ≡*

/// Zone 2: the selected channel sampled along the line in this variant's own pixel grid.

```

struct ProfileChart: View {
    let variant: ImageVariant
    let session: Session

    var body: some View {
        let profile = session.profile(for: variant)
        let channel = session.channelName
        Chart {
            LinePlot(profile.points, x: .value("Position", \.x), y: .value(channel, \.y))
                .foregroundColor(channelColor)
                .lineStyle(StrokeStyle(lineWidth: 1.2))
        }
        .chartXScale(domain: 0...1)
        .chartYScale(domain: session.colorModel.range(ofChannel: session.channelIndex))
        .chartXAxisLabel("Position along line (\(profile.samples.colors.count) samples)")
        .chartYAxisLabel(channel)
        .padding(10)
    }
}

```

<ProfileChart: a color for each channel 165>

```

}

```

Used in chunk 163.

The curve takes the color of its channel, so a glance tells which channel is plotted. Channels without a natural hue (alpha, lightness, black) use the primary text color.

165 *<ProfileChart: a color for each channel 165> ≡*

```
private var channelColor: Color {
    switch (session.colorModel, session.channelIndex) {
    case (.rgba, 0): .red
    case (.rgba, 1): .green
    case (.rgba, 2): .blue
    case (.cmyk, 0): .cyan
    case (.cmyk, 1): .pink
    case (.cmyk, 2): .yellow
    case (.hsl, 0): .purple
    case (.hsl, 1): .orange
    default: .primary
    }
}
```

Used in chunk 164.

The spectrum chart transforms the same profile. Its horizontal axis ends at the Nyquist frequency, the highest frequency the samples can represent, and a dashed rule marks it. In cycles per pixel, Nyquist is about $\frac{1}{2}$ for every variant, since the samples are about one pixel apart. In cycles per line it is half the number of samples, so it moves: a copy at half size has its Nyquist limit at half the frequency. Energy that the original has above that limit either disappears (a good low-pass filter) or folds back below it (aliasing). The rule shows at once which side of the limit a feature is on.

166 *<Charts.swift: the spectrum chart 166> ≡*

/// Zone 3: amplitude spectrum of the zone-2 profile.

```
struct SpectrumChart: View {
    let variant: ImageVariant
    let session: Session

    var body: some View {
        let profile = session.profile(for: variant)
        let spectrum = session.spectrum(for: profile)
        let points = session.spectrumPoints(spectrum)
        let nyquist = nyquist(spectrum)
        Chart {
            LinePlot(points, x: .value("Frequency", \.frequency), y: .value("Amplitude", \.magnitude))
                .foregroundColor(Color.accentColor)
                .lineStyle(StrokeStyle(lineWidth: 1.2))
            if let nyquist {
                RuleMark(x: .value("Nyquist", nyquist))
                    .foregroundColor(.secondary)
                    .lineStyle(StrokeStyle(lineWidth: 1, dash: [4, 3]))
                    .annotation(position: .leading, alignment: .top) {
                        Text("Nyquist").font(.caption2).foregroundColor(.secondary)
                    }
            }
        }
        .chartXScale(domain: 0...max(nyquist ?? 0.5, 1e-9))
        .chartYScale(domain: yDomain(points))
        .chartXAxisLabel("Frequency (\(session.frequencyAxis.displayName)), FFT size
↪ \((spectrum.fftSize))")
        .chartYAxisLabel(session.logMagnitude ? "Amplitude (dB)" : "Amplitude")
        .padding(10)
    }
}
```

<SpectrumChart: the Nyquist frequency on the current axis 167>

<SpectrumChart: the amplitude range 168>

```
}
```

Used in chunk 163.

The last bin of the one-sided spectrum is the Nyquist bin, so its frequency, in whichever unit the axis uses, is the limit. An empty spectrum (a line too short to sample) has no limit.

167 *<SpectrumChart: the Nyquist frequency on the current axis 167> ≡*

```
private func nyquist(_ s: SpectrumResult) -> Double? {
    guard s.fftSize > 0 else { return nil }
    switch session.frequencyAxis {
    case .cyclesPerPixel: return s.cyclesPerPixel.last
    case .cyclesPerLine:  return s.cyclesPerLine.last
    }
}
```

Used in chunk 166.

In decibels, the vertical range is fixed at the bottom at -100 dB, the floor to which the session clips tiny amplitudes. The top is the next multiple of 10 dB above the peak (at least 0), so charts of different variants mostly share one scale. On the linear scale the peak sets the range, with 5% of headroom.

168 *<SpectrumChart: the amplitude range 168> ≡*

```
private func yDomain(_ points: [SpectrumPoint]) -> ClosedRange<Double> {
    let top = points.map(\.magnitude).max() ?? 1
    if session.logMagnitude {
        return -100...max(0, (top / 10).rounded(.up) * 10)
    }
    return 0...max(top * 1.05, 1e-6)
}
```

Used in chunk 166.

9.6 The parameter sheet

Operations that take parameters (the blurs, sharpening, noise reduction and every resampling method) ask for them in a sheet before running. The menu only sets the session's `pendingRequest`; the window presents this sheet for it; and Apply hands a finished `ImageOperation` to the session, which runs it on the focused variant in the background.

169 *<LineScope/Views/ParameterSheet.swift 169> ≡*

```
import LineScopeCore
import SwiftUI

/// Collects parameters for a filter or resample, then runs it on the focused variant.
struct ParameterSheet: View {
    let request: OperationRequest
    let session: Session
    @Environment(\.dismiss) private var dismiss

    <ParameterSheet: parameters and their defaults 170>

    <ParameterSheet: layout 171>

    <ParameterSheet: fields for each request 172>

    <ParameterSheet: a numeric field 173>

    <ParameterSheet: validating a resample 174>

    <ParameterSheet: building the operation 175>
}
```

Root chunk, tangled to LineScope/Views/ParameterSheet.swift.

One sheet serves every request, so it simply holds all the parameters any of them might need, with defaults that give a visible but moderate effect. A scale of 0.5 is the most instructive default for resampling: halving an image is where aliasing first shows.

170 *⟨ParameterSheet: parameters and their defaults 170⟩* ≡

```
@State private var radius = 2.0
@State private var intensity = 0.5
@State private var noiseLevel = 0.02
@State private var sharpness = 0.4
@State private var scale = 0.5

private static let maxOutputSide = 16_384
```

Used in chunk 169.

The sheet names the variant it will act on. Operations chain, so the source is often not the original, and the user should be able to see which variant it is. Return applies and Escape cancels.

171 *⟨ParameterSheet: layout 171⟩* ≡

```
var body: some View {
    let source = session.focusedVariant
    VStack(alignment: .leading, spacing: 14) {
        Text(request.title).font(.headline)
        Text("Applies to: \(source.name) \(source.buffer.width)×\(source.buffer.height)")
            .font(.caption)
            .foregroundColor(.secondary)

        Form { fields(source: source) }

        HStack {
            Spacer()
            Button("Cancel", role: .cancel) { dismiss() }
                .keyboardShortcut(.cancelAction)
            Button("Apply") {
                session.perform(operation)
                dismiss()
            }
                .keyboardShortcut(.defaultAction)
                .disabled(!isValid(source: source))
        }
    }
    .padding(20)
    .frame(width: 380)
}
```

Used in chunk 169.

Each request shows only its own fields. For resampling, the sheet shows the resulting size as the scale is typed, computed with the same rounding the resampler will use (`Resampler.outputSize`). What the user sees is exactly what the operation will produce.

172 *⟨ParameterSheet: fields for each request 172⟩* ≡

```
@ViewBuilder
private func fields(source: ImageVariant) -> some View {
    switch request {
    case .gaussianBlur, .boxBlur:
        number("Radius (px)", $radius, range: 0...100)
    case .unsharpMask:
        number("Radius (px)", $radius, range: 0...100)
        number("Intensity", $intensity, range: 0...10)
    case .noiseReduction:
        number("Noise level", $noiseLevel, range: 0...0.1)
        number("Sharpness", $sharpness, range: 0...2)
    }
```

```

case .resample:
    LabeledContent("Scale factor") {
        HStack {
            TextField("Scale", value: $scale, format: .number.precision(.fractionLength(0...4)))
                .frame(width: 80)
            Menu("Presets") {
                ForEach([0.25, 0.5, 0.75, 1.5, 2, 4], id: \.self) { s in
                    Button("×\($s.formatted())") { scale = s }
                }
            }
            .fixedSize()
        }
    }
    LabeledContent("Result") {
        Text("\((Resampler.outputSize(for: source.buffer.width, scale: scale)) ×
↪ \((Resampler.outputSize(for: source.buffer.height, scale: scale)) px")
            .monospacedDigit()
    }
}
}

```

Used in chunk 169.

A numeric parameter is a slider for exploring together with a text field for exact values. Both are bound to the same state.

173 *⟨ParameterSheet: a numeric field 173⟩* ≡

```

private func number(_ label: String, _ value: Binding<Double>, range: ClosedRange<Double>) -> some View {
    LabeledContent(label) {
        HStack {
            Slider(value: value, in: range)
            TextField(label, value: value, format: .number.precision(.fractionLength(0...3)))
                .frame(width: 60)
                .labelsHidden()
        }
    }
}

```

Used in chunk 169.

Only resampling can be asked for something unreasonable: a zero or negative scale, or an output so large it would exhaust memory. With four `Float` components per pixel, a 16384×16384 image already takes 4 GiB, so that is the limit.

174 *⟨ParameterSheet: validating a resample 174⟩* ≡

```

private func isValid(source: ImageVariant) -> Bool {
    guard case .resample = request else { return true }
    let w = Resampler.outputSize(for: source.buffer.width, scale: scale)
    let h = Resampler.outputSize(for: source.buffer.height, scale: scale)
    return scale > 0 && max(w, h) <= Self.maxOutputSide
}

```

Used in chunk 169.

Finally the request and the collected values become an operation, a plain value that can run anywhere, record itself in a variant's lineage, and name the new tab.

175 *⟨ParameterSheet: building the operation 175⟩* ≡

```

private var operation: ImageOperation {
    switch request {
    case .gaussianBlur: .filter(.gaussianBlur(radius: radius))
    case .boxBlur: .filter(.boxBlur(radius: radius))
    case .unsharpMask: .filter(.unsharpMask(radius: radius, intensity: intensity))
    case .noiseReduction: .filter(.noiseReduction(level: noiseLevel, sharpness: sharpness))
    case .resample(let method): .resample(method, scale: scale)
    }
}

```

}

Used in chunk 169.

9.7 The inspector

The inspector is a separate window that describes the focused variant of the most recently active document window. As explained with `MainView`, it can't find that window through focused values. The inspector *is* the key window while it is being read or scrolled, and then the focused value would be empty. It observes `ActiveSession.shared` instead, which document windows update when they become key.

176 *<LineScope/Views/InspectorView.swift 176> ≡*

```
import LineScopeCore
import SwiftUI

/// Separate window describing the focused variant of the most recently active document window.
struct InspectorView: View {
    static let windowID = "inspector"

    private var active = ActiveSession.shared

    var body: some View {
        Group {
            if let session = active.session {
                InspectorContent(session: session)
            } else {
                ContentUnavailableView("No Image", systemImage: "photo", description: Text("Open an
↔ image to inspect it."))
            }
        }
        .frame(minWidth: 300, idealWidth: 340, minHeight: 320, idealHeight: 760)
    }
}
```

<InspectorView.swift: the inspector's content 177>

Root chunk, tangled to LineScope/Views/InspectorView.swift.

The content is a grouped form of read-only rows. It computes the same profile and spectrum as the charts, so its numbers always agree with what the zones show.

177 *<InspectorView.swift: the inspector's content 177> ≡*

```
private struct InspectorContent: View {
    let session: Session

    var body: some View {
        let variant = session.focusedVariant
        let profile = session.profile(for: variant)
        let spectrum = session.spectrum(for: profile)
        let info = session.sourceInfo
        let original = session.original.buffer

        Form {
            <InspectorContent: the file section 178>

            <InspectorContent: the focused-image section 179>

            <InspectorContent: the operation-chain section 180>

            <InspectorContent: the line section 181>
```

```

    }
    .formStyle(.grouped)
}

<InspectorContent: row and number helpers 183>
}

```

Used in chunk 176.

The file section describes the image as it was stored, before it was converted to the float buffer all processing uses. A 16-bit or CMYK file shows that here, even though every variant is processed in the same format.

178 *<InspectorContent: the file section 178> ≡*

```

Section("File") {
    row("Name", session.fileName)
    row("Type", info.fileType)
    row("Stored size", "\((info.pixelWidth) × \((info.pixelHeight) px)")
    row("Bit depth", "\((info.bitsPerComponent) bpc, \((info.bitsPerPixel) bpp)")
    row("Color space", info.colorSpaceName)
    row("Alpha", info.hasAlpha ? "Yes" : "No")
    if let dpi = info.dpi { row("Resolution", "\((num(dpi, 0)) dpi)") }
}

```

Used in chunk 177.

For the focused variant, the scale is given separately for each axis. Rounding to whole pixels makes a nominal $\times 0.5$ of an odd-sized image slightly different horizontally and vertically.

179 *<InspectorContent: the focused-image section 179> ≡*

```

Section("Focused Image") {
    row("Tab", variant.name)
    row("Size", "\((variant.buffer.width) × \((variant.buffer.height) px)")
    row(
        "Scale vs. original",
        "×\((num(Double(variant.buffer.width) / Double(original.width), 4)) ·"
        ↪ "×\((num(Double(variant.buffer.height) / Double(original.height), 4))"
    )
    row("Working format", "RGBA Float32, sRGB-encoded")
}

```

Used in chunk 177.

The operation chain lists, in order, every step from the original to this variant. This is the full history, even though each step was applied to whatever variant was focused at the time.

180 *<InspectorContent: the operation-chain section 180> ≡*

```

Section("Operation Chain") {
    Text("Original").foregroundColor(.secondary)
    ForEach(Array(variant.lineage.enumerated()), id: \.offset) { i, op in
        Text("\((i + 1). \((op.displayName)")
    }
}

```

Used in chunk 177.

The line is shared in normalized coordinates, but here it is shown in *this* variant's pixels. Its length, sample count and spacing follow, so you can see directly that a resampled copy is sampled more coarsely along the same line.

181 *<InspectorContent: the line section 181> ≡*

```

Section("Line") {
    let w = Double(variant.buffer.width), h = Double(variant.buffer.height)
    let l = session.line
    row("Start", "\(\num(l.start.x * w, 1)), \(\num(l.start.y * h, 1)) px")
    row("End", "\(\num(l.end.x * w, 1)), \(\num(l.end.y * h, 1)) px")
    row("Length", "\(\num(profile.samples.pixelLength, 1)) px")
    row("Samples", "\(\num(profile.values.count), spacing \(\num(profile.samples.spacing, 3)) px")
}

```

Used in chunk 177.

Last come summary statistics of the profile and the strongest frequency in its spectrum, given in both units of the frequency axis. With the mean removed, that peak is the dominant periodicity along the line: for a grating, its frequency.

182 *<InspectorContent: the channel-statistics section 182> ≡*

```

Section("\(\session.colorModel.displayName) · \(\session.channelName)") {
    let v = profile.values
    row("Min", num(Double(v.min() ?? 0), 4))
    row("Max", num(Double(v.max() ?? 0), 4))
    row("Mean", num(Double(v.reduce(0, +)) / Double(max(v.count, 1)), 4))
    if let k = spectrum.peakIndex {
        row("Peak frequency", "\(\num(spectrum.cyclesPerPixel[k], 4)) c/px ·
        ↪ \(\num(spectrum.cyclesPerLine[k], 2)) c/line")
        row("Peak amplitude", num(spectrum.amplitudes[k], 4))
    }
}

```

Used in chunk 177.

Values are selectable so they can be copied, and numbers are formatted with a fixed number of decimals in the user's locale.

183 *<InspectorContent: row and number helpers 183> ≡*

```

private func row(_ label: String, _ value: String) -> some View {
    LabeledContent(label) {
        Text(value).textSelection(.enabled).multilineTextAlignment(.trailing)
    }
}

private func num(_ v: Double, _ digits: Int) -> String {
    v.formatted(.number.precision(.fractionLength(digits)))
}

```

Used in chunk 177.

9.8 Test-pattern windows

A generated test pattern (Chapter 6) is shown in exactly the same analysis window as an image file. It is only a different way of producing the `ImageDocument`. Two views are needed: the window for a pattern, and a generator window where a pattern is designed before it is created.

184 *<LineScope/Views/TestPatternViews.swift 184> ≡*

```

import LineScopeCore
import SwiftUI

```

<TestPatternViews.swift: the pattern analysis window 185>

<TestPatternViews.swift: the generator window 186>

Root chunk, tangled to LineScope/Views/TestPatternViews.swift.

A pattern window receives only the pattern's `TestPatternSpec`, a small value that SwiftUI can store and restore with the window. The image must be generated from it, and that must happen *once*. Doing it in `init` or `body` would regenerate a possibly large image on every redraw, and a fresh document would also mean a fresh session, losing all derived variants. So the document lives in `@State` and is filled by a task keyed on the spec, which runs once when the window appears. The generation itself runs detached, off the main thread, while a progress indicator holds the window's place.

185 *⟨TestPatternViews.swift: the pattern analysis window 185⟩* ≡

/// Analysis window for a generated pattern. Generation runs once per window, not on every view update.

```
struct PatternWindow: View {
    let spec: TestPatternSpec
    @State private var document: ImageDocument?

    var body: some View {
        Group {
            if let document {
                MainView(document: document, fileName: spec.title)
            } else {
                ProgressView("Generating \(spec.kind.displayName)...")
                    .frame(minWidth: 720, minHeight: 560)
            }
        }
        .navigationTitle(spec.title)
        .task(id: spec) {
            let spec = spec
            document = await Task.detached(priority: .userInitiated) { ImageDocument(pattern: spec)
            }
        }
    }
}
```

Used in chunk 184.

The generator window edits a spec: the kind of pattern, its size, and its one parameter. It shows a live preview, and Create opens a pattern window for it.

186 *⟨TestPatternViews.swift: the generator window 186⟩* ≡

/// Window for choosing a pattern, its size and parameter before creating it.

```
struct TestPatternGeneratorView: View {
    static let windowID = "pattern-generator"

    @Environment(\.openWindow) private var openWindow
    @State private var spec = TestPatternSpec(kind: .zonePlate)
    @State private var preview: CGImage?

    private static let sizes = [128, 256, 512, 1024, 2048]
    private static let maxSide = 8192
    private static let maxPreviewPixels = 2048 * 2048

    var body: some View {
        HStack(alignment: .top, spacing: 20) {
            Form {
                ⟨Generator window: pattern picker and description 187⟩

                ⟨Generator window: size fields 188⟩

                ⟨Generator window: parameter field 189⟩

                ⟨Generator window: the create button 190⟩
            }
            .formStyle(.grouped)
            .frame(width: 380)
        }
    }
}
```

```

        previewView
    }
    .padding(16)
    .task(id: spec) {
        <Generator window: rendering the preview 191>
    }
}

<Generator window: the preview box 192>

<Generator window: changing the pattern kind 193>

<Generator window: validation and clamping 194>
}

```

Used in chunk 184.

Each kind of pattern carries a sentence on what it is good for. Showing that under the picker turns the list into a small catalogue of test targets.

187 <Generator window: pattern picker and description 187> ≡

```

Picker("Pattern", selection: kindBinding) {
    ForEach(TestPatternKind.allCases) { Text($0.displayName).tag($0) }
}
Text(spec.kind.purpose)
    .font(.caption)
    .foregroundStyle(.secondary)
    .fixedSize(horizontal: false, vertical: true)

```

Used in chunk 186.

The size is typed or picked from square presets. Grouping separators are turned off, so a width reads 2048 and not 2,048.

188 <Generator window: size fields 188> ≡

```

LabeledContent("Size") {
    HStack {
        TextField("Width", value: $spec.width, format: .number.grouping(.never))
            .frame(width: 64)
        Text("x")
        TextField("Height", value: $spec.height, format: .number.grouping(.never))
            .frame(width: 64)
        Menu("Presets") {
            ForEach(Self.sizes, id: \.self) { s in
                Button("\($s) × \($s)") { spec.width = s; spec.height = s }
            }
        }
        .fixedSize()
    }
    .labelsHidden()
}

```

Used in chunk 186.

The parameter's name and range come from the pattern kind itself. Kinds without a parameter show nothing. The noise seed gets no slider: neighboring seeds are unrelated, so dragging through them means nothing.

189 *⟨Generator window: parameter field 189⟩* ≡

```

if let p = spec.kind.parameter {
    LabeledContent(p.name) {
        HStack {
            if spec.kind != .whiteNoise {
                Slider(value: $spec.parameter, in: p.range)
            }
            TextField(p.name, value: $spec.parameter, format: .number.precision(.fractionLength(0...4)))
                .frame(width: 70)
                .labelsHidden()
        }
    }
}

```

Used in chunk 186.

Create opens a new pattern window *by value*. SwiftUI finds the window group for `TestPatternSpec`, and creating the same spec twice brings the existing window forward instead of opening a duplicate. A typed parameter can lie outside the slider’s range, so it is clamped first.

190 *⟨Generator window: the create button 190⟩* ≡

```

HStack {
    Spacer()
    Button("Create") { openWindow(value: clamped(spec)) }
        .keyboardShortcut(.defaultAction)
        .disabled(!isValid)
}

```

Used in chunk 186.

The preview must show what the pattern will really look like. Generating a small version of the pattern would be wrong: a zone plate made at 256×256 is a different image from the center of one made at 1024×1024 , with different frequencies and different aliasing. So the preview generates the full pattern and crops its central 256×256 pixels, which are shown at 1:1. Huge patterns are not previewed at all.

The task is keyed on the spec, so SwiftUI cancels it whenever the spec changes. The short sleep debounces typing and slider drags: only a spec that stays unchanged for 80 ms is rendered.

191 *⟨Generator window: rendering the preview 191⟩* ≡

```

guard isValid, spec.width * spec.height <= Self.maxPreviewPixels else { preview = nil; return }
// Generate the real pattern and show its central pixels at 1:1, so aliasing looks as it will
// in the analysis window.
let s = clamped(spec)
try? await Task.sleep(for: .milliseconds(80))
guard !Task.isCancelled else { return }
preview = await Task.detached {
    guard let full = TestPatternGenerator.make(s).makeCGImage() else { return nil }
    let w = min(s.width, 256), h = min(s.height, 256)
    let rect = CGRect(x: (s.width - w) / 2, y: (s.height - h) / 2, width: w, height: h)
    return full.cropping(to: rect)
}.value

```

Used in chunk 186.

The preview box always has the same size, so the window doesn’t jump as specs change. As in the image canvas, interpolation is off: a preview pixel is an image pixel.

192 *⟨Generator window: the preview box 192⟩* ≡

```

private var previewView: some View {
    VStack(spacing: 6) {
        Group {
            if let preview {
                Image(decorative: preview, scale: 1).interpolation(.none)
            } else {

```

```

        Color.clear
    }
}
.frame(width: 256, height: 256)
.background(Color(nsColor: .underPageBackgroundColor))
.clipShape(RoundedRectangle(cornerRadius: 4))
Text(spec.width * spec.height <= Self.maxPreviewPixels
    ? "Preview: center 256 × 256 px at 1:1" : "Too large to preview")
    .font(.caption)
    .foregroundStyle(.secondary)
}
}

```

Used in chunk 186.

Parameters mean different things for different patterns: a frequency in cycles per pixel for a chirp, a period in pixels for bars, a number of spokes for the star. Carrying the value over when the kind changes would produce nonsense, such as a checkerboard with cells 0.5 pixels wide. So the custom binding rebuilds the spec, keeping the size and taking the new kind's default parameter.

193 *⟨Generator window: changing the pattern kind 193⟩* ≡

/// Resets the parameter to the new pattern's default when the kind changes.

```

private var kindBinding: Binding<TestPatternKind> {
    Binding(
        get: { spec.kind },
        set: { spec = TestPatternSpec(kind: $0, width: spec.width, height: spec.height) }
    )
}

```

Used in chunk 186.

Sizes are limited to 8192 pixels per side, and the parameter is limited to its kind's range.

194 *⟨Generator window: validation and clamping 194⟩* ≡

```

private var isValid: Bool {
    (1...Self.maxSide).contains(spec.width) && (1...Self.maxSide).contains(spec.height)
}

private func clamped(_ s: TestPatternSpec) -> TestPatternSpec {
    var s = s
    if let p = s.kind.parameter {
        s.parameter = min(max(s.parameter, p.range.lowerBound), p.range.upperBound)
    }
    return s
}

```

Used in chunk 186.

Chapter 10

Menus

Everything the user can do to an image goes through the menu bar: derive a copy with a filter or a resampler, create a test pattern, split a zone, export, and open the inspector. SwiftUI describes menus with a `Commands` value, attached to the app’s scenes in `LineScopeApp` (Chapter 8).

195 *<LineScope/Commands/LineScopeCommands.swift 195> ≡*

```
import LineScopeCore
import SwiftUI

struct LineScopeCommands: Commands {
    <Commands: finding the target session 196>

    var body: some Commands {
        <Commands: File menu, test patterns 197>

        <Commands: File menu, export and close 198>

        <Commands: Filter menu 199>

        <Commands: Resample menu 200>

        <Commands: View menu 201>
    }
}
```

Root chunk, tangled to LineScope/Commands/LineScopeCommands.swift.

10.1 Which document does a menu act on?

Menus are global, but their actions are for one document: the one in the frontmost window. SwiftUI’s way to connect the two is a *focused value*. Each document window publishes its `Session` with `focusedSceneValue`, and `@FocusedValue` here reads the value of whichever window is key.

That alone has one bad consequence. The inspector is a separate window, and while the user works in it, *it* is the key window. It publishes no session, so the focused value is `nil`, and every menu item would turn gray at the moment the inspector is showing the image one wants to filter. So the menus fall back to `ActiveSession.shared`, the session of the most recently key document window, which the document windows keep up to date (Chapter 9). When a document window is key, the two agree. When the inspector is key, the fallback names the document it describes.

196 *⟨Commands: finding the target session 196⟩* ≡

```
@FocusedValue(\.session) private var focusedSession
@Environment(\.openWindow) private var openWindow

/// Falls back to the last active document so the menus keep working while the inspector is key.
private var session: Session? { focusedSession ?? ActiveSession.shared.session }

Used in chunk 195.
```

Every action below starts with `session?..` With no document open, the optional chain makes an action do nothing, and a `.disabled(session == nil)` makes that visible.

10.2 The File menu

Test patterns don't need a document: choosing one opens a new window for it. The value passed to `openWindow(value:)` is the pattern's specification with default size and parameter. SwiftUI finds the `WindowGroup` that presents that type (Chapter 9). The `Custom...` item opens the generator window, where size and parameter can be set first.

197 *⟨Commands: File menu, test patterns 197⟩* ≡

```
CommandGroup(after: .newItem) {
    Menu("New Test Pattern") {
        ForEach(TestPatternKind.allCases) { kind in
            Button(kind.displayName) { openWindow(value: TestPatternSpec(kind: kind)) }
        }
        Divider()
        Button("...Custom") { openWindow(id: TestPatternGeneratorView.windowID) }
            .keyboardShortcut("n", modifiers: [.command, .option])
    }
}
```

Used in chunk 195.

Export and Close Tab both act on the *focused variant*: the tab last selected or clicked in any zone. The original can't be closed, because every other variant is derived from it and every pane falls back to it.

198 *⟨Commands: File menu, export and close 198⟩* ≡

```
CommandGroup(after: .saveItem) {
    Button("Export ...Image") { session?.exportFocused() }
        .keyboardShortcut("e", modifiers: [.command, .shift])
        .disabled(session == nil)
    Button("Close Tab") {
        if let session { session.closeVariant(session.focusedVariantID) }
    }
    .keyboardShortcut(.delete, modifiers: [.command])
    .disabled(session?.focusedVariant.isOriginal ?? true)
}
```

Used in chunk 195.

10.3 Filters and resampling

A menu item either runs its operation at once or asks for parameters first. Items that need parameters end in an ellipsis, as the macOS convention asks. They set the session's `pendingRequest`, and the document window reacts by presenting the parameter sheet. The menu never shows user interface itself. The others call `perform`, which runs the operation in the background on the focused variant and adds the result as a new tab. Since the source is whichever tab is focused, operations chain naturally: blur a copy, then shrink the blurred copy.

The filters are grouped as the people using them think of them: smoothing, then sharpening and edge detection, then point operations on the colors.

199 *⟨Commands: Filter menu 199⟩* ≡

```

CommandMenu("Filter") {
  Group {
    Button("Gaussian ...Blur") { session?.pendingRequest = .gaussianBlur }
    Button("Box ...Blur") { session?.pendingRequest = .boxBlur }
    Button("Median 3×3") { session?.perform(.filter(.median)) }
    Button("Noise ...Reduction") { session?.pendingRequest = .noiseReduction }
    Divider()
    Button("Sharpen (Unsharp Mask)...") { session?.pendingRequest = .unsharpMask }
    Button("Edge Detect (Sobel)") { session?.perform(.filter(.sobel)) }
    Button("Laplacian") { session?.perform(.filter(.laplacian)) }
    Button("Emboss") { session?.perform(.filter(.emboss)) }
    Divider()
    Button("Grayscale") { session?.perform(.filter(.grayscale)) }
    Button("Invert") { session?.perform(.filter(.invert)) }
  }
  .disabled(session == nil)
}

```

Used in chunk 195.

The Resample menu is generated from `ResampleMethod.allCases`. A method added to that enumeration therefore appears in the menu without any change here. Every method needs a scale factor, so every item opens the sheet.

200 *⟨Commands: Resample menu 200⟩* ≡

```

CommandMenu("Resample") {
  ForEach(ResampleMethod.allCases) { method in
    Button("\(method.displayName)...") { session?.pendingRequest = .resample(method) }
  }
  .disabled(session == nil)
}

```

Used in chunk 195.

10.4 The View menu

The View menu repeats the split and unsplit buttons of the zone headers, so they have menu items for keyboard users. It also holds the display option “Actual Device Pixels”. That option lives in the session, so the menu adapts it with a `Binding` whose getter and setter go through the optional session. The inspector gets the shortcut Command-Option-I, which is conventional for inspectors on macOS.

201 *⟨Commands: View menu 201⟩* ≡

```

CommandGroup(before: .toolbar) {
  ForEach(Zone.allCases) { zone in
    Button("Split \(zone.title)") { session?.split(zone) }
    .disabled(session == nil)
  }
  ForEach(Zone.allCases) { zone in
    Button("Unsplit \(zone.title)") { session?.unsplit(zone) }
    .disabled((session?.panes[zone.rawValue].count ?? 1) < 2)
  }
  Divider()
  Toggle("Actual Device Pixels", isOn: Binding(
    get: { session?.useDevicePixels ?? false },
    set: { session?.useDevicePixels = $0 }
  ))
  .disabled(session == nil)
  Button("Reset Line") { session?.line = .initial }
  .disabled(session == nil)
  Divider()
}

```

```
Button("Show Inspector") { openWindow(id: InspectorView.windowID) }  
    .keyboardShortcut("i", modifiers: [.command, .option])  
Divider()  
}
```

Used in chunk 195.

Bibliography

- [1] Donald E. Knuth. Literate Programming. *The Computer Journal* 27(2):97–111, 1984.
- [2] Norman Ramsey. Literate Programming Simplified. *IEEE Software* 11(5):97–105, 1994.
- [3] Don P. Mitchell and Arun N. Netravali. Reconstruction Filters in Computer Graphics. *Computer Graphics (SIGGRAPH '88)* 22(4):221–228, 1988.
- [4] Robert G. Keys. Cubic Convolution Interpolation for Digital Image Processing. *IEEE Transactions on Acoustics, Speech, and Signal Processing* 29(6):1153–1160, 1981.
- [5] Fredric J. Harris. On the Use of Windows for Harmonic Analysis with the Discrete Fourier Transform. *Proceedings of the IEEE* 66(1):51–83, 1978.
- [6] XcodeGen. <https://github.com/yonaskolb/XcodeGen>.

Index of Chunks

Every chunk name, the chunks that define it, and the chunks that use it. Root chunks are tangled to the file named by the chunk.

- ⟨*Charts.swift: the profile chart* 164⟩
Defined in 164. Used in 163.
- ⟨*Charts.swift: the spectrum chart* 166⟩
Defined in 166. Used in 163.
- ⟨*ColorSpaces: channel extraction* 56⟩
Defined in 56. Used in 52.
- ⟨*ColorSpaces: color model* 51⟩
Defined in 51. Used in 50.
- ⟨*ColorSpaces: conversions* 52⟩
Defined in 52. Used in 50.
- ⟨*ColorSpaces: HSL to RGB* 54⟩
Defined in 54. Used in 52.
- ⟨*ColorSpaces: RGB to CMYK and back* 55⟩
Defined in 55. Used in 52.
- ⟨*ColorSpaces: RGB to HSL* 53⟩
Defined in 53. Used in 52.
- ⟨*Commands: File menu, export and close* 198⟩
Defined in 198. Used in 195.
- ⟨*Commands: File menu, test patterns* 197⟩
Defined in 197. Used in 195.
- ⟨*Commands: Filter menu* 199⟩
Defined in 199. Used in 195.
- ⟨*Commands: finding the target session* 196⟩
Defined in 196. Used in 195.
- ⟨*Commands: Resample menu* 200⟩
Defined in 200. Used in 195.
- ⟨*Commands: View menu* 201⟩
Defined in 201. Used in 195.
- ⟨*Filters: 3x3 convolution* 38⟩
Defined in 38. Used in 29.
- ⟨*Filters: Core Image cases* 33⟩
Defined in 33. Used in 32.
- ⟨*Filters: CPU cases* 34⟩
Defined in 34. Used in 32.
- ⟨*Filters: dispatching a filter* 32⟩
Defined in 32. Used in 29.

- ⟨*Filters: number formatting for names* 31⟩
Defined in 31. Used in 29.
- ⟨*Filters: per-pixel color maps* 37⟩
Defined in 37. Used in 29.
- ⟨*Filters: running a Core Image filter on a buffer* 36⟩
Defined in 36. Used in 29.
- ⟨*Filters: Sobel gradient magnitude* 39⟩
Defined in 39. Used in 29.
- ⟨*Filters: the `FilterKind` enumeration* 30⟩
Defined in 30. Used in 29.
- ⟨*Filters: the Core Image context* 35⟩
Defined in 35. Used in 29.
- ⟨*Generator window: changing the pattern kind* 193⟩
Defined in 193. Used in 186.
- ⟨*Generator window: parameter field* 189⟩
Defined in 189. Used in 186.
- ⟨*Generator window: pattern picker and description* 187⟩
Defined in 187. Used in 186.
- ⟨*Generator window: rendering the preview* 191⟩
Defined in 191. Used in 186.
- ⟨*Generator window: size fields* 188⟩
Defined in 188. Used in 186.
- ⟨*Generator window: the create button* 190⟩
Defined in 190. Used in 186.
- ⟨*Generator window: the preview box* 192⟩
Defined in 192. Used in 186.
- ⟨*Generator window: validation and clamping* 194⟩
Defined in 194. Used in 186.
- ⟨*ImageCanvas.swift: the line overlay* 162⟩
Defined in 162. Used in 153.
- ⟨*ImageCanvas: coordinate conversions* 161⟩
Defined in 161. Used in 153.
- ⟨*ImageCanvas: drawing the image at true scale* 155⟩
Defined in 155. Used in 153.
- ⟨*ImageCanvas: hit testing the line* 157⟩
Defined in 157. Used in 153.
- ⟨*ImageCanvas: moving the whole segment without leaving the image* 159⟩
Defined in 159. Used in 158.
- ⟨*ImageCanvas: Shift snapping* 160⟩
Defined in 160. Used in 153.
- ⟨*ImageCanvas: the line gesture* 156⟩
Defined in 156. Used in 153.
- ⟨*ImageCanvas: the state of a drag* 154⟩
Defined in 154. Used in 153.
- ⟨*ImageCanvas: updating the line during a drag* 158⟩
Defined in 158. Used in 153.
- ⟨*ImageDocument: a generated test pattern* 105⟩
Defined in 105. Used in 101.

- ⟨*ImageDocument: decoding the first image* 103⟩
Defined in 103. Used in 102.
- ⟨*ImageDocument: reading a file* 102⟩
Defined in 102. Used in 101.
- ⟨*ImageDocument: recording facts about the source* 104⟩
Defined in 104. Used in 102.
- ⟨*ImageDocument: refusing to write* 106⟩
Defined in 106. Used in 101.
- ⟨*ImageDocument: source information* 100⟩
Defined in 100. Used in 99.
- ⟨*ImageDocument: the document type* 101⟩
Defined in 101. Used in 99.
- ⟨*ImageOperation: applying the step* 42⟩
Defined in 42. Used in 40.
- ⟨*ImageOperation: display name* 41⟩
Defined in 41. Used in 40.
- ⟨*ImageVariant: operation requests* 112⟩
Defined in 112. Used in 107.
- ⟨*ImageVariant: panes* 110⟩
Defined in 110. Used in 107.
- ⟨*ImageVariant: the frequency axis* 111⟩
Defined in 111. Used in 107.
- ⟨*ImageVariant: the variant type* 108⟩
Defined in 108. Used in 107.
- ⟨*ImageVariant: zones* 109⟩
Defined in 109. Used in 107.
- ⟨*InspectorContent: row and number helpers* 183⟩
Defined in 183. Used in 177.
- ⟨*InspectorContent: the channel-statistics section* 182⟩
Defined in 182. Used in 177.
- ⟨*InspectorContent: the file section* 178⟩
Defined in 178. Used in 177.
- ⟨*InspectorContent: the focused-image section* 179⟩
Defined in 179. Used in 177.
- ⟨*InspectorContent: the line section* 181⟩
Defined in 181. Used in 177.
- ⟨*InspectorContent: the operation-chain section* 180⟩
Defined in 180. Used in 177.
- ⟨*InspectorView.swift: the inspector's content* 177⟩
Defined in 177. Used in 176.
- ⟨*LineSampler: bilinear interpolation* 49⟩
Defined in 49. Used in 47.
- ⟨*LineSampler: line segment* 45⟩
Defined in 45. Used in 43.
- ⟨*LineSampler: normalized point* 44⟩
Defined in 44. Used in 43.
- ⟨*LineSampler: sample along the line* 48⟩
Defined in 48. Used in 47.

- ⟨*LineSampler: sample record* 46⟩
Defined in 46. Used in 43.
- ⟨*LineSampler: sampler* 47⟩
Defined in 47. Used in 43.
- ⟨*LineScope/Commands/LineScopeCommands.swift* 195⟩
Defined in 195. Used in root.
- ⟨*LineScope/Document/ImageDocument.swift* 99⟩
Defined in 99. Used in root.
- ⟨*LineScope/LineScopeApp.swift* 94⟩
Defined in 94. Used in root.
- ⟨*LineScope/Model/ImageVariant.swift* 107⟩
Defined in 107. Used in root.
- ⟨*LineScope/Model/Session.swift* 113⟩
Defined in 113. Used in root.
- ⟨*LineScope/Views/Charts.swift* 163⟩
Defined in 163. Used in root.
- ⟨*LineScope/Views/ImageCanvas.swift* 153⟩
Defined in 153. Used in root.
- ⟨*LineScope/Views/InspectorView.swift* 176⟩
Defined in 176. Used in root.
- ⟨*LineScope/Views/MainView.swift* 131⟩
Defined in 131. Used in root.
- ⟨*LineScope/Views/PaneView.swift* 147⟩
Defined in 147. Used in root.
- ⟨*LineScope/Views/ParameterSheet.swift* 169⟩
Defined in 169. Used in root.
- ⟨*LineScope/Views/TestPatternViews.swift* 184⟩
Defined in 184. Used in root.
- ⟨*LineScope/Views/ZoneView.swift* 138⟩
Defined in 138. Used in root.
- ⟨*LineScopeApp: document windows* 95⟩
Defined in 95. Used in 94.
- ⟨*LineScopeApp: inspector window* 98⟩
Defined in 98. Used in 94.
- ⟨*LineScopeApp: pattern generator window* 97⟩
Defined in 97. Used in 94.
- ⟨*LineScopeApp: test-pattern windows* 96⟩
Defined in 96. Used in 94.
- ⟨*MainView.swift: the window observer* 136⟩
Defined in 136. Used in 131.
- ⟨*MainView: publishing the session to menus and inspector* 134⟩
Defined in 134. Used in 131.
- ⟨*MainView: sheets, alerts and the progress indicator* 135⟩
Defined in 135. Used in 131.
- ⟨*MainView: the session is created once per window* 132⟩
Defined in 132. Used in 131.
- ⟨*MainView: three zones stacked vertically* 133⟩
Defined in 133. Used in 131.

- ⟨*Packages/LineScopeCore/Package.swift* 2⟩
Defined in 2. Used in root.
- ⟨*Packages/LineScopeCore/Sources/LineScopeCore/ColorSpaces.swift* 50⟩
Defined in 50. Used in root.
- ⟨*Packages/LineScopeCore/Sources/LineScopeCore/Filters.swift* 29⟩
Defined in 29. Used in root.
- ⟨*Packages/LineScopeCore/Sources/LineScopeCore/ImageOperation.swift* 40⟩
Defined in 40. Used in root.
- ⟨*Packages/LineScopeCore/Sources/LineScopeCore/LineSampler.swift* 43⟩
Defined in 43. Used in root.
- ⟨*Packages/LineScopeCore/Sources/LineScopeCore/PixelBuffer.swift* 3⟩
Defined in 3. Used in root.
- ⟨*Packages/LineScopeCore/Sources/LineScopeCore/Resampling.swift* 12⟩
Defined in 12. Used in root.
- ⟨*Packages/LineScopeCore/Sources/LineScopeCore/Spectrum.swift* 57⟩
Defined in 57. Used in root.
- ⟨*Packages/LineScopeCore/Sources/LineScopeCore/TestPattern.swift* 67⟩
Defined in 67. Used in root.
- ⟨*Packages/LineScopeCore/Tests/LineScopeCoreTests/LineScopeCoreTests.swift* 82⟩
Defined in 82. Used in root.
- ⟨*PaneView.swift: the tab strip* 150⟩
Defined in 150. Used in 147.
- ⟨*PaneView: tab strip above the zone's content* 149⟩
Defined in 149. Used in 147.
- ⟨*PaneView: which variant this pane shows* 148⟩
Defined in 148. Used in 147.
- ⟨*ParameterSheet: a numeric field* 173⟩
Defined in 173. Used in 169.
- ⟨*ParameterSheet: building the operation* 175⟩
Defined in 175. Used in 169.
- ⟨*ParameterSheet: fields for each request* 172⟩
Defined in 172. Used in 169.
- ⟨*ParameterSheet: layout* 171⟩
Defined in 171. Used in 169.
- ⟨*ParameterSheet: parameters and their defaults* 170⟩
Defined in 170. Used in 169.
- ⟨*ParameterSheet: validating a resample* 174⟩
Defined in 174. Used in 169.
- ⟨*PixelBuffer: decoding a CGImage* 10⟩
Defined in 10. Used in 3.
- ⟨*PixelBuffer: encoding a CGImage* 11⟩
Defined in 11. Used in 3.
- ⟨*PixelBuffer: pixel subscript* 7⟩
Defined in 7. Used in 5.
- ⟨*PixelBuffer: premultiplication* 8⟩
Defined in 8. Used in 5.
- ⟨*PixelBuffer: stored properties and initializers* 6⟩
Defined in 6. Used in 5.

- ⟨*PixelBuffer: the buffer type* 5⟩
Defined in 5. Used in 3.
- ⟨*PixelBuffer: the color sample RGBA* 4⟩
Defined in 4. Used in 3.
- ⟨*PixelBuffer: the sRGB color space* 9⟩
Defined in 9. Used in 3.
- ⟨*ProfileChart: a color for each channel* 165⟩
Defined in 165. Used in 164.
- ⟨*project.yml* 1⟩
Defined in 1. Used in root.
- ⟨*Resampler: area-average contribution* 22⟩
Defined in 22. Used in 20.
- ⟨*Resampler: contributions of input pixels* 20⟩
Defined in 20. Used in 14.
- ⟨*Resampler: horizontal pass* 18⟩
Defined in 18. Used in 17.
- ⟨*Resampler: kernel contribution* 23⟩
Defined in 23. Used in 20.
- ⟨*Resampler: kernel evaluation* 28⟩
Defined in 28. Used in 14.
- ⟨*Resampler: kernel supports* 25⟩
Defined in 25. Used in 14.
- ⟨*Resampler: nearest-neighbor contribution* 21⟩
Defined in 21. Used in 20.
- ⟨*Resampler: resampling by a scale factor* 16⟩
Defined in 16. Used in 14.
- ⟨*Resampler: resampling to a given size* 17⟩
Defined in 17. Used in 14.
- ⟨*Resampler: sinc radius and output size* 15⟩
Defined in 15. Used in 14.
- ⟨*Resampler: the BC-spline family* 27⟩
Defined in 27. Used in 14.
- ⟨*Resampler: the sinc function* 26⟩
Defined in 26. Used in 14.
- ⟨*Resampler: trimming and normalizing weights* 24⟩
Defined in 24. Used in 14.
- ⟨*Resampler: vertical pass* 19⟩
Defined in 19. Used in 17.
- ⟨*Resampling: the ResampleMethod enumeration* 13⟩
Defined in 13. Used in 12.
- ⟨*Resampling: the Resampler namespace* 14⟩
Defined in 14. Used in 12.
- ⟨*Session: analysis settings* 118⟩
Defined in 118. Used in 116.
- ⟨*Session: closing a variant* 123⟩
Defined in 123. Used in 115.
- ⟨*Session: computing a profile* 126⟩
Defined in 126. Used in 115.

- ⟨*Session: computing a spectrum* 127⟩
Defined in 127. Used in 115.
- ⟨*Session: exporting the focused variant* 128⟩
Defined in 128. Used in 115.
- ⟨*Session: identity, variants and the line* 117⟩
Defined in 117. Used in 116.
- ⟨*Session: initialization* 120⟩
Defined in 120. Used in 115.
- ⟨*Session: looking up variants* 121⟩
Defined in 121. Used in 115.
- ⟨*Session: panes, focus and pending work* 119⟩
Defined in 119. Used in 116.
- ⟨*Session: performing an operation in the background* 124⟩
Defined in 124. Used in 115.
- ⟨*Session: profile and spectrum point types* 114⟩
Defined in 114. Used in 113.
- ⟨*Session: selecting and splitting panes* 122⟩
Defined in 122. Used in 115.
- ⟨*Session: showing a new variant* 125⟩
Defined in 125. Used in 115.
- ⟨*Session: stored state* 116⟩
Defined in 116. Used in 115.
- ⟨*Session: the active session* 130⟩
Defined in 130. Used in 113.
- ⟨*Session: the focused-value entry* 129⟩
Defined in 129. Used in 113.
- ⟨*Session: the session class* 115⟩
Defined in 115. Used in 113.
- ⟨*Spectrum: analyzer* 60⟩
Defined in 60. Used in 57.
- ⟨*Spectrum: choose the FFT size* 61⟩
Defined in 61. Used in 60.
- ⟨*Spectrum: condition the signal* 62⟩
Defined in 62. Used in 60.
- ⟨*Spectrum: frequency axes* 66⟩
Defined in 66. Used in 60.
- ⟨*Spectrum: packed real FFT* 64⟩
Defined in 64. Used in 60.
- ⟨*Spectrum: result* 59⟩
Defined in 59. Used in 57.
- ⟨*Spectrum: single-sided amplitudes* 65⟩
Defined in 65. Used in 60.
- ⟨*Spectrum: window functions* 58⟩
Defined in 58. Used in 57.
- ⟨*Spectrum: zero-pad* 63⟩
Defined in 63. Used in 60.
- ⟨*SpectrumChart: the amplitude range* 168⟩
Defined in 168. Used in 166.

- ⟨*SpectrumChart: the Nyquist frequency on the current axis* 167⟩
Defined in 167. Used in 166.
- ⟨*TabStrip: a tab's context menu* 152⟩
Defined in 152. Used in 150.
- ⟨*TabStrip: a tab's label* 151⟩
Defined in 151. Used in 150.
- ⟨*TestPattern: chirps and gratings* 77⟩
Defined in 77. Used in 74.
- ⟨*TestPattern: display names* 70⟩
Defined in 70. Used in 68.
- ⟨*TestPattern: edges and impulses* 78⟩
Defined in 78. Used in 74.
- ⟨*TestPattern: generator* 74⟩
Defined in 74. Used in 67.
- ⟨*TestPattern: gray-level helper* 76⟩
Defined in 76. Used in 74.
- ⟨*TestPattern: noise and color ramp* 80⟩
Defined in 80. Used in 74.
- ⟨*TestPattern: pattern kinds* 68⟩
Defined in 68. Used in 67.
- ⟨*TestPattern: set up the canvas* 75⟩
Defined in 75. Used in 74.
- ⟨*TestPattern: Siemens star* 79⟩
Defined in 79. Used in 74.
- ⟨*TestPattern: specification* 73⟩
Defined in 73. Used in 67.
- ⟨*TestPattern: SplitMix64* 81⟩
Defined in 81. Used in 67.
- ⟨*TestPattern: the cases* 69⟩
Defined in 69. Used in 68.
- ⟨*TestPattern: the tunable parameter* 72⟩
Defined in 72. Used in 68.
- ⟨*TestPattern: what each pattern is for* 71⟩
Defined in 71. Used in 68.
- ⟨*TestPatternViews.swift: the generator window* 186⟩
Defined in 186. Used in 184.
- ⟨*TestPatternViews.swift: the pattern analysis window* 185⟩
Defined in 185. Used in 184.
- ⟨*Tests: area average halves* 88⟩
Defined in 88. Used in 85.
- ⟨*Tests: color conversions* 90⟩
Defined in 90. Used in 82.
- ⟨*Tests: constants stay constant* 89⟩
Defined in 89. Used in 85.
- ⟨*Tests: Core Graphics bridge and filters* 92⟩
Defined in 92. Used in 82.
- ⟨*Tests: gradient fixture* 83⟩
Defined in 83. Used in 82.

- ⟨*Tests: identity at scale one* 87⟩
Defined in 87. Used in 85.
- ⟨*Tests: kernel shapes* 86⟩
Defined in 86. Used in 85.
- ⟨*Tests: line sampler* 91⟩
Defined in 91. Used in 82.
- ⟨*Tests: resampling* 85⟩
Defined in 85. Used in 82.
- ⟨*Tests: spectrum* 84⟩
Defined in 84. Used in 82.
- ⟨*Tests: test patterns* 93⟩
Defined in 93. Used in 82.
- ⟨*WindowKeyObserver: the observing NSView* 137⟩
Defined in 137. Used in 136.
- ⟨*ZoneView: controls specific to each zone* 141⟩
Defined in 141. Used in 138.
- ⟨*ZoneView: describing the line* 146⟩
Defined in 146. Used in 138.
- ⟨*ZoneView: header above side-by-side panes* 139⟩
Defined in 139. Used in 138.
- ⟨*ZoneView: image-zone controls* 142⟩
Defined in 142. Used in 141.
- ⟨*ZoneView: profile-zone controls* 143⟩
Defined in 143. Used in 141.
- ⟨*ZoneView: spectrum-zone controls* 144⟩
Defined in 144. Used in 141.
- ⟨*ZoneView: split and unsplit buttons* 145⟩
Defined in 145. Used in 138.
- ⟨*ZoneView: the header row* 140⟩
Defined in 140. Used in 138.